FUJINET FOR THE INTELLIVISION

PROGRAMMER'S GUIDE

IntyBASIC Edition

Covers the cartridge mailbox, the Network and Fuji devices, AppKeys, and three complete networked games with their servers.

FUJINET PROGRAMMER'S GUIDE

for the Mattel Intellivision — IntyBASIC Edition

Every command, register cell, byte offset and wire format in this guide was verified against the sources it documents: the cartridge firmware in `fujinet-firmware/pico/intellivision/`, the ESP32-S3 command handlers in `lib/device/rs232/` and `lib/bus/rs232/`, the CONFIG client in `fujinet-config/intv/`, the three game clients, and the game servers in `fujinet-game-system/`.

The example programs `hello.bas` and `netcat.bas` compile with IntyBASIC v1.4.2 and assemble with as1600 without errors.

The network is as easy as PEEK and POKE.

TABLE OF CONTENTS

CHAPTER 1	Introduction	1
CHAPTER 2	The Cartridge Mailbox	3
CHAPTER 3	Your First Transaction	7
CHAPTER 4	The Network Device	11
CHAPTER 5	The Fuji Device	18
CHAPTER 6	The Clock Device	28
CHAPTER 7	The fujinet.bas Library	30
CHAPTER 8	Game One: 5 Card Stud	33
CHAPTER 9	Game Two: Battleship	37
CHAPTER 10	Game Three: Fujitzee	40
CHAPTER 11	The Lobby	43
APPENDIX A	Error Codes	44
APPENDIX B	Quick Reference	47
APPENDIX C	Program Listings	51
APPENDIX D	Netcat	111

CHAPTER 1

INTRODUCTION

What a FujiNet is, what is inside the cartridge, and what your IntyBASIC program can do with it.

FujiNet brings the Internet to the Intellivision the same way it came to the Atari, the ADAM, the Apple II and a dozen other classic machines: as a peripheral that does the hard part — WiFi, TCP/IP, TLS, HTTP, JSON — on a modern processor, and hands your 8-bit (well, 16-bit!) program a simple, byte-oriented command interface.

On most FujiNet platforms that interface is the machine's own peripheral bus. The Intellivision has no such bus for cartridges to speak on, so the Intellivision FujiNet takes a different road: **the cartridge itself is the peripheral.** Inside the cartridge are two computers:

The RP2040 (or RP2350) runs the cartridge: it serves the ROM your program executes from, and it maps a small window of cartridge RAM at addresses `$9C00`–`$9F3F` that both sides can read and write. That window is the **mailbox**. Your program fills in a request — a device number, a command number, parameters, maybe a payload — and rings a bell. The RP2040 notices, re-encodes the request as a FujiBus packet, and sends it over an internal USB serial link to the ESP32-S3, which is running the same fujinet-firmware that powers every other FujiNet. The reply comes back the same way and lands in the mailbox, where your program PEEKs it out.

The beautiful consequence: **everything is PEEK and POKE.** No handlers, no interrupts, no bus timing. If you can poke a byte and wait for another byte to change, you can talk to the Internet.

WHAT YOU CAN DO

- Open network connections by name — `N:HTTPS://server/path`, `N:TCP://host:port/`, TELNET, UDP, TNFS and more — then read, write and poll them (Chapter 4).
- Ask the Fuji control device to scan WiFi, manage host and device slots, browse server directories, mount disk images — and on the Intellivision, **boot another ROM over the network** (Chapter 5).
- Store small records — a player name, a saved game, a server URL — on the FujiNet itself with AppKeys, so they survive power-off (Chapter 5).
- Read the real-world clock in seven formats (Chapter 6).
- Encode and decode Base64, compute MD5/SHA hashes, even build QR codes, using the FujiNet as a coprocessor (Chapter 5).

HOW THIS GUIDE IS ARRANGED

Chapters 2 and 3 teach the mailbox itself — the memory map, the handshake, and a first working program. Chapters 4 through 6 are the command reference, one device at a time. Chapter 7 documents `fujinet.bas`, the drop-in IntyBASIC library used by every program in this guide. Chapters 8 through 10 dissect three complete, playable network games — **5 Card Stud**, **Battleship** and **Fujitzee** — client and server both, down to the byte. Chapter 11 covers the Lobby that ties them together. The appendices hold the error tables, a quick reference, the full program listings (with numbered call-outs (1) referenced from the chapters), and a network terminal you can type in.

> **NOTE:** Numbers written like `$9C00` are hexadecimal; plain numbers are decimal. Mailbox cells are named `FN_` the way `fujinet.bas` names them. Controller keys look like ENTER.

CHAPTER 2

THE CARTRIDGE MAILBOX

One page of shared RAM is the whole hardware interface. Learn it and you know everything the cartridge can do.

The mailbox is a window of cartridge RAM the RP2040 maps into the Intellivision's address space at `$9C00`–`$9F3F` — 832 locations. Each location holds one byte in the low 8 bits of a 16-bit word (writes from the Intellivision side are hardware-truncated to 8 bits), so always mask what you read: `PEEK(addr) AND 255`.

It sits at the **top** of the `$8000`–`$9FFF` region deliberately: on a JLP-enabled cartridge that whole region is JLP RAM, and parking the mailbox at the top leaves `$8000`–`$9BFF` — 7168 words, 87% of the window — free for JLP while the mailbox keeps its 832.

THE MEMORY MAP

ADDRESS	NAME	WHO WRITES	MEANING
`$9C00`	`MAGIC0`	RP2040	ASCII `F` (70) — mailbox present
`$9C01`	`MAGIC1`	RP2040	ASCII `N` (78)
`$9C02`	`PROTO_VER`	RP2040	protocol version, currently 1
`$9C03`	`SEQ`	Inty	bump to start a transaction (wraps, skip 0)
`$9C04`	`ACKSEQ`	RP2040	set equal to SEQ when the reply is ready
`$9C05`	`DEVICE`	Inty	device id (`$70` Fuji, `$71` N1: …)
`$9C06`	`CMD`	Inty	command id
`$9C07`	`NPARAM`	Inty	parameter count, 0–8
`$9C08/09`	`TXLEN`	Inty	payload length, little-endian
`$9C0A`	`STATUS`	RP2040	diagnostic only: 0 idle, 1 busy, 2 ok, 3 err
`$9C0B`	`ERR`	RP2040	link result of last transaction (Appendix A)

ADDRESS	NAME	WHO WRITES	MEANING
$9C0C/0D	RXLEN	RP2040	reply payload length, little-endian
$9C0E	REPLY_CMD	RP2040	$06 ACK or $15 NAK
$9C0F	LINK	RP2040	1 if the ESP32-S3 is up on the USB link
$9C10-17	PARAM_SIZE	Inty	size of each parameter: 1, 2 or 4 bytes
$9C18	BOOT_STATE	RP2040	network-boot progress (see below)
$9C19	BOOT_PCT	RP2040	network-boot progress, 0–100
$9C1A	BOOT_ERR	RP2040	reason, when BOOT_STATE = failed
$9C1B	DOORBELL	Inty	write $B5 → RP2040 reboots to BOOTSEL
$9C20-3F	PARAM_VAL	Inty	8 slots × 4 bytes, little-endian
$9C40-	TX	Inty	request payload, up to 256 bytes
$9D40-	RX	RP2040	reply payload, up to 512 bytes

THE HANDSHAKE

A transaction is four steps, and the order of the last one is the whole protocol:

1. Fill in DEVICE, CMD, NPARAM, the parameter table, TXLEN and the TX payload — everything **except** SEQ.
2. Read ACKSEQ, add 1 (wrap past 255, and skip 0), and write that to SEQ. **Last.** The RP2040 polls for SEQ ≠ ACKSEQ writing SEQ is what submits the request.
3. Wait for ACKSEQ to become equal to the SEQ you wrote. Give up after a generous timeout — the library allows 900 frames (15 seconds).
4. Check REPLY_CMD: $06 is ACK — the reply payload, if any, is at RX with its length in RXLEN. Anything else, look at ERR for the link-level reason (Appendix A).

WATCH OUT: Always derive the new SEQ from the `ACKSEQ` the RP2040 itself publishes, never from a variable of your own. Pressing Reset restarts your program and zeroes your variables — but not the RP2040. A locally-counted SEQ would recompute the same value it already used, the RP2040 would see nothing new, and your first transaction after every reset would hang. This is call-out (2) in Listing 1.

The interlock is a **sequence number**, not a busy flag: re-poking the same SEQ while a transaction is in flight is a no-op, so a nervous retry can never make the RP2040 run a command twice. `STATUS` at `$9C0A` is a debugging aid only — poll `ACKSEQ`, not STATUS.

PARAMETERS AND PAYLOADS

Commands take up to eight small numeric parameters — the things other FujiNet platforms pass in "aux" bytes. Parameter `i` occupies one word at `PARAM_SIZE + i` giving its size (1, 2 or 4 bytes) and up to four words at `PARAM_VAL + i×4` holding its value, little-endian, one byte per word. Bulk data — a URL, a filename, a WiFi password — goes in the TX window instead, as consecutive bytes, with the count in `TXLEN`.

Replies land in the RX window, length in `RXLEN`, and **stay there until the next transaction overwrites them**. The IntyBASIC discipline that follows from this is the most important habit in this guide: **read replies in place.** IntyBASIC gives you 228 eight-bit variables; a / state reply from a game server is 400+ bytes. Never copy a reply into variables — PEEK what you need straight out of `FN_RX`, and copy only the few bytes that must survive the next transaction into cartridge scratch RAM (`$9000`–`$9BFF` is free on a FujiNet cartridge when JLP is off).

TIMEOUTS AND THE LINK BYTE

The RP2040 gives an ordinary command 5 seconds on the USB link, and `MOUNT_IMAGE` 60 (a ROM can take a while to fetch). It also waits up to 3 seconds for the ESP32 to enumerate before the very first command — the Intellivision boots much faster than the ESP32 does. `LINK` at `$9C0F` shows the live state of the USB link; a program can show "waiting for FujiNet" instead of a mysterious timeout.

BOOTING A ROM THROUGH THE MAILBOX

`MOUNT_IMAGE` on the Fuji device does something on this platform it does nowhere else: the ESP32 pushes the selected ROM — plus its `.cfg` sibling, if one exists — back over the same USB link, the RP2040 decodes it straight into cartridge ROM space, remaps, and resets your console into the new program. While that happens the ordinary handshake is still outstanding, so progress is published out-of-band in three cells your wait loop can PEEK: `BOOT_STATE` (0 idle, 1 opening, 2 transferring, 3 mapping, `$80` failed), `BOOT_PCT` (0–100), and `BOOT_ERR` (Appendix A). Flat `.bin`, self-describing Intellicart `.rom`, and `.bin+.cfg` (with JLP variables) all load; a mapping that would collide with the mailbox itself is refused.

DECLARING THE WINDOW IN INTYBASIC

ASM MEMATTR $8000, $9BFF, "+RWN"

Tells the assembler that the cartridge RAM below the mailbox is readable and writable, so POKE reaches your scratch buffers.

Declare `$8000`–`$9BFF` — and stop there. On real hardware the RP2040 maps the whole window regardless of what your `.cfg` says, but jzIntv's `--fujinet` emulation registers its own handler for `$9C00`–`$9FFF`, and a cartridge that claims those addresses as plain RAM **shadows the emulated FujiNet** — the mailbox never comes up under emulation even though the same ROM works on the real cartridge. Call-out (8) in Listing 1.

CHAPTER 3

YOUR FIRST TRANSACTION

A complete program in under fifty lines, and the IntyBASIC survival rules that keep the rest of them working.

Here is the whole idea of the library in three procedures. First, wait for the mailbox to introduce itself:

```
GOSUB fn_wait_mailbox
```

> *Polls for the two magic bytes — "F", "N" — at $9C00 for up to three seconds. Sets fn_ok to 1 when the cartridge answers, 0 when it does not (this is not a FujiNet cartridge, or the mailbox is shadowed — see Chapter 2).*

Then describe a transaction and run it. Ask the Fuji device (`$70`) for its WiFi status (command `$FA`):

```
mb_dev = $70 : mb_cmd = $FA : mb_nparam = 0
#fn_txlen = 0
GOSUB fn_transact
```

> *Stages the request, bumps SEQ, blocks until ACKSEQ answers or 900 frames pass. On return fn_ok is 1 and the one-byte reply is waiting at FN_RX; a 3 means the WiFi is up.*

```
IF (PEEK(FN_RX) AND 255) = 3 THEN PRINT AT 40, "CONNECTED"
```

> *Reads the reply in place. The AND 255 mask matters — mailbox words carry a byte in their low half and whatever in their high half.*

That is genuinely all there is. `hello.bas` below is the complete program; type it in, or find it with the listings.

HELLO.BAS — ONE COMPLETE TRANSACTION

```
' hello.bas -- the smallest possible FujiNet program: ask the Fuji device
' for its WiFi status and report it. Demonstrates one complete mailbox
' transaction with no payload and no parameters.
'
```

```
' Build:  intybasic hello.bas hello.asm && as1600 -o hello -l hello.lst hello.asm
    GOTO main

    INCLUDE "fujinet.bas"

    CONST COL_WHITE = 7

main:
    MODE 0, 0, 0, 0, 0 : WAIT
    CLS
    PRINT AT 0 COLOR COL_WHITE, "FUJINET HELLO"
    PRINT AT 20, "WAITING FOR MAILBOX"

    GOSUB fn_wait_mailbox
    IF fn_ok = 0 THEN
        PRINT AT 40, "NO MAILBOX - IS THIS"
        PRINT AT 60, "A FUJINET CARTRIDGE?"
        GOTO halt
    END IF

    ' One transaction: device $70 (Fuji), command $FA (GET WIFI STATUS),
    ' no parameters, no payload. The one-byte reply lands in FN_RX.
    mb_dev = $70
    mb_cmd = $FA
    mb_nparam = 0
    #fn_txlen = 0
    GOSUB fn_transact

    IF fn_ok = 0 THEN
        PRINT AT 40, "TRANSACTION FAILED "
        PRINT AT 60, "FN ERR = "
        PRINT AT 69, <2>#mb_err
        GOTO halt
    END IF

    IF (PEEK(FN_RX) AND 255) = 3 THEN
        PRINT AT 40, "WIFI IS CONNECTED  "
    ELSE
        PRINT AT 40, "WIFI NOT CONNECTED "
    END IF
```

```
halt:
    WAIT
    GOTO halt
```

SURVIVAL RULES FOR INTYBASIC NETWORK CODE

Every rule below was paid for in debugging time on the three games. They are worth a page of anyone's attention.

JUMP OVER YOUR INCLUDES

An INCLUDE pastes its text where it stands, and falling into a `PROCEDURE` or `DATA` block by straight-line execution corrupts the return stack. Make line one of your program `GOTO main` (or `boot_start`), and put every INCLUDE between it and the label.

RESPECT THE MEMORY MAP

IntyBASIC compiles into `$5000`–`$6FFF` and keeps going if you outgrow it — straight into `$7000`, which is **not** in the manual's list of ranges usable on modern cartridge PCBs. Both Battleship and Fujitzee overflowed; the fix is an explicit `ASM ORG $D000` placed so the overflow lands in documented-safe `$C100`–`$FFFF` territory. Fujitzee's boot loader genuinely hung on real hardware from a 20-word spill that jzIntv forgave. Safe ranges: `$2000`–`$2FFF`, `$5000`–`$6FFF`, `$A000`–`$BFFF`, `$C100`–`$FFFF`.

PARENTHESIZE (PEEK(X) AND 255)

IntyBASIC's = binds **tighter** than `AND` — the same trap as C's `a & b == c`. An unparenthesized `DEF FN v = PEEK(x) AND 255` substituted into `IF v = 0` compiles as `PEEK(x) AND (255 = 0)`, which is always zero. Fujitzee's turn logic silently never ran until the generated assembly was read. Write `(PEEK(x) AND 255)`, always.

MASK INTO 16-BIT VARIABLES

IntyBASIC v1.4.2 silently drops the `AND 255` when the destination of `var = PEEK(...) AND 255` is an 8-bit variable — no ANDI is emitted at all, and high-byte garbage survives. #-prefixed 16-bit destinations compile correctly. The library's `#mb_err` and `#net_err` are 16-bit for exactly this reason.

NEST IFS INSTEAD OF CHAINING AND

Two equality comparisons joined by `AND`, where a side comes from a `DEF FN` containing its own `AND 255`, can fuse into code that is always false. Write nested single-condition IFs. (Fujitzee again; confirmed in the compiled listing.)

NUMBER FIELDS: <.N>, NOT <N>

`PRINT <3>` (zero-padded) can print garbage in its leading digit — its runtime entry point never initializes the pad register. `PRINT <.3>` (space-padded) initializes it. Use `<.n>` for any value that can need fewer digits than the field.

WAIT FOR KEY RELEASE

`CONT1.KEY` reads 12 when nothing is pressed. When a key opens a menu, wait for 12 before accepting input, or the same press instantly triggers the menu's own handler. And check `CONT1.B1/B2` **before** `CONT1.BUTTON` — BUTTON is a mask that is non-zero for all three action buttons.

CHAPTER 4

THE NETWORK DEVICE

Eight channels to anywhere: URLs in, bytes out. Devices $71 through $78.

The Network device turns a URL into a byte channel. Eight independent units — N1: through N8:, device ids $71 through $78 — can each hold one open connection. Everything in this guide uses N1:.

A connection is named by a **devicespec** — an ASCII string staged in the TX window when you OPEN:

```
N:PROTOCOL://HOST[:PORT]/PATH[?QUERY]
```

The protocols in current firmware: HTTP, HTTPS, TCP, UDP, TELNET, TNFS, FTP, SMB, NFS, SSH, plus a few specials. HTTPS certificates, redirects, name resolution — the ESP32 handles all of it, so you never see it.

THE LIFE OF A CONNECTION

1. `OPEN` with an access mode and translation mode.
2. `STATUS` until data is waiting (or the connection reports an error).
3. `READ` the bytes; `WRITE` your own.
4. `CLOSE`.

Two subtleties both games hit, worth learning before writing code. First: for an HTTP devicespec, **the actual request happens at the first STATUS**, not at OPEN. OPEN parses and stores; the first STATUS performs the transaction and its result code lands in the status reply's error byte. Second: STATUS reports bytes **available so far** — a response still arriving over WiFi grows between polls. The library's `api_call` polls until two consecutive STATUS reads agree (call-out 6, Listing 1) before trusting the count. An HTTP error page is still a readable body, so the byte count alone cannot tell success from a 400 — check the error byte, which reads 1 (SUCCESS) on a good channel (call-out 5).

COMMAND REFERENCE

OPEN

`DEV $71-78` `CMD $4F 'O'`

PARAM	SIZE	VALUE
0	1	access mode — table below
1	1	translation — 0 none, 1 CR, 2 LF, 3 CR LF, 4 PETSCII

PAYLOAD devicespec, ASCII, no terminator needed — TXLEN is the length

REPLY ACK on success; NAK if the protocol could not connect

Opens the channel. Re-opening an open unit closes it first. Translation rewrites line endings between the wire and your program; the games use 0 and handle bytes raw.

MODE	MEANING	PROTOCOLS
4	READ / HTTP GET	all
5	HTTP DELETE	HTTP(S)
6	DIRECTORY	filesystem protocols
8	WRITE / HTTP PUT	all
9	APPEND / HTTP DELETE with headers	file / HTTP(S)
12	READ-WRITE / HTTP GET with header access	sockets, HTTP(S)
13	HTTP POST	HTTP(S)
14	HTTP PUT with header access	HTTP(S)

NOTE: The games open with mode 12 — on HTTP it is a GET that also permits header interaction through the M command, and on TCP it is plain read-write. Mode 4 works for fire-and-forget GETs.

CLOSE

`DEV $71-78` `CMD $43 'C'`

Tears the connection down. Always close — even after an error — so the unit is free for the next OPEN. Closing writes a fresh (empty) reply length, so capture **RXLEN** from a READ before you CLOSE (the library does: call-out (4) territory in Listing 1).

READ

`DEV $71-78` `CMD $52 'R'`

PARAM	SIZE	VALUE
0	2	byte count to read

REPLY the bytes, in RX; RXLEN says how many arrived

Never ask for more than STATUS said was available — a short channel pads with NULs and flags an error. Ask STATUS first, clamp, then READ.

WRITE

`DEV $71-78` `CMD $57 'W'`

PARAM	SIZE	VALUE
0	2	byte count to send

PAYLOAD the bytes

Sends payload bytes out the channel. The count rides twice — as parameter 0 and as TXLEN — and both must agree.

STATUS

`DEV $71-78` `CMD $53 'S'`

PARAM	SIZE	VALUE
0	1	0
1	1	request type — 0 for channel status

REPLY 4 bytes: available-low, available-high, connected, error

The heartbeat of every network program. **available** is a 16-bit little-endian count of bytes ready to READ; **connected** is 1 while the far end holds the connection; **error** is 1 for success or a code from Appendix A. With no channel open, request types 1–4 return instead the adapter's IP address, netmask, gateway and DNS, 4 bytes each.

CHANNEL MODE

DEV $71-78 CMD $FC

PARAM	SIZE	VALUE
0	1	0
1	1	0 protocol (raw), 1 JSON, 2 SGML

Switches the open channel between raw bytes and a parsed view. In JSON mode, PARSE ingests the body and QUERY extracts values by path — see the worked sequence below.

PARSE

DEV $71-78 CMD $50 'P'

JSON/SGML mode: reads the entire response body off the protocol and parses it. Do this once, after STATUS says the body has arrived.

QUERY

DEV $71-78 CMD $51 'Q'

PAYLOAD a query path, e.g. `/0/name` — 256 bytes, NUL-padded

Selects a value out of the parsed document. After QUERY, STATUS reports the extracted value's length and READ returns it as text.

SET CHANNEL MODE (HTTP)

DEV $71-78 CMD $4D 'M'

PARAM	SIZE	VALUE
0	1	0
1	1	0 body, 1 collect headers, 2 get headers, 3 set headers, 4 set POST data

HTTP only: redirects READ/WRITE at the header machinery instead of the body — set request headers before the transaction, read response headers after.

TRANSLATION

DEV $71-78 CMD $54 'T'

PARAM	SIZE	VALUE
0	1	0
1	1	as OPEN parameter 1

Changes line-ending translation on an open channel.

SET EOL

DEV $71-78 CMD $4C 'L'

PARAM	SIZE	VALUE
0	1	first EOL byte, or 0 to restore the default
1	1	optional second EOL byte

Overrides what the translation layer treats as your machine's native line ending.

SEEK / TELL

DEV $71-78 CMD $25 / $26

PARAM	SIZE	VALUE
0	4	SEEK only: absolute byte offset

REPLY TELL: 4 bytes, little-endian, current offset

Repositions within a seekable resource. On HTTP this issues a Range request — valid for GET reads in body mode.

GETCWD / CHDIR

DEV $71-78 CMD $30 / $2C

PAYLOAD CHDIR: the new prefix/path

REPLY GETCWD: 256 bytes, the prefix

Maintains a working-directory prefix for filesystem protocols, so later devicespecs can be relative.

USERNAME / PASSWORD

DEV $71-78 CMD $FD / $FE

PAYLOAD the credential, up to 256 bytes

Stages login credentials before OPEN, for protocols that need them (SMB, FTP, SSH…).

TCP: ACCEPT / CLOSE CLIENT

DEV $71-78 CMD $41 / $63

A TCP devicespec with no host — `N:TCP://:6502/` — listens. `$41` accepts a waiting caller onto the channel; `$63` hangs up on the caller and resumes listening. Your Intellivision can be the server.

UDP: SET DESTINATION

DEV $71-78 CMD $44 'D'

PAYLOAD `host:port` for subsequent datagrams

UDP is connectionless; this sets where WRITEs go. (The companion `$72` GET-REMOTE query exists only in the PC build of the firmware, not on the ESP32.)

FILE OPS

`DEV $71-78` `CMD $20/$21/$23/$24/$2A/$2B`

PAYLOAD a devicespec (RENAME: `old,new`)

RENAME `$20`, DELETE `$21`, LOCK `$23`, UNLOCK `$24`, MKDIR `$2A`, RMDIR `$2B` — for protocols with a filesystem behind them (TNFS, SMB, FTP…).

A JSON ROUND TRIP

The game servers in this guide sidestep JSON with `?bin=1` binary replies (Chapter 8), but the parser is there when you want it:

```
' open, then: switch the channel to JSON
mb_cmd = $FC : mb_nparam = 2
pm_i = 0 : pm_size = 1 : #pm_val = 0 : GOSUB fn_param
pm_i = 1 : pm_size = 1 : #pm_val = 1 : GOSUB fn_param
GOSUB fn_transact
```

Channel mode 1 = JSON.

```
mb_cmd = $50 : mb_nparam = 0 : #fn_txlen = 0 : GOSUB fn_transact
```

PARSE — the FujiNet reads and digests the whole body.

```
' stage "/0/title" in FN_TX, NUL-padded, then:
mb_cmd = $51 : GOSUB fn_transact
```

QUERY selects one value by path.

```
GOSUB net_status : #net_readlen = #net_avail : GOSUB net_read
```

STATUS now reports the extracted value's length; READ fetches it as ASCII, ready to draw with draw_field.

CHAPTER 5

THE FUJI DEVICE

Device $70: the FujiNet's own control panel — WiFi, hosts, directories, disk images, AppKeys, and a bag of coprocessor tricks.

Everything that is **about the FujiNet itself** — rather than about one network connection — lives on device `$70`. CONFIG is nothing but a client of this device, and everything CONFIG does, your program can do.

WIFI

GET WIFI STATUS

DEV $70 CMD $FA

REPLY 1 byte: 3 = connected; other values are the ESP32 WL codes (0 idle, 1 no SSID, 4 connect failed, 5 connection lost, 6 disconnected)

The first thing `hello.bas` asks. Poll it after SET SSID — the join takes a few seconds.

SCAN NETWORKS

DEV $70 CMD $FD

REPLY 1 byte: number of networks found

Blocks while the ESP32 scans. Follow with GET SCAN RESULT for each index.

GET SCAN RESULT

DEV $70 CMD $FC

PARAM	SIZE	VALUE
0	1	network index, 0-based

REPLY 34 bytes: SSID (33, NUL-terminated) + RSSI (1, signed)

SET SSID

DEV $70 CMD $FB

PARAM	SIZE	VALUE
0	1	any value — at least one parameter must be present

PAYLOAD 97 bytes exactly: SSID (33) + password (64), NUL-padded

Joins and **saves** the network. The parameter is ignored but required. The 97-byte shape is not negotiable — see the payload rule below.

GET SSID

DEV $70 CMD $FE

REPLY 97 bytes: the stored SSID + password in the same shape

GET WIFI ENABLED

DEV $70 CMD $EA

REPLY 1 byte, 0/1

WATCH OUT: **The exact-length payload rule.** The ESP32 handler for a fixed-size payload fails the transaction if **fewer** bytes arrive than the structure it expects — SET SSID wants exactly 97, OPEN DIRECTORY and SET DEVICE FULLPATH want exactly 256. Sending a short, "obviously enough" payload reads back as a NAK. Pad with NULs to the documented size, every time.

HOSTS AND MOUNTS

Eight **host slots** name the file servers CONFIG browses — TNFS hosts, SMB shares. Eight **device slots** bind a file on some host to an emulated disk. On the Intellivision the disk story is what powers network booting.

READ HOST SLOTS

DEV $70 CMD $F4

REPLY 256 bytes: 8 hostnames × 32, NUL-padded

WRITE HOST SLOTS

DEV $70 CMD $F3

PAYLOAD the same 256-byte block, all 8 slots at once

There is no single-slot write — read, modify one, write all.

MOUNT HOST

DEV $70 CMD $F9

PARAM	SIZE	VALUE
0	1	host slot, 0–7

Connects to the named server. Required before directory or file operations on that slot.

SET HOST PREFIX / GET HOST PREFIX

DEV $70 CMD $E1 / $E0

PARAM	SIZE	VALUE
0	1	host slot

PAYLOAD SET: the prefix string

REPLY GET: the prefix

A per-host working directory.

OPEN DIRECTORY

DEV $70 CMD $F7

PARAM	SIZE	VALUE
0	1	host slot

PAYLOAD 256 bytes exactly: path, NUL, wildcard filter (may be empty), NUL, then NUL padding

Opens a directory listing on a mounted host. The filter understands * and ? only.

READ DIR ENTRY

DEV $70 CMD $F6

PARAM	SIZE	VALUE
0	1	maximum reply length
1	1	flags: 0 plain, $80 with details

REPLY plain: the filename, NUL-terminated — directories carry a trailing / end of directory is two $7F bytes. With $80: a 10-byte prefix (modified date 6, size 4 LE) plus flags and media type precede the name

CONFIG's file browser is this command in a loop.

CLOSE DIRECTORY

DEV $70 CMD $F5

GET / SET DIRECTORY POSITION

DEV $70 CMD $E5 / $E4

PARAM	SIZE	VALUE
0	2	SET: absolute entry index

REPLY GET: 2 bytes, little-endian

Random access into the listing — how CONFIG pages backward without rereading from the top.

READ / WRITE DEVICE SLOTS

DEV $70 CMD $F2 / $F1

PAYLOAD WRITE: 8 records of {host slot, mode, filename[36]} = 304 bytes

REPLY READ: the same 304-byte block

The table binding device slots to files.

SET DEVICE FULLPATH

DEV $70 CMD $E2

PARAM	SIZE	VALUE
0	1	device slot
1	1	host slot
2	1	mode: 1 read

PAYLOAD 256 bytes exactly: full path, NUL-padded

Points a device slot at a file. On the Intellivision this is also where the cartridge learns the filename it will derive a JLP flash save-file name from — CONFIG always sends it immediately before MOUNT IMAGE.

GET DEVICE FULLPATH

DEV $70 CMD $DA

PARAM	SIZE	VALUE
0	1	device slot

REPLY the path

MOUNT IMAGE

DEV $70 CMD $F8

PARAM	SIZE	VALUE
0	1	device slot
1	1	access flags: 1 read

On every FujiNet: mounts the image. On the Intellivision: **boots it** — the ESP32 pushes the ROM back through the cartridge as described in Chapter 2, and this transaction is allowed 60 seconds. Watch `BOOT_PCT` for a progress bar while you wait.

UNMOUNT IMAGE

`DEV $70` `CMD $E9`

PARAM	SIZE	VALUE
0	1	device slot

MOUNT ALL / NEW DISK / COPY FILE

`DEV $70` `CMD $D7 / $E7 / $D8`

Housekeeping from the wider FujiNet family: mount every configured slot; create a blank image ({sectors u16, sector size u16, host slot, device slot, filename[256]} payload); copy between hosts (parameters source and destination slot, payload the copy spec).

CONFIG BOOT / SET BOOT MODE

`DEV $70` `CMD $D9 / $D6`

PARAM	SIZE	VALUE
0	1	flag / mode

Controls whether the FujiNet offers CONFIG at power-up. The Intellivision cartridge boots CONFIG from its own flash, so CONFIG BOOT has no effect here — and CONFIG deliberately never sends it.

APPKEYS — SMALL FACTS THAT SURVIVE POWER-OFF

An AppKey is up to 64 bytes stored on the FujiNet, addressed by a registered **creator id** (16-bit), an **app id** and a **key id**. The games use them for the player name and the Lobby handoff (Chapter 11). The protocol is open–operate–close:

OPEN APPKEY

`DEV $70` `CMD $DC`

PAYLOAD 6 bytes: creator low, creator high, app, key, mode (0 read, 1 write, 2 read-256), reserved (0)

Selects which key the next read or write touches. Mode 2 selects 256-byte keys; plain mode 0 reads 64-byte keys. **All six bytes are required** — a 5-byte payload leaves the firmware waiting for the last byte and reads back as a timeout (call-out 7 neighborhood, Listing 1).

READ APPKEY

`DEV $70` `CMD $DD`

REPLY 2-byte little-endian length, then the data

The length prefix is specific to this bus (an appkey read takes no length parameter, so the reply describes itself). RXLEN covers prefix + data. Call-out 7 in Listing 1 shows the client-side handling.

WRITE APPKEY

`DEV $70` `CMD $DE`

PAYLOAD the value — length is TXLEN

CLOSE APPKEY

`DEV $70` `CMD $DB`

NOTE: Creator 1 / app 1 is the FujiNet game system. Key 0 holds the shared player name every lobby game reads; keys 1, 3 and 5 are the 5 Card Stud, Fujitzee and Battleship server-handoff slots. New creator ids are registered on the fujinet-firmware wiki's AppKey page.

ADAPTER INFORMATION

GET ADAPTERCONFIG EXTENDED

`DEV $70` `CMD $C4`

REPLY 240 bytes — table below. The older **$E8** form is the first 140 bytes only, without the printable strings

One call answers "what is my IP" for an About screen — CONFIG's info page is this struct drawn on screen.

OFFSET	SIZE	FIELD	NOTES
0	33	ssid	NUL-terminated
33	64	hostname	
97	4	localIP	binary, one octet per byte
101	4	gateway	
105	4	netmask	
109	4	dnsIP	
113	6	macAddress	binary
119	6	bssid	binary
125	15	fn_version	printable
140	16	sLocalIP	dotted-quad string
156	16	sGateway	
172	16	sNetmask	
188	16	sDnsIP	
204	18	sMacAddress	colon string
222	18	sBssid	

STATUS

`DEV $70` `CMD $53 'S'`

PARAM	SIZE	VALUE
0	1	1 = mount-time report

REPLY one timestamp per disk device — 0 means unmounted

RESET / DEVICE READY / GENERATE GUID

DEV $70 CMD $FF / $00 / $BB

RESET reboots the ESP32 (the mailbox link drops and comes back). DEVICE READY answers 512 bytes of A — a link self-test. GENERATE GUID returns a fresh unique id, handy for session tokens.

THE COPROCESSOR BAG

Three little engines share a buffer-in / compute / length / buffer-out shape. Feed input in chunks, compute, ask the length, then drain.

BASE64 ENCODE

DEV $70 CMD $D0/$CF/$CE/$CD

PARAM	SIZE	VALUE
0	2	INPUT and OUTPUT: byte count

PAYLOAD INPUT: the bytes

REPLY LENGTH: 4 bytes; OUTPUT: the requested bytes, consumed as read

INPUT $D0 accumulates, COMPUTE $CF transforms the buffer in place, LENGTH $CE, OUTPUT $CD. Decode is the mirror set $CC–$C9.

HASH

DEV $70 CMD $C8/$C7/$C6/$C5/$C2

PARAM	SIZE	VALUE
0	2	INPUT: byte count. COMPUTE: algorithm — 0 MD5, 1 SHA-1, 2 SHA-256, 3 SHA-512. LENGTH/OUTPUT: 1 = hex text, 0 = binary

PAYLOAD INPUT: the bytes

REPLY LENGTH: 1 byte; OUTPUT: the digest

`$C3` computes without clearing the input, for incremental use; CLEAR `$C2` abandons it. A SHA-256 of a game state in one transaction — try that on a CP-1610.

QR CODE

`DEV $70` `CMD $BC/$BD/$BE/$BF`

PARAM	SIZE	VALUE
0	2	INPUT: byte count. ENCODE: version. LENGTH/OUTPUT: output mode
1	1	ENCODE: error-correction level
2	1	ENCODE: shorten flag

PAYLOAD INPUT: the text

REPLY LENGTH: 4 bytes; OUTPUT: the module bitmap

Render the output on the colored-squares screen (the kernel in Chapter 9 draws 40×24 pixels) and you have a scannable screen.

CHAPTER 6

THE CLOCK DEVICE

Device $45: real time, seven ways, time-zone aware.

The FujiNet keeps real time via NTP. Device **$45** serves it in every format the wider FujiNet world has ever asked for. All GET commands accept an optional parameter 0: a value of 1 selects the **alternate** time zone set by **$74**, letting a program show a second clock without touching the saved configuration.

CMD	NAME	REPLY
`$93`	GET TIME	6 bytes: day, month, year-2000, hour, minute, second
`$9A`	GET TZ TIME	the same 6 bytes, always in the alternate zone
`$47 'G'`	GET SIMPLE	7 bytes: century, year, month, day, hour, minute, second
`$4D 'M'`	GET SIMPLE + HUNDREDTHS	8 bytes: simple + hundredths (0–99)
`$50 'P'`	GET PRODOS	4 bytes, ProDOS packed date/time
`$53 'S'`	GET SOS	NUL-terminated Apple SOS string
`$49 'I'`	GET ISO LOCAL	NUL-terminated ISO-8601 string, local
`$5A 'Z'`	GET ISO UTC	NUL-terminated ISO-8601 string, UTC
`$99`	SET TZ	— payload: POSIX TZ string, saved to config
`$74 't' /` `$54 'T'`	SET ALT TZ	— payload: POSIX TZ string, this session only
`$4C 'L'`	GET TZ LENGTH	1 byte: length of the saved TZ string
`$47`	GET TZ	the saved TZ string, NUL-terminated

```
mb_dev = $45 : mb_cmd = $93 : mb_nparam = 0
#fn_txlen = 0 : GOSUB fn_transact
day = PEEK(FN_RX) AND 255
```

Six PEEKs later your game has a real-world timestamp — for a tournament clock, a daily challenge seed, or just a clock on the title screen.

CHAPTER 7

THE FUJINET.BAS LIBRARY

Sixteen procedures, tested in three shipped games. INCLUDE it and go.

Listing 1 is the complete library — the exact transport all three game chapters build on, in its newest revision (the games in the field carry two vintages; this one folds in the compiler-bug fixes from Chapter 3 and adds `net_write` and `fn_putnum`). Include it after your opening GOTO:

```
    GOTO main
    INCLUDE "fujinet.bas"
main:
```

Remember: never fall into an include. Chapter 3's first survival rule.

TRANSPORT

FN_WAIT_MAILBOX

Waits up to 180 frames for the mailbox magic bytes. Sets `fn_ok`. Call once at boot, before anything else. (1)

FN_TRANSACT

The engine. In: `mb_dev`, `mb_cmd`, `mb_nparam`, `#fn_txlen` (payload already at `FN_TX`), parameters staged via `fn_param`. Out: `fn_ok` on failure `#mb_err` holds the link error (0 on timeout — the RP2040 never answered). Derives SEQ from ACKSEQ (2), polls up to 900 frames (3), and verifies the ACK (4).

FN_PARAM

Stages parameter `pm_i` (0-based) of size `pm_size` (1 or 2 bytes here; the mailbox allows 4) with value `#pm_val` into the parameter table.

FN_PUTSTR / FN_STRLEN

The string kit. `fn_putstr` appends `fn_len` bytes from address `#fn_src` — ROM DATA via VARPTR, or any RAM buffer — to the TX window, advancing `#fn_txlen`. `fn_strlen` measures a NUL-padded field first, so padded storage never leaks NULs into a URL (the wire sends TXLEN bytes verbatim; an embedded NUL corrupts the HTTP request downstream).

NETWORK SHORTHAND

NET_OPEN

Opens the devicespec staged at `FN_TX` for HTTP GET (mode 12, translation 0).

NET_STATUS

STATUS with the two zero parameters; unpacks available count into `#net_avail` and the error byte into `#net_err` [5] — and folds "error ≠ 1" into `fn_ok`, which is what catches an HTTP error page masquerading as a good response.

NET_READ

Reads `#net_readlen` bytes; the count that actually arrived lands in `#net_gotlen` — captured before CLOSE can overwrite RXLEN.

NET_WRITE

Sends `fn_len` bytes already at `FN_TX`. New in this guide's revision; the netcat in Appendix D is its first customer.

NET_CLOSE

Closes N1:. Unconditionally safe.

API_CALL

The one-shot HTTP GET: open → settle-poll STATUS [6] → clamp → read → close. Set `#net_readlen` to your reply buffer budget first.

This is the only network call the three games ever make.

APPKEYS

APPKEY_OPEN / APPKEY_READ / APPKEY_WRITE / APPKEY_CLOSE

Set `ak_creator_lo/hi`, `ak_app`, `ak_key`, `ak_mode` and open; then read into a buffer at `#fn_src` (bounded by `ls_max`, always NUL-terminated, actual length in `fn_len` — the 2-byte reply prefix is consumed for you [7]) or write `fn_len` bytes from `#fn_src` then close.

UTILITY

FN_PUTNUM

Appends `pn_val` (0–999) to the TX window as decimal ASCII — the missing `itoa` for building URLs like `/attack/47`.

> **NOTE:** The library owns scratch RAM `$9100`–`$917F` for three buffers: `SC_NAME` (player name, 9), `SC_ENDPT` (server endpoint, 64), `SC_QUERY` (48). Everything below `$9C00` and above your own use is yours to allocate; the games start theirs at `$9180`.

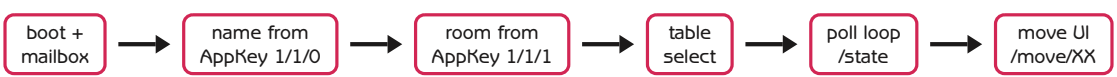
CHAPTER 8

GAME ONE: 5 CARD STUD

Eight seats, a felt table, and every bet a URL. The founding game of the FujiNet game system, on a 159×96 screen.

5 Card Stud (Listing 2) is the pattern the other two games copy: a thin IntyBASIC client that draws whatever a stateless-feeling HTTP server says the table looks like. The client never knows the rules of poker. It knows how to draw a Game structure, and how to ask again.

THE SHAPE OF THE CLIENT



At boot (Listing 2 ❶) the client reads the shared player name from AppKey creator 1 / app 1 / key 0 — **with one retry**, because the very first transaction after power-on can catch the USB link still waking — and validates every byte to A–Z 0–9. That slot is shared by every FujiNet game on every platform, so it can hold anything; anything outside the name-entry alphabet goes straight into a URL query string unescaped, and a stray & turns into a broken request the server answers with an HTTP error page. Fall back to the letter-picker if in doubt.

Then the Lobby handoff (Chapter 11, and ❷ in Listing 2): AppKey creator 1 / app 1 / key 1 may hold `https://host/?table=xyz` written by the Lobby client. If it parses (`split_room_url`), the client skips straight to that table; if not, it seeds the compiled-in default endpoint and shows table select.

URLS, COMPOSED BYTE BY BYTE

IntyBASIC has no strings, so URLs are assembled by `compose_url` (3) from ASCII `DATA` literals and scratch buffers, directly into the TX window:

```
N:https://5card.carr-designs.com/state?table=r1&player=THOM&bin=1
```

PATH	WHEN	REPLY
`tables?bin=1`	table select	Tables structure
`state?…&bin=1`	every poll	Game structure
`move/XX?…&bin=1`	your move (XX = 2-char code)	Game structure
`leave?…&bin=1`	quit table	`bye`

`?bin=1` asks the server for its binary serialization — fixed offsets, no JSON parser needed. Multi-byte numbers are little-endian. (There is a `&be=1` big-endian option; the Intellivision client tried it and retired it after pot values kept landing on suspicious multiples of 256 — the signature of a byte-order mismatch. Little-endian is the road the whole client family exercises daily. Stay on it.)

THE TABLES STRUCTURE

OFFSET	SIZE	FIELD	NOTES
`0`	`1`	count	tables that follow
`1+36i`	`9`	table id	NUL-padded, lowercased
`10+36i`	`21`	name	
`31+36i`	`6`	players	literal text like `2 / 8`

The client validates before trusting: reply length must cover `1 + count×36` bytes, or the "Tables structure" was actually an error page and the poll is retried (4).

THE GAME STRUCTURE

OFFSET	SIZE	FIELD	NOTES
0	81	lastResult	one-shot message, NUL-padded
81	1	round	0 wait, 1–4 streets, 5 showdown
82	2	pot	u16 LE
84	1	activePlayer	signed; $FF = none, 0 = you
85	1	moveTime	seconds left, server-computed
86	1	viewing	1 = you are a spectator
87	1	validMoveCount	
88	65	validMoves[5]	13 each: code[3] + name[10]
153	1	playerCount	
154	33×N	players[N]	name[9] status(1) bet(2 LE) move[8] purse(2 LE) hand[11]

Wire index 0 is **always you** — the server rotates the array per client. Hands are ASCII pairs (rank `2`–`9`, `t j q k a` suit `d h c s`); a hidden card is `??`, which the renderer converts to the card-back glyph. Status: 0 waiting, 1 playing, 2 folded, 3 left.

Validation is layered (5): length ≥ the 154-byte fixed prefix; round ≤ 5 and playerCount ≤ 8 (an HTTP error page fails this instantly); and length ≥ `154 + playerCount×33` — because a short read leaves stale bytes from a previous, larger reply in the RX window, and fixed-offset reads would render them as garbage.

DRAWING WITHOUT FLICKER

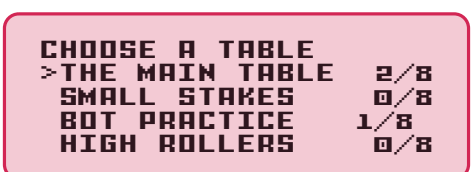
The STIC has no page-flip: BACKTAB is live every frame. The client's answer (6, and the long comment in Listing 2 is worth reading whole) is **differential rendering**: a full CLS only when the layout genuinely changed (first render, player count changed, new hand); otherwise only cells whose values changed are poked, and the redraw yields a `WAIT` between seats so no single burst outruns vblank — the cause of a half-drawn frame a screenshot once caught and made look like data corruption. Seat placement comes from the C

clients' `seatmap` table (player count → seat ring); each seat draws name, bet and its dealt cards, clicking `sound_deal` only for cards not shown last poll.

YOUR TURN

When `activePlayer` is 0 and you are not a spectator, `move_ui` (7) lists up to five server-supplied moves on the status row — **the server names the moves** the client just draws `validMoves[i].name` and submits `.move` — with the cursor defaulting to the second entry (not Fold), a countdown seeded from `moveTime` (the server already subtracted a 4-second network grace), and timeout submitting the highlighted move, exactly like the C clients. Move codes you will see: `FO` fold, `CH` check, `CA` call, `BL`/`BH` bet low/high, `RA` raise, `BB` post bring-in.

The one-shot `lastResult` ("fry bot won with pair, sixes") is easy to miss at bot speed, so the client hashes the field each poll; a changed, non-empty hash blacks out the bottom rows, shows the message for four seconds (8), and forces a redraw afterward.



THE SERVER

`servers/fujinet-game-system/5cardstud` is a Go/gin service. Every route does the same four things: lock the table, apply game logic, save state, serialize the client-centric view — JSON by default, the binary layout above with `?bin=1` (that serializer is `util.go`'s `appendFixedLengthString` plus little-endian `appendUint16` every string is lowercased and gets one NUL terminator, which is why fields are "size + 1" on the wire). State lives in a `sync.Map` keyed by table, one mutex per table; bots move after 3 seconds, humans get 39, minus the 4-second grace the client shows. A hidden `test` table is reserved for client developers — polls there never register on the public Lobby.

CHAPTER 9

GAME TWO: BATTLESHIP

Four 10×10 oceans on one screen, and one shot that hits all of them. A lesson in the STIC's other graphics mode.

Battleship (Listing 6) needed something 5 Card Stud did not: four 10×10 grids visible at once. Text cards cannot do it — but the STIC's **colored-squares** mode can. Its distinctive rule: every shot you fire lands on the same coordinate of **every** opposing board simultaneously. One cursor, up to three victims.

THE COLORED-SQUARES KERNEL

A BACKTAB word with bit 12 set and bit 11 clear stops being a character and becomes four independently colored 4×4-pixel quadrants. `board.bas` (Listing 8) wraps that in three calls: `cs_fill` (whole card), `cs_plot` (read-modify-write one quadrant, [1]), and the board-level `board_cell` (quadrant of a 5×5-card board = one grid cell). Two hardware quirks are baked into its tables: the bottom-right quadrant's high color bit lives up at bit 13, not bit 11 (the pre-shifted `cs_val` table hides this); and the update mask must clear the enable bit before adding it back — an early version preserved it, the ADD doubled it into bit 13, and the card snapped back to text mode showing a stray letter (the comment at [1] tells the story).

The screen: quadrants q1/q2 on the top row, q0 (always you) / q3 below, black divider lines, a 9-column chrome panel on the right for names and ships-left, and the prompt row at the bottom.

CLIENT FLOW AND WIRE FORMAT

Boot, name, Lobby handoff and table select are 5 Card Stud verbatim (AppKey key 5 this time). The poll loop differs in one habit: the reply's **header** decides everything. Requests add `&bin=1&v=2` — version 2 matters, because at game over v1 hides the winner's ships and the game-over screen wants to reveal them ([2]).

OFFSET	SIZE	FIELD	NOTES
0	1	playerCount	
1	33	prompt	server-composed status line
34	1	status	0 lobby, 1 place, 10 start, 11 miss, 12 hit, 13 sunk, 99 over
35	1	playerStatus	you: 0 playing, 1 defeated, 2 viewing, 3 ready, 10 placing
36	1	activePlayer	signed, $FF none
37	1	moveTime	seconds

Then it branches. In the lobby (status 0): server name (21) plus 10-byte records {name[9], ready}. In play: lastAttackPos (1), myShips (10 — yours in [0..4], the winner's in [5..9] at game over), then 115-byte records {name[9], status, gamefield[100], shipsLeft[5]}. A gamefield byte is 0 unknown, 1 hit, 2 miss; a ship byte encodes $\text{cell} + 100\times\text{direction}$ (0 across, 1 down) — so 47 is a horizontal ship starting at column 7 row 4, and 147 the same cell going down. `validate_state` (Listing 7 (3)) computes the expected length for whichever branch applies before any offset is trusted.

PLACING AND SHOOTING

Placement ((4)) is fully client-side until the last moment: five ships (5,4,3,3,2) start at random legal spots; the disc slides, `B1` rotates, the action button confirms against a local occupancy map — no server round-trip to discover an overlap. The result is five encoded bytes, comma-joined into `/place/25,113,4,167,89`. The server answers with authoritative myShips, and the renderer draws **those**, which is what makes a reconnect after a reset come back with your real fleet.

Targeting ((5)) draws one yellow cursor on every live enemy quadrant in lockstep, counts down `moveTime`, and on fire sends `/attack/47`. The status codes 11/12/13 then narrate the result — paired with `lastAttackPos` so each miss/hit/sunk sound plays once per actual event, not once per poll that repeats the status ((6)).

Rendering diffs each visible gamefield against a 100-byte shadow copy per quadrant in scratch RAM ((7)) — the same differential-drawing lesson as 5 Card Stud, with the same one-WAIT-per-board vblank discipline.

THE SERVER

Routes: `/state`, `/ready`, `/place/:ships`, `/attack/:pos`, `/leave`, `/tables`, `/view` — plus a dev-only `/debugEndGame/:winner`. Same table-mutex pattern as 5 Card Stud. Timing: bots 3 s, humans 45 s (250 s when you are alone at a table, capped so it still fits the one-byte moveTime), 4-second network grace, a 5-second bonus after a status change. The hard-coded rooms: two open seas and three AI tables (one, two and three bots), plus the hidden `test` room. When only bots remain, the game resets to the lobby.

CHAPTER 10

GAME THREE: FUJITZEE

Five dice, thirteen rounds, sixteen scores — and a scorecard bigger than the screen.

Fujitzee (Listing 10) is the dice game you think it is, and the most UI-dense of the three: a 15-category scorecard per player simply does not fit on a 20×12 screen next to six seats and five dice. The client's answer is **one card at a time** — a permanent seat strip, dice row and prompt, with the disc paging whose scorecard fills the middle.

WIRE FORMAT

Unlike Battleship, the structure is the same shape in every round:

OFFSET	SIZE	FIELD	NOTES
0	1	playerCount	seats + spectators, up to 12
1	21	serverName	
22	41	prompt	
63	1	round	0 lobby, 1–13 play, 99 over
64	1	rollsLeft	
65	1	activePlayer	signed
66	1	moveTime	seconds
67	1	viewing	
68	6	dice	5 ASCII digits 1–6 + NUL; empty at turn start
74	6	keepRoll	5 ASCII 0/1 + NUL — 1 rerolls
80	15	validScores	signed bytes; $FF = not selectable
95	42×N	players[N]	name[9] alias(1) scores[16] (u16 LE each)

Scores are sixteen little-endian words: six upper categories, computed upper total and bonus (indices 6, 7 — a player never picks those directly), seven lower categories (8–14, index 14 is Fujitzee itself), and the running total at 15. `$FFFF` is "unset"; in the lobby, `scores[0]` moonlights as the ready flag (1 ready, 0 not, `$FFFE` spectator).

WATCH OUT: Twelve 42-byte records plus the 95-byte header is 599 bytes — more than the mailbox's 512-byte RX window. Up to nine player records fit; a table with more spectators than that will fail the length gate and the poll simply retries. Seats are capped at six, so play is never affected — but know where the ceiling is.

TURN MACHINERY

Your turn is a little two-mode machine (1, Listing 10): **DICE** mode — cursor over the ROLL tile and five dice, the action button toggles a die between reroll (the default, wire `1`) and keep (`0`), ROLL submits `/roll/01100`-style masks; and **SCORE** mode — cursor over the open categories, with your current roll's would-be score already painted green in each open cell straight from `validScores`. Out of rolls, the client hops to SCORE mode by itself and parks the cursor on the highest-scoring open category (2) — the same courtesy the C clients extend. Submitting sends `/score/11` with the **wire** index (the display collapses the computed rows 6–7, so the cursor-to-index map adds 2 past the upper section — 3).

Two touches keep the multiplayer legible. Whenever the active player changes, the viewed card snaps to them. And when **someone else's** turn ends, the client diffs their scores against a shadow copy it refreshed on every poll (4) to find which category they just filled, then holds their card on screen for three-quarters of a second with that cell highlighted — so a bot's instant move is still a visible event.

Rolls animate on **every** card, yours or not: a change in `rollsLeft` (or a turn change — that covers the server's automatic opening roll) flickers the dice the wire's keepRoll says were thrown (5).

THE SERVER

Routes `/state`, `/ready`, `/roll/:keep`, `/score/:index`, `/leave`, `/tables`, `/view`, with the familiar per-table locking. Timing: bots 3 s, humans 45 s — 15 for a player being penalized for a previous timeout, 250 when alone — game-start countdowns of 31/6/3 seconds, and the same 4-second grace baked into moveTime. Scoring — including the upper bonus at 63, the Fujitzee bonus scores and the automatic totals — is entirely server-side: the client never adds a die.

CHAPTER 11

THE LOBBY

How players find your table — and how a game remembers where it was.

The FujiNet Lobby (`lobby.fujinet.online`) is the directory of every live game room on every platform. The pieces:

- **Game servers upsert themselves.** Each server POSTs a JSON GameServer record — game name, region, server URL (with `?table=` suffix per room), current and max players, plus a per-platform list of client download URLs — to the Lobby whenever a room's population changes, and game results when a match ends. A QA Lobby at `qalobby.fujinet.online` receives the same traffic from development servers.
- **The Lobby client hands off through AppKeys.** When a player picks a room in the Lobby browser, the Lobby writes `https://server/?table=id` into that game's registered AppKey slot (creator 1 / app 1 / key: 1 five-card-stud, 3 fujitzee, 5 battleship) and boots the game client.
- **The game rejoins on its own.** At boot each game reads its slot, splits the URL at the `?` (`split_room_url`, validating that the query really is `table=` plus letters and digits — the value goes into rebuilt URLs unescaped), and if it parses, skips table select entirely. Picking a table by hand **writes** the same slot back, so a console reset rejoins your game; deliberately quitting **clears** it, so a reset lands on table select instead of dragging you back into a table you left.

The shared username slot (key 0) rounds out the handshake: every lobby game on every platform reads and writes the same player name. Treat both slots as hostile input — Chapters 8 through 10 all validate them byte-by-byte, and so should you.

APPENDIX A

ERROR CODES

MAILBOX LINK ERRORS (FN_ERR, $9C0B)

VALUE	NAME	MEANING
0	OK	transaction completed (or, after a timeout, never started)
1	ENOLINK	no USB link to the ESP32 — check FN_LINK, wait, retry
2	ETIMEOUT	the ESP32 did not answer within the deadline
3	EBADFRAME	framing/checksum failure on the link
4	ETOOBIG	request or reply exceeded the buffers

NETWORK STATUS ERRORS (STATUS BYTE 3)

VALUE	NAME	MEANING
1	SUCCESS	all is well
128	WRITE ONLY	channel not open for reading
131	READ ONLY / WRITE ONLY	direction not open
132	INVALID COMMAND	
135	READ ONLY	
136	END OF FILE	
138	GENERAL TIMEOUT	
144	GENERAL ERROR	
146	NOT IMPLEMENTED	
151	FILE EXISTS	

VALUE	NAME	MEANING
162	NO SPACE ON DEVICE	
165	INVALID DEVICESPEC	the URL did not parse
166	INVALID POINT	bad SEEK
167	ACCESS DENIED	
170	FILE NOT FOUND	
200	CONNECTION REFUSED	
201	NETWORK UNREACHABLE	
202	SOCKET TIMEOUT	
203	NETWORK DOWN	
204	CONNECTION RESET	
205	CONNECTION ALREADY IN PROGRESS	
206	ADDRESS IN USE	
207	NOT CONNECTED	READ/WRITE with no OPEN
208	SERVER NOT RUNNING	
209	NO CONNECTION WAITING	TCP ACCEPT with no caller
210	SERVICE NOT AVAILABLE	
211	CONNECTION ABORTED	
212	INVALID USERNAME OR PASSWORD	
213	COULD NOT PARSE JSON	
214	CLIENT ERROR	HTTP 4xx
215	SERVER ERROR	HTTP 5xx
255	COULD NOT ALLOCATE BUFFERS	

NETWORK-BOOT ERRORS (FN_BOOT_ERR, $9C1A)

VALUE	NAME	MEANING
1	REJECTED	ROM header malformed (bad address range)
2	TRUNCATED	Intellicart stream ended mid-segment
3	NOMAP	no `.cfg`, no header, and the size matches no known layout
4	MAILBOX	a segment would overlap the mailbox window
5	CFGBAD	a `.cfg` arrived but held no mapping line

APPENDIX B

QUICK REFERENCE

THE TRANSACTION RECIPE

1. Stage parameters: `PARAM_SIZE[i]` = 1/2/4, value LE at `PARAM_VAL + 4i`
2. Stage payload at `TX`, length in `TXLEN` (LE)
3. Write `DEVICE`, `CMD`, `NPARAM`
4. `SEQ` = `ACKSEQ` + 1 (wrap, skip 0) — **last write**
5. Poll until `ACKSEQ` = `SEQ` (timeout ≈ 15 s)
6. `REPLY_CMD` = `$06`? Reply at `RX`, length `RXLEN` : else `ERR`

DEVICES

ID	DEVICE	NOTES
`$70`	Fuji control	WiFi, hosts, mounts, AppKeys, encoders
`$71-78`	Network N1:–N8:	one connection each
`$45`	Clock	APETime command set
`$31-38`	Disk	block devices (not used by the Inty client)

NETWORK COMMANDS (DEV $71–$78)

CMD	NAME	PARAMS / PAYLOAD
`$4F 'O'`	OPEN	p0 mode, p1 translation; payload devicespec
`$43 'C'`	CLOSE	—
`$52 'R'`	READ	p0 length(2) → data

CMD	NAME	PARAMS / PAYLOAD
$57 'W'	WRITE	p0 length(2); payload data
$53 'S'	STATUS	p0 0, p1 type → 4 bytes: avail lo/hi, conn, err
$FC	CHANNEL MODE	p1: 0 raw, 1 JSON, 2 SGML
$50 'P'	PARSE	—
$51 'Q'	QUERY	payload query path
$4D 'M'	HTTP CHANNEL	p1: 0 body, 1–3 headers, 4 POST data
$54 'T'	TRANSLATION	p1 mode
$4C 'L'	SET EOL	p0/p1 EOL bytes
$25 '%'	SEEK	p0 offset(4)
$26 '&'	TELL	→ 4-byte offset
$30 '0'	GETCWD	→ prefix
$2C ','	CHDIR	payload path
$FD	USERNAME	payload
$FE	PASSWORD	payload
$41 'A'	TCP ACCEPT	—
$63 'c'	TCP CLOSE CLIENT	—
$44 'D'	UDP DESTINATION	payload host:port
$20/$21	RENAME / DELETE	payload spec
$23/$24	LOCK / UNLOCK	payload spec
$2A/$2B	MKDIR / RMDIR	payload spec

FUJI COMMANDS (DEV $70)

CMD	NAME	PARAMS / PAYLOAD
$FF	RESET	—

CMD	NAME	PARAMS / PAYLOAD
$FE	GET SSID	→ 97 bytes
$FD	SCAN NETWORKS	→ count
$FC	GET SCAN RESULT	p0 index → 34 bytes
$FB	SET SSID	p0 any; payload 97 bytes
$FA	GET WIFI STATUS	→ 1 byte, 3 = connected
$F9	MOUNT HOST	p0 slot
$F8	MOUNT IMAGE	p0 slot, p1 flags — boots the ROM on Inty
$F7	OPEN DIRECTORY	p0 slot; payload 256 bytes path+filter
$F6	READ DIR ENTRY	p0 maxlen, p1 flags → name / $7F7F
$F5	CLOSE DIRECTORY	—
$F4	READ HOST SLOTS	→ 256 bytes
$F3	WRITE HOST SLOTS	payload 256 bytes
$F2/$F1	READ / WRITE DEVICE SLOTS	304-byte table
$EA	GET WIFI ENABLED	→ 1 byte
$E9	UNMOUNT IMAGE	p0 slot
$E8	GET ADAPTERCONFIG	→ 140 bytes
$E7	NEW DISK	payload spec
$E5/$E4	GET / SET DIR POSITION	p0 pos(2)
$E2	SET DEVICE FULLPATH	p0 dev, p1 host, p2 mode; payload 256
$E1/$E0	SET / GET HOST PREFIX	p0 slot
$DE	WRITE APPKEY	payload value
$DD	READ APPKEY	→ 2-byte length + value
$DC	OPEN APPKEY	payload 6 bytes
$DB	CLOSE APPKEY	—

CMD	NAME	PARAMS / PAYLOAD
$DA	GET DEVICE FULLPATH	p0 slot → path
$D9	CONFIG BOOT	p0 flag
$D8	COPY FILE	p0 src, p1 dst; payload spec
$D7	MOUNT ALL	—
$D6	SET BOOT MODE	p0 mode
$D0-$C9	BASE64 ENC/DEC	input / compute / length / output
$C8-$C2	HASH	p0: algo 0 MD5, 1 SHA1, 2 SHA256, 3 SHA512
$C4	ADAPTERCONFIG EXTENDED	→ 240 bytes
$BF-$BC	QR CODE	input / encode / length / output
$BB	GENERATE GUID	→ guid
$53	STATUS	p0 1 = mount times
$00	DEVICE READY	→ 512 test bytes

APPKEY REGISTRY (CREATOR 1, APP 1)

KEY	HOLDER	CONTENTS
0	all lobby games	shared player name
1	5 Card Stud	server?table= handoff
3	Fujitzee	server?table= handoff
5	Battleship	server?table= handoff

APPENDIX C

PROGRAM LISTINGS

The complete, building source of the library and all three games, line-numbered, with the call-outs the chapters reference. Each game also includes the community-standard `constants.bas` (Mark Ball's, from the IntyBASIC distribution), not reprinted here.

LISTING 1 fujinet.bas — the FujiNet library (Chapter 7)

```
' fujinet.bas -- FujiNet mailbox transport + network/appkey primitives.
'
' Mailbox layout is the hand-synchronized copy of
' fujinet-firmware/pico/intellivision/firmware/fuji_mailbox.h, exactly as
' proven working by intv/fujitest.bas in that repo. Do not change these
' addresses without re-checking that file.
'
' MEMATTR intentionally stops at $9BFF, short of the $9C00-$9FFF mailbox
' itself: on real PiRTO II hardware the RP2040 maps that whole window as
' RAM unconditionally (inty_cart.c hardcodes it, independent of what this
' .cfg says), so declaring less here doesn't affect real hardware or what
' POKE can reach at runtime. But jzIntv's --fujinet peripheral emulation
' registers its own handler for $9C00-$9FFF *after* the cart's generic
' MEMATTR RAM, and its layered bus dispatch lets whichever peripheral
' registered first answer a given address -- so declaring the full
' $8000-$9FFF range here (as fujitest.bas does) silently shadows the
' emulator's FujiNet peripheral with inert RAM, and the mailbox never
' comes up under --fujinet even though it works on real hardware.
    ASM MEMATTR $8000, $9BFF, "+RWN"  (8)

    CONST FN_MAGIC0     = $9C00
    CONST FN_MAGIC1     = $9C01
    CONST FN_SEQ        = $9C03
    CONST FN_ACKSEQ     = $9C04
    CONST FN_DEVICE     = $9C05
    CONST FN_CMD        = $9C06
    CONST FN_NPARAM     = $9C07
    CONST FN_TXLEN_LO   = $9C08
    CONST FN_TXLEN_HI   = $9C09
    CONST FN_ERR        = $9C0B
    CONST FN_RXLEN_LO   = $9C0C
    CONST FN_RXLEN_HI   = $9C0D
    CONST FN_REPLY_CMD  = $9C0E
    CONST FN_PARAM_SIZE = $9C10
    CONST FN_PARAM_VAL  = $9C20
    CONST FN_TX         = $9C40
    CONST FN_RX         = $9D40

    CONST FUJICMD_ACK = $06
    CONST FUJICMD_NAK = $15

    ' Fuji device (config/appkey) commands.
    CONST FUJI_DEVICEID       = $70
    CONST FUJICMD_OPEN_APPKEY  = $DC
    CONST FUJICMD_CLOSE_APPKEY = $DB
    CONST FUJICMD_WRITE_APPKEY = $DE
    CONST FUJICMD_READ_APPKEY  = $DD

    ' Network device (N1:) commands.
    CONST NET_DEVICEID = $71
    CONST NETCMD_OPEN   = $4F
    CONST NETCMD_CLOSE  = $43
    CONST NETCMD_READ   = $52
    CONST NETCMD_WRITE  = $57
    CONST NETCMD_STATUS = $53

    ' $0C is HTTP "GET with header access" and, on file/socket protocols,
    ' plain READ-WRITE -- the same value serves both names.
    CONST OPEN_MODE_HTTP_GET_H = $0C
    CONST OPEN_MODE_RW = $0C
    CONST OPEN_TRANS_NONE = $00

    ' Scratch RAM ($9000-$97FF) -- ours, outside the mailbox proper. Holds
    ' state that must survive across multiple mailbox transactions (the
    ' mailbox's own TX/RX buffers get overwritten by every call).
    CONST SC_NAME    = $9100 ' playerName, 9 bytes (8 + NUL)
    CONST SC_ENDPT   = $9110 ' serverEndpoint, 64 bytes
```

```
    CONST SC_QUERY   = $9150 ' query (?table=...&player=...), 48 bytes

    ' fn_ok: 1 = last transaction produced ACK, 0 = timeout or NAK.
    ' mb_err: FN_ERR value on failure (0 on timeout, since the RP2040 never
answered).
    DIM fn_ok, #mb_err
    DIM mb_dev, mb_cmd, mb_nparam, mb_seq
    DIM #fn_txlen
    DIM #fn_t           ' generic frame-count timeout counter
    DIM #fn_src         ' VARPTR source for putstr/getstr
    DIM fn_len, fn_i    ' generic length/index for putstr/getstr

' ------------------------------------------------------------------------
' fn_wait_mailbox: bounded wait for the RP2040 magic bytes at boot.
' Sets fn_ok = 1 if the mailbox came up within 180 frames (3s), else 0.
' ------------------------------------------------------------------------
fn_wait_mailbox: PROCEDURE
    #fn_t = 0
    WHILE ((PEEK(FN_MAGIC0) AND 255) <> 70) AND ((PEEK(FN_MAGIC1) AND 255) <>
78) AND (#fn_t < 180)
        #fn_t = #fn_t + 1
        WAIT
    WEND
    IF #fn_t >= 180 THEN
        fn_ok = 0
    ELSE (1)
        fn_ok = 1
    END IF
END

' ------------------------------------------------------------------------
' fn_transact: issue the transaction described by mb_dev/mb_cmd/mb_nparam/
' #fn_txlen (payload already staged at FN_TX) and block for the reply.
' seq is ALWAYS derived from the RP2040's own FN_ACKSEQ, never from a local
' counter -- a console reset zeroes IntyBASIC vars but not the RP2040, so a
' locally incrementing seq would recompute the same value forever and never
' trigger a second transaction after the first boot.
' Sets fn_ok = 1 on ACK, 0 on timeout/NAK (mb_err holds the reason).
'
' mb_err/net_err are deliberately 16-bit (#-prefixed), not the 8-bit plain
' variables their DIM comments once declared them as: IntyBASIC v1.4.2
' silently drops the AND 255 mask on any assignment of the literal shape
' "plain_var = PEEK(...) AND 255" (with or without wrapping parens) when
' the destination is an 8-bit variable -- confirmed by reading the
' generated .lst, no ANDI instruction is emitted at all, so whatever
' garbage sits in the peeked word's upper byte survives into the
' variable. #-prefixed (16-bit) destinations don't hit this; the ANDI
' shows up correctly. net_err in particular is compared with "<> 1"
' immediately after, so a stray upper byte there would misreport a good
' status as a failure.
' ------------------------------------------------------------------------
fn_transact: PROCEDURE
    POKE (FN_DEVICE), mb_dev
    POKE (FN_CMD), mb_cmd
    POKE (FN_NPARAM), mb_nparam (2)

    POKE (FN_TXLEN_LO), #fn_txlen AND 255
    POKE (FN_TXLEN_HI), #fn_txlen / 256

    mb_seq = (PEEK(FN_ACKSEQ) AND 255) + 1
    IF mb_seq = 0 THEN mb_seq = 1 (3)

    POKE (FN_SEQ), mb_seq

    #fn_t = 0
    WHILE ((PEEK(FN_ACKSEQ) AND 255) <> mb_seq) AND (#fn_t < 900)
        #fn_t = #fn_t + 1
        WAIT
    WEND

    IF #fn_t >= 900 THEN
        fn_ok = 0
        #mb_err = 0 (4)

        RETURN
    END IF

    IF (PEEK(FN_REPLY_CMD) AND 255) <> FUJICMD_ACK THEN
        fn_ok = 0
        #mb_err = (PEEK(FN_ERR) AND 255)
        RETURN
    END IF

    fn_ok = 1
END

' ------------------------------------------------------------------------
' fn_param: stage transaction parameter #pm_i (0-based), #pm_size bytes
' (1 or 2), value #pm_val, little-endian, into the mailbox's param table.
' ------------------------------------------------------------------------
DIM pm_i, pm_size
DIM #pm_val
fn_param: PROCEDURE
    POKE (FN_PARAM_SIZE + pm_i), pm_size
    POKE (FN_PARAM_VAL + pm_i * 4), #pm_val AND 255
    IF pm_size > 1 THEN POKE (FN_PARAM_VAL + pm_i * 4 + 1), #pm_val / 256
END

' ------------------------------------------------------------------------
' fn_putstr: append fn_len ASCII bytes into FN_TX, starting at the current
' #fn_txlen, from the address in #fn_src -- either VARPTR of a ROM DATA
' label (a literal like lit_tables(0)) or a RAM address (a scratch buffer
' like SC_NAME, or even FN_RX). PEEK is uniform across ROM/RAM on the
' CP-1610's unified address space, so one procedure covers both. Advances
' #fn_txlen.
' ------------------------------------------------------------------------
```

```
fn_putstr: PROCEDURE
    FOR fn_i = 0 TO fn_len - 1
        POKE (FN_TX + #fn_txlen + fn_i), PEEK(#fn_src + fn_i) AND 255
    NEXT fn_i
    #fn_txlen = #fn_txlen + fn_len
END

' ------------------------------------------------------------------------
' fn_strlen: scan the NUL-padded field at #fn_src (max ls_max bytes) and set
' fn_len to the length up to (but not including) the first NUL. Use this
' before fn_putstr whenever the source is a fixed-width padded field (table
' id, player name) -- putstr has no idea where the real string ends, and
' blindly copying the full padded width embeds a literal $00 byte plus
' whatever garbage follows it into the URL, which is sent as-is (the URL's
' length is a byte count, not NUL-terminated, so nothing downstream stops
' at that NUL either -- it corrupts the HTTP request on the wire).
' ------------------------------------------------------------------------
DIM ls_max
fn_strlen: PROCEDURE
    fn_len = 0
    WHILE (fn_len < ls_max) AND ((PEEK(#fn_src + fn_len) AND 255) <> 0)
        fn_len = fn_len + 1
    WEND
END

' ------------------------------------------------------------------------
' net_open: open devicespec (ASCII bytes already staged at FN_TX, length in
' #fn_txlen) for HTTP GET. Leaves fn_ok set.
' ------------------------------------------------------------------------
net_open: PROCEDURE
    mb_dev = NET_DEVICEID
    mb_cmd = NETCMD_OPEN
    mb_nparam = 2
    pm_i = 0 : pm_size = 1 : #pm_val = OPEN_MODE_HTTP_GET_H : GOSUB fn_param
    pm_i = 1 : pm_size = 1 : #pm_val = OPEN_TRANS_NONE : GOSUB fn_param
    GOSUB fn_transact
END

' ------------------------------------------------------------------------
' net_status: query byte count available. Result in #net_avail; fn_ok as usual.
' ------------------------------------------------------------------------
DIM #net_avail
' net_err: the 4th byte of the NDeviceStatus reply (nDevStatus_t). 1 =
' SUCCESS. For an HTTP-backed devicespec, the *actual* GET is deferred
' until this first STATUS call (see NetworkProtocolHTTP::status_file()/
' http_transaction() in fujinet-firmware) and its result code lands here
' -- an HTTP error response (e.g. a 400) still has a real, readable body
' (the error page), so `avail` alone can't tell it apart from a good
' response. Checking net_err is what actually can.
DIM #net_err
net_status: PROCEDURE
    mb_dev = NET_DEVICEID
    mb_cmd = NETCMD_STATUS
    mb_nparam = 2
    pm_i = 0 : pm_size = 1 : #pm_val = 0 : GOSUB fn_param
    pm_i = 1 : pm_size = 1 : #pm_val = 0 : GOSUB fn_param
    #fn_txlen = 0
    GOSUB fn_transact
    IF fn_ok THEN
        #net_avail = (PEEK(FN_RX) AND 255) + (PEEK(FN_RX + 1) AND 255) * 256
        #net_err = (PEEK(FN_RX + 3) AND 255)
        IF #net_err <> 1 THEN fn_ok = 0
    ELSE
        #net_avail = 0
    END IF
END

' ------------------------------------------------------------------------
' net_read: read #net_readlen bytes into FN_RX (reply payload lands there
' directly -- callers PEEK it in place, per the "never buffer in IntyBASIC
' variables" rule). fn_ok as usual.
' ------------------------------------------------------------------------
DIM #net_readlen
' #net_gotlen: the actual byte count the RP2040/FujiNet peripheral reported
' receiving (RXLEN), captured here because the CLOSE transaction that
' follows in api_call overwrites FN_RXLEN_LO/HI with its own (always 0)
' reply length -- callers need this checked before that happens.
DIM #net_gotlen
net_read: PROCEDURE
    mb_dev = NET_DEVICEID
    mb_cmd = NETCMD_READ
    mb_nparam = 1
    pm_i = 0 : pm_size = 2 : #pm_val = #net_readlen : GOSUB fn_param
    #fn_txlen = 0
    GOSUB fn_transact
    IF fn_ok THEN
        #net_gotlen = (PEEK(FN_RXLEN_LO) AND 255) + (PEEK(FN_RXLEN_HI) AND 255)
* 256
    ELSE
        #net_gotlen = 0
    END IF
END

' ------------------------------------------------------------------------
' net_write: send fn_len bytes already staged at FN_TX out the open
' channel. The byte count rides twice -- as parameter 0 (the command's
' length argument) and as the transaction payload length -- and both come
' from fn_len here. fn_ok as usual. The three games never write to the
' network (their moves are GET query strings), but a TCP/TELNET client
' needs this, so the library carries it.
' ------------------------------------------------------------------------
net_write: PROCEDURE
    mb_dev = NET_DEVICEID
    mb_cmd = NETCMD_WRITE
    mb_nparam = 1
```

```
    pm_i = 0 : pm_size = 2 : #pm_val = fn_len : GOSUB fn_param
    #fn_txlen = fn_len
    GOSUB fn_transact
END

' ------------------------------------------------------------------------
' net_close
' ------------------------------------------------------------------------
net_close: PROCEDURE
    mb_dev = NET_DEVICEID
    mb_cmd = NETCMD_CLOSE
    mb_nparam = 0
    #fn_txlen = 0
    GOSUB fn_transact
END

' ------------------------------------------------------------------------
' api_call: full round trip -- open the URL staged at FN_TX/#fn_txlen,
' check status, read up to #net_readlen bytes (caller sets this to the max
' expected reply size, e.g. 418 for /state, 361 for /tables), close.
' Leaves fn_ok = 1 and the reply in FN_RX on success.
' ------------------------------------------------------------------------
' ac_i/#ac_prev: loop counter and previous STATUS reading for api_call's
' stabilization poll (see below). 6

DIM ac_i
DIM #ac_prev
api_call: PROCEDURE
    GOSUB net_open
    IF fn_ok = 0 THEN RETURN

    ' The RP2040/ESP32 can report STATUS as soon as *some* bytes of the
    ' real internet response have arrived, well before the full response
    ' does -- open+status+read all happening within the same poll can
    ' easily race ahead of that. Rather than guess a fixed delay, poll
    ' STATUS repeatedly and treat two consecutive equal (nonzero)
    ' readings as "the download has settled" before committing to a
    ' read length. Bounded to ~20 frames (~0.33s) of polling so a
    ' genuinely stuck connection still falls through to the normal
    ' fn_ok=0/timeout handling instead of hanging here.
    #ac_prev = 0
    FOR ac_i = 0 TO 19
        WAIT
        GOSUB net_status
        IF fn_ok = 0 THEN RETURN
        IF #net_avail > 0 AND #net_avail = #ac_prev THEN EXIT FOR
        #ac_prev = #net_avail
    NEXT ac_i

    IF #net_avail < #net_readlen THEN #net_readlen = #net_avail

    GOSUB net_read
    GOSUB net_close   ' close regardless of read result
END

' ------------------------------------------------------------------------
' AppKey. The wire struct (fujiDevice.h's `struct appkey`, packed) is 6
' bytes: creator_lo, creator_hi, app, key, mode, reserved. mode: 0=read,
' 1=write. Sending only 5 bytes (omitting reserved) leaves the firmware's
' transaction_get() blocked waiting for a byte that never arrives, which
' reads back as a timeout, not a protocol error -- easy to misdiagnose.
' appkey_open must be followed by appkey_read or appkey_write, then
' appkey_close, mirroring the C client's read_appkey()/write_appkey().
' ------------------------------------------------------------------------
DIM ak_creator_lo, ak_creator_hi, ak_app, ak_key, ak_mode

appkey_open: PROCEDURE
    mb_dev = FUJI_DEVICEID
    mb_cmd = FUJICMD_OPEN_APPKEY
    mb_nparam = 0
    POKE (FN_TX + 0), ak_creator_lo
    POKE (FN_TX + 1), ak_creator_hi
    POKE (FN_TX + 2), ak_app
    POKE (FN_TX + 3), ak_key
    POKE (FN_TX + 4), ak_mode
    POKE (FN_TX + 5), 0 ' reserved
    #fn_txlen = 6
    GOSUB fn_transact
END

' Reads into SC_ names buffer at address #fn_src (caller sets), up to
' fn_len bytes. Actual byte count returned in fn_len after the call (from
' RXLEN); callers should NUL-terminate at that offset.
' Caller sets #fn_src (destination) and ls_max (destination buffer size,
' including room for the NUL this always writes at fn_len). Clamps to 7

' ls_max-1 bytes copied -- appkeys can be up to 64 bytes but destinations
' like SC_NAME are much smaller, and always NUL-terminates at fn_len so a
' short stored value doesn't leave trailing bytes undefined for callers
' (like compose_url via fn_strlen) that scan for the terminator.
appkey_read: PROCEDURE
    mb_dev = FUJI_DEVICEID
    mb_cmd = FUJICMD_READ_APPKEY
    mb_nparam = 0
    #fn_txlen = 0
    GOSUB fn_transact
    IF fn_ok THEN
        ' The rs232 transport (rs232Fuji::fujicore_read_app_key, verified
        ' against fujinet-firmware source and a live byte dump) prepends
        ' a 2-byte little-endian length ahead of the actual appkey bytes
        ' -- unlike a network READ, an appkey READ takes no length
        ' parameter, so the firmware makes the reply self-describing
        ' instead. This is specific to appkey reads on this transport;
        ' network reads carry no such prefix. RXLEN (FN_RXLEN_LO/HI)
        ' covers the prefix + data together, so use the embedded prefix
        ' itself as the real data length and skip past it.
```

```
        fn_len = (PEEK(FN_RX) AND 255) + (PEEK(FN_RX + 1) AND 255) * 256
        IF fn_len > ls_max - 1 THEN fn_len = ls_max - 1
        FOR fn_i = 0 TO fn_len - 1
            POKE (#fn_src + fn_i), PEEK(FN_RX + 2 + fn_i) AND 255
        NEXT fn_i
        POKE (#fn_src + fn_len), 0
    ELSE
        fn_len = 0
    END IF
END

' Writes fn_len bytes from #fn_src (an SC_ buffer) as the appkey payload.
appkey_write: PROCEDURE
    mb_dev = FUJI_DEVICEID
    mb_cmd = FUJICMD_WRITE_APPKEY
    mb_nparam = 0
    FOR fn_i = 0 TO fn_len - 1
        POKE (FN_TX + fn_i), PEEK(#fn_src + fn_i) AND 255
    NEXT fn_i
    #fn_txlen = fn_len
    GOSUB fn_transact
END

appkey_close: PROCEDURE
    mb_dev = FUJI_DEVICEID
    mb_cmd = FUJICMD_CLOSE_APPKEY
    mb_nparam = 0
    #fn_txlen = 0
    GOSUB fn_transact
END

' ------------------------------------------------------------------------
' fn_putnum: append the decimal (no leading zeros) representation of pn_val
' (0-999) into FN_TX at the current #fn_txlen. IntyBASIC has no built-in
' itoa; three digits covers every value the game servers take in a URL
' (board positions, score indices, ship placements).
' ------------------------------------------------------------------------
DIM pn_val, pn_h, pn_t, pn_o, pn_started
fn_putnum: PROCEDURE
    pn_h = pn_val / 100
    pn_t = (pn_val / 10) % 10
    pn_o = pn_val % 10
    pn_started = 0
    IF pn_h > 0 THEN
        POKE (FN_TX + #fn_txlen), pn_h + 48 : #fn_txlen = #fn_txlen + 1
        pn_started = 1
    END IF
    IF pn_t > 0 OR pn_started THEN
        POKE (FN_TX + #fn_txlen), pn_t + 48 : #fn_txlen = #fn_txlen + 1
    END IF
    POKE (FN_TX + #fn_txlen), pn_o + 48 : #fn_txlen = #fn_txlen + 1
END

```

LISTING 2 5card.bas — 5 Card Stud client (Chapter 8)

```
' 5card.bas -- 5 Card Stud for Intellivision, entry point + state machine.
'
' Screen flow (simplified from the C clients' full parity, see the plan's
' Risks section): name entry (disc letter picker, falls back to it whenever
' the appkey read fails or comes back empty) -> table select (always shown
' at boot; the server endpoint is a compiled-in default rather than an
' appkey-persisted URL, since table select runs every boot anyway) -> game
' loop -> in-game menu (keypad CLEAR: quit table or resume).
'
' Rendering only fully clears the screen when the table layout actually
' changes (see render_game); otherwise it updates cells in place, matching
' the CoCo/MSX C clients' SINGLE_BUFFER mode -- IntyBASIC has no true
' page-flip double buffering, so avoiding a per-poll CLS is what keeps the
' screen from flashing blank on every state refresh.
'
' GOTO boot_start jumps over every INCLUDE below before any of it runs.
' This isn't optional: fujinet.bas/gfx.bas are libraries whose PROCEDURE
' (and gfx.bas's DATA) bodies are meant to be reached only via GOSUB, per
' the IntyBASIC manual's warning that falling into a PROCEDURE or DATA
' block by straight-line execution corrupts the return stack. Since
' INCLUDE pastes their text in at this point, and this file's own
' DATA/PROCEDURE blocks (below) have the same requirement, the single
' cheapest fix is to never let the CPU reach any of it except via GOSUB --
' hence jumping straight to boot_start before the includes even begin.
    GOTO boot_start

    INCLUDE "constants.bas"
    INCLUDE "fujinet.bas"
    INCLUDE "gfx.bas"
    INCLUDE "state.bas"
    INCLUDE "sound.bas"

    CONST ROWCELLS = 20
    CONST STATUS_ROW = 220

    CONST COL_NAME  = FG_WHITE + BG_DARKGREEN
    CONST COL_HILITE = FG_YELLOW + BG_DARKGREEN
    CONST COL_STATUS = FG_WHITE + BG_BLACK

' Seat base offsets (name row), seat 0 is always you (bottom-center).
seat_name_off:
    DATA 167, 140, 80, 20, 7, 34, 94, 154

' playerCountIndex: for a given playerCount (2-8, row = playerCount-2),
' the seat assigned to server player index i (array position i). 255 =
' unused. Ported unchanged from the C client's seat-assignment table.
seatmap:
    DATA 0,4,255,255,255,255,255,255       ' 2 players
    DATA 0,2,6,255,255,255,255,255         ' 3
    DATA 0,2,4,6,255,255,255,255           ' 4
    DATA 0,2,3,5,6,255,255,255             ' 5
    DATA 0,2,3,4,5,6,255,255               ' 6
    DATA 0,2,3,4,5,6,7,255                 ' 7
    DATA 0,1,2,3,4,5,6,7                   ' 8

lit_n_colon: DATA 78,58
lit_https: DATA 104,116,116,112,115,58,47,47,53,99,97,114,100,46,99,97,114,114,
45,100,101,115,105,103,110,115,46,99,111,109,47
lit_tables: DATA 116,97,98,108,101,115
lit_state: DATA 115,116,97,116,101
lit_move: DATA 109,111,118,101,47
lit_leave: DATA 108,101,97,118,101
' "&be=1" (big-endian) dropped: the Go server's binary.BigEndian path is
' the road much less travelled (CoCo, 6809, is the only other client that
' ever requests it -- everything else in this client family is little-
' endian x86/6502/Z80), and pot/bet corruption that always landed on
' exact multiples of 256 (i.e. a real small value sitting in the high
' byte, low byte zero) pointed straight at a byte-order mismatch on that
' path specifically. Little-endian is what the rest of the ecosystem
' exercises constantly.
lit_qbin: DATA 63,98,105,110,61,49
lit_abin: DATA 38,98,105,110,61,49
lit_qtable: DATA 63,116,97,98,108,101,61
lit_aplayer: DATA 38,112,108,97,121,101,114,61

    CONST LEN_HTTPS = 31

    ' Lobby appkey: creator 1 / app 1, key = this game's registered slot.
    ' See fujinet-5cardstud/src/misc.h AK_LOBBY_KEY_SERVER.
    CONST AK_LOBBY_KEY_SERVER = 1

    ' Selected table id, 9 bytes (8 + NUL). SC_ENDPT used to double as this
    ' buffer, but it's now needed for the full lobby-supplied endpoint, so
    ' the table id gets its own slot -- free RAM just past fujinet.bas's
    ' own SC_* buffers (which stop at SC_QUERY $9150 + 48 = $9180) and
    ' well below the $9C00 mailbox.
    CONST SC_TABLE = $9180

' ---------------------------------------------------------------------------
' Globals
' ---------------------------------------------------------------------------
    DIM gs_i, gs_j, #gs_c
    DIM ne_cur, ne_len
    DIM ne_buf(8)
    DIM tbl_count, tbl_sel
    DIM sel_seat, sel_i
    DIM poll_wait
    DIM mv_sel, mv_count
    DIM mv_col(5)
    DIM inp_lock
```

```
    DIM mvcode_a, mvcode_b
    DIM has_move
    DIM #tmp_addr
    DIM #tmp_num
    DIM #prev_pot
    DIM #prev_purse
    DIM prev_bet(8)
    DIM prev_cards(8)
    DIM deal_count
    DIM #tmp_expect
    DIM #tmp_hash
    DIM #prev_result_hash
    DIM first_poll

    DIM df_pos, df_len, #df_color, df_i, df_c, df_stop
    DIM #df_src

    ' split_room_url locals (see procedure below)
    DIM sp_i, sp_found, sp_ok, sp_j, sp_k, sp_c, sp_valid, sp_m

' ------------------------------------------------------------------------
' draw_field: render df_len ASCII bytes from #df_src onto the screen at
' BACKTAB offset df_pos, in color #df_color. Uppercases (MODE 1 has no
' lowercase GROM; the wire is lowercase anyway), stops at a NUL and pads
' the rest of the field with spaces, and clamps anything outside the
' printable GROM range (32-95) to a space rather than drawing garbage.
' ------------------------------------------------------------------------
' ------------------------------------------------------------------------
' fill_bg: felt-table background -- green everywhere except the status row
' (which stays black). Call right after every CLS.
' ------------------------------------------------------------------------
fill_bg: PROCEDURE
    FOR gs_i = 0 TO STATUS_ROW - 1
        #BACKTAB(gs_i) = COL_NAME
    NEXT gs_i
END

draw_field: PROCEDURE
    df_stop = 0
    FOR df_i = 0 TO df_len - 1
        df_c = PEEK(#df_src + df_i) AND 255
        IF df_c = 0 THEN df_stop = 1
        IF df_stop THEN
            df_c = 32
        ELSEIF df_c >= 97 AND df_c <= 122 THEN
            df_c = df_c - 32
        END IF
        IF df_c < 32 OR df_c > 95 THEN df_c = 32
        #BACKTAB(df_pos + df_i) = (df_c - 32) * 8 + #df_color
    NEXT df_i
END

' ------------------------------------------------------------------------
' compose_url: build "N:<endpoint><path...><suffix>" directly into FN_TX.
' Callers set gs_path (0=tables 1=state 2=move 3=leave) and, for move,
' mvcode_a/mvcode_b (the 2-char move code) beforehand.
' ------------------------------------------------------------------------
    DIM gs_path
compose_url: PROCEDURE

    #fn_txlen = 0
    #fn_src = VARPTR lit_n_colon(0) : fn_len = 2 : GOSUB fn_putstr
    #fn_src = SC_ENDPT : ls_max = 65 : GOSUB fn_strlen : GOSUB fn_putstr

    IF gs_path = 0 THEN
        #fn_src = VARPTR lit_tables(0) : fn_len = 6 : GOSUB fn_putstr
        #fn_src = VARPTR lit_qbin(0) : fn_len = 6 : GOSUB fn_putstr
    ELSE
        IF gs_path = 1 THEN
            #fn_src = VARPTR lit_state(0) : fn_len = 5 : GOSUB fn_putstr
        ELSEIF gs_path = 2 THEN
            #fn_src = VARPTR lit_move(0) : fn_len = 5 : GOSUB fn_putstr
            POKE (FN_TX + #fn_txlen), mvcode_a : #fn_txlen = #fn_txlen + 1
            POKE (FN_TX + #fn_txlen), mvcode_b : #fn_txlen = #fn_txlen + 1
        ELSE
            #fn_src = VARPTR lit_leave(0) : fn_len = 5 : GOSUB fn_putstr
        END IF

        #fn_src = VARPTR lit_qtable(0) : fn_len = 7 : GOSUB fn_putstr
        #fn_src = SC_TABLE : ls_max = 9 : GOSUB fn_strlen : GOSUB fn_putstr   '
table id, NUL-terminated within its 9-byte field
        #fn_src = VARPTR lit_aplayer(0) : fn_len = 8 : GOSUB fn_putstr
        #fn_src = SC_NAME : ls_max = 9 : GOSUB fn_strlen : GOSUB fn_putstr    '
player name, likewise
        #fn_src = VARPTR lit_abin(0) : fn_len = 6 : GOSUB fn_putstr
    END IF
END

' ------------------------------------------------------------------------
' split_room_url: given a lobby-supplied "https://host/?table=xyz" value
' already sitting in SC_ENDPT with its length in fn_len (as left by
' appkey_read), split it into the bare endpoint (SC_ENDPT, truncated at
' the '?') and the table id (SC_TABLE). Mirrors the C clients'
' welcomeActionVerifyServerDetails() '?' scan. Leaves SC_TABLE empty (a
' leading NUL) if there's no '?', the query isn't "table=...", or the id
' contains anything other than A-Z/a-z/0-9 -- it goes straight into a
' rebuilt query string unescaped, same reasoning as the username check
' above.
' ------------------------------------------------------------------------
split_room_url: PROCEDURE
    POKE SC_TABLE, 0

    sp_found = 255 ' sentinel: not found (mirrors st_boot.bas's bt_slash
convention)
    sp_i = 0
    WHILE sp_i < fn_len
```

```
        IF (PEEK(SC_ENDPT + sp_i) AND 255) = 63 THEN ' '?'
            sp_found = sp_i
            EXIT WHILE
        END IF
        sp_i = sp_i + 1
    WEND
    IF sp_found = 255 THEN RETURN

    ' Truncate the endpoint at the '?' regardless of what follows.
    POKE (SC_ENDPT + sp_found), 0

    ' Require "table=" (6 bytes) right after the '?'.
    sp_ok = 0
    IF sp_found + 7 <= fn_len THEN
        sp_ok = 1
        IF (PEEK(SC_ENDPT + sp_found + 1) AND 255) <> 116 THEN sp_ok = 0 ' t
        IF (PEEK(SC_ENDPT + sp_found + 2) AND 255) <> 97  THEN sp_ok = 0 ' a
        IF (PEEK(SC_ENDPT + sp_found + 3) AND 255) <> 98  THEN sp_ok = 0 ' b
        IF (PEEK(SC_ENDPT + sp_found + 4) AND 255) <> 108 THEN sp_ok = 0 ' l
        IF (PEEK(SC_ENDPT + sp_found + 5) AND 255) <> 101 THEN sp_ok = 0 ' e
        IF (PEEK(SC_ENDPT + sp_found + 6) AND 255) <> 61  THEN sp_ok = 0 ' =
    END IF
    IF sp_ok = 0 THEN RETURN

    ' Copy the id: up to 8 bytes, stopping at NUL or '&'.
    sp_j = sp_found + 7
    sp_k = 0
    WHILE sp_k < 8 AND sp_j < fn_len
        sp_c = PEEK(SC_ENDPT + sp_j) AND 255
        IF sp_c = 0 OR sp_c = 38 THEN EXIT WHILE ' NUL or '&'
        POKE (SC_TABLE + sp_k), sp_c
        sp_k = sp_k + 1
        sp_j = sp_j + 1
    WEND
    POKE (SC_TABLE + sp_k), 0

    ' Validate: A-Z/a-z/0-9 only.
    sp_valid = 1
    FOR sp_m = 0 TO sp_k - 1
        sp_c = PEEK(SC_TABLE + sp_m) AND 255
        IF (sp_c < 65 OR sp_c > 90) AND (sp_c < 97 OR sp_c > 122) AND (sp_c < 48
OR sp_c > 57) THEN sp_valid = 0
    NEXT sp_m
    IF sp_valid = 0 THEN POKE SC_TABLE, 0
END

' ------------------------------------------------------------------------
' write_room_appkey: persist SC_ENDPT + "?table=" + SC_TABLE back to the
' lobby server appkey slot, so a reboot without going back through the
' lobby rejoins the same room (mirrors the C clients' "Update server app
' key in case of reboot"). Builds the payload directly in FN_TX and writes
' it from there -- appkey_write's byte-by-byte copy from #fn_src into
' FN_TX is a safe no-op when #fn_src is FN_TX itself.
' ------------------------------------------------------------------------
write_room_appkey: PROCEDURE
    #fn_txlen = 0
    #fn_src = SC_ENDPT : ls_max = 65 : GOSUB fn_strlen : GOSUB fn_putstr
    #fn_src = VARPTR lit_qtable(0) : fn_len = 7 : GOSUB fn_putstr
    #fn_src = SC_TABLE : ls_max = 9 : GOSUB fn_strlen : GOSUB fn_putstr

    ak_creator_lo = 1 : ak_creator_hi = 0 : ak_app = 1
    ak_key = AK_LOBBY_KEY_SERVER : ak_mode = 1
    GOSUB appkey_open
    IF fn_ok THEN
        fn_len = #fn_txlen : #fn_src = FN_TX
        GOSUB appkey_write
        GOSUB appkey_close
    END IF
END

' ------------------------------------------------------------------------
' clear_room_appkey: blank the lobby server appkey slot on a deliberate
' leave, so a later reboot lands on table select instead of silently
' rejoining a table the player just quit.
' ------------------------------------------------------------------------
clear_room_appkey: PROCEDURE
    ak_creator_lo = 1 : ak_creator_hi = 0 : ak_app = 1
    ak_key = AK_LOBBY_KEY_SERVER : ak_mode = 1
    GOSUB appkey_open
    IF fn_ok THEN
        fn_len = 0 : #fn_src = FN_TX
        GOSUB appkey_write
        GOSUB appkey_close
    END IF
    POKE SC_TABLE, 0
END

' ========================================================================
' Boot
' ========================================================================
boot_start:
    MODE 1
    CLS
    GOSUB fill_bg
    GOSUB gfx_init

    PRINT AT 0 COLOR COL_STATUS, "5 CARD STUD"
    PRINT AT 20, "IN GAME, PRESS KEYPAD"
    PRINT AT 40, "CLEAR FOR THE MENU"
    PRINT AT 60, "CONNECTING TO FUJINET"
    GOSUB fn_wait_mailbox
    IF fn_ok = 0 THEN
        PRINT AT 60, "NO CARTRIDGE MAILBOX"
        GOTO halt
    END IF
```

```
' ------------------------------------------------------------------------
' Name: try the lobby appkey first; fall back to the disc letter picker if
' the transaction fails or the stored name is empty.
' ------------------------------------------------------------------------
    ' Every other network call in this game gets an implicit retry via the
    ' outer poll loop; this one-shot boot-time appkey read didn't, so a
    ' single cold-connection timeout (the RP2040/ESP32 link needing a
    ' moment right after the mailbox first comes up -- the same thing
    ' that made the very first transaction after boot occasionally time
    ' out earlier in testing) meant falling back to manual name entry for
    ' the whole session even though a name was genuinely stored. Retry
    ' once before giving up.
    gs_i = 0
    FOR ak_try = 0 TO 1 (1)
        ak_creator_lo = 1 : ak_creator_hi = 0 : ak_app = 1 : ak_key = 0 :
ak_mode = 0
        GOSUB appkey_open
        IF fn_ok THEN EXIT FOR
    NEXT ak_try
    IF fn_ok THEN
        #fn_src = SC_NAME : ls_max = 9 : GOSUB appkey_read
        GOSUB appkey_close
        IF fn_len > 0 THEN
            ' Validate every character -- the appkey slot (creator=1,
            ' app=1, key=0) is the *shared* lobby username used by every
            ' FujiNet client on this server, so it can hold stale or
            ' outright incompatible data left by other testing (this
            ' repo's own earlier test runs included). Our name-entry
            ' screen can only ever produce A-Z/0-9, so anything outside
            ' that -- a space, punctuation, an '&' or '?' -- is untrusted:
            ' it goes straight into the /state URL's query string
            ' unescaped, and a stray reserved character there is exactly
            ' the kind of thing that gets the request rejected by the
            ' server (seen as an HTTP 400 page landing in FN_RX, which
            ' render_game then tries to draw as if it were game data).
            gs_i = 1
            FOR gs_j = 0 TO fn_len - 1
                gs_char = PEEK(SC_NAME + gs_j) AND 255
                IF gs_char >= 97 AND gs_char <= 122 THEN gs_char = gs_char - 32
' fold to uppercase for the check
                IF (gs_char < 65 OR gs_char > 90) AND (gs_char < 48 OR gs_char >
57) THEN gs_i = 0
            NEXT gs_j
        END IF
    END IF
    IF gs_i = 0 THEN GOSUB name_entry_screen

' ------------------------------------------------------------------------
' Room: check the lobby server appkey (creator 1 / app 1 / key
' AK_LOBBY_KEY_SERVER). If the lobby wrote a room there, split it into
' SC_ENDPT/SC_TABLE and skip straight past table select. Seed the
' compiled-in default endpoint first so SC_ENDPT is always valid even if
' the appkey is empty or the read fails -- compose_url relies on it from
' here on for every request, not just /tables.
' ------------------------------------------------------------------------
    FOR gs_i = 0 TO LEN_HTTPS - 1
        POKE (SC_ENDPT + gs_i), PEEK(VARPTR lit_https(0) + gs_i) AND 255
    NEXT gs_i
    POKE (SC_ENDPT + LEN_HTTPS), 0
    POKE SC_TABLE, 0

    FOR ak_try = 0 TO 1 (2)
        ak_creator_lo = 1 : ak_creator_hi = 0 : ak_app = 1
        ak_key = AK_LOBBY_KEY_SERVER : ak_mode = 0
        GOSUB appkey_open
        IF fn_ok THEN EXIT FOR
    NEXT ak_try
    IF fn_ok THEN
        #fn_src = SC_ENDPT : ls_max = 65 : GOSUB appkey_read
        GOSUB appkey_close
        IF fn_len > 0 THEN
            GOSUB split_room_url
        ELSE
            ' appkey_read always NUL-terminates at fn_len, even when
            ' empty, which would otherwise wipe out the default just
            ' seeded above -- restore it.
            FOR gs_i = 0 TO LEN_HTTPS - 1
                POKE (SC_ENDPT + gs_i), PEEK(VARPTR lit_https(0) + gs_i) AND 255
            NEXT gs_i
            POKE (SC_ENDPT + LEN_HTTPS), 0
        END IF
    END IF

    IF (PEEK(SC_TABLE) AND 255) <> 0 THEN GOTO table_joined

' ========================================================================
' Table select (re-entered after leaving a table)
' ========================================================================
table_select:
    CLS
    GOSUB fill_bg
    PRINT AT 0 COLOR COL_STATUS, "CHOOSE A TABLE"
    PRINT AT 40, "REFRESHING..."

    gs_path = 0 : GOSUB compose_url
    #net_readlen = TABLES_MAXLEN
    GOSUB api_call

    ' As with /state: the mailbox transaction can report success (the
    ' RP2040 relayed *a* reply) even when the HTTP request itself was
    ' rejected -- an error page's bytes would otherwise get misread as a
    ' Tables struct. Validate the response length against what the
    ' reported count actually implies before trusting it.
```

```
    IF fn_ok = 1 THEN 4

        #tmp_expect = 1 + (PEEK(FN_RX) AND 255) * TABLE_STRIDE
        IF #net_gotlen < #tmp_expect THEN fn_ok = 0
    END IF

    IF fn_ok = 0 THEN
        PRINT AT 40, "SERVER UNREACHABLE  "
        ' TEMP DEBUG: show the exact outgoing URL so a repeat failure is
        ' diagnosable from a screenshot instead of needing another round trip.
        #df_src = FN_TX : df_pos = 60 : df_len = 20 : #df_color = COL_STATUS
        GOSUB draw_field
        #df_src = FN_TX + 20 : df_pos = 80 : df_len = 20 : #df_color =
COL_STATUS
        GOSUB draw_field
        #df_src = FN_TX + 40 : df_pos = 100 : df_len = 20 : #df_color =
COL_STATUS
        GOSUB draw_field
        poll_wait = 90
        WHILE poll_wait > 0
            poll_wait = poll_wait - 1
            WAIT
        WEND
        GOTO table_select
    END IF

    tbl_count = PEEK(FN_RX) AND 255
    IF tbl_count > 6 THEN tbl_count = 6
    IF tbl_count = 0 THEN
        PRINT AT 40, "NO TABLES AVAILABLE "
        poll_wait = 90
        WHILE poll_wait > 0
            poll_wait = poll_wait - 1
            WAIT
        WEND
        GOTO table_select
    END IF

    CLS
    GOSUB fill_bg
    PRINT AT 0 COLOR COL_STATUS, "CHOOSE A TABLE"
    FOR gs_i = 0 TO tbl_count - 1
        #tmp_addr = table_addr(gs_i)
        #df_src = #tmp_addr + TBL_NAME : df_pos = 20 + gs_i * 20 + 1 : df_len =
14 : #df_color = COL_NAME
        GOSUB draw_field
        #df_src = #tmp_addr + TBL_PLAYERS : df_pos = 20 + gs_i * 20 + 15 :
df_len = 5 : #df_color = COL_NAME
        GOSUB draw_field
    NEXT gs_i

    tbl_sel = 0
    inp_lock = 0

tbl_input:
    WAIT
    FOR gs_i = 0 TO tbl_count - 1
        #gs_c = COL_NAME
        IF gs_i = tbl_sel THEN #gs_c = COL_HILITE
        #BACKTAB(20 + gs_i * 20) = (62 - 32) * 8 + #gs_c ' '>' cursor glyph
    NEXT gs_i

    IF inp_lock > 0 THEN inp_lock = inp_lock - 1 : GOTO tbl_input

    IF CONT1.DOWN THEN
        tbl_sel = tbl_sel + 1
        IF tbl_sel >= tbl_count THEN tbl_sel = 0
        inp_lock = 8
        GOSUB sound_cursor
        GOTO tbl_input
    END IF
    IF CONT1.UP THEN
        IF tbl_sel = 0 THEN tbl_sel = tbl_count
        tbl_sel = tbl_sel - 1
        inp_lock = 8
        GOSUB sound_cursor
        GOTO tbl_input
    END IF
    IF CONT1.BUTTON = 0 THEN GOTO tbl_input
    GOSUB sound_select

    #tmp_addr = table_addr(tbl_sel)
    FOR gs_i = 0 TO 8
        POKE (SC_TABLE + gs_i), PEEK(#tmp_addr + TBL_ID + gs_i) AND 255
    NEXT gs_i

    ' Update the lobby server appkey so a reboot without going back
    ' through the lobby rejoins this table.
    GOSUB write_room_appkey

table_joined:
' ============================================================================
' Main game loop
' ============================================================================
    GOSUB sound_join
    has_move = 0
    poll_wait = 0
    force_redraw = 0
    prev_round = 255      ' sentinel: forces a full CLS on the first render_game
    prev_playercount = 255
    #prev_pot = 65535
    #prev_purse = 65535
    #prev_result_hash = 65535 ' sentinel: max possible real hash is 20*255=5100
    first_poll = 1 ' arm the baseline on the first poll instead of showing it --
                   ' lastResult can already hold a message from a hand that
                   ' finished before we joined/sat down, which isn't "new"
    FOR gs_i = 0 TO 7
```

```
508         prev_bet(gs_i) = 255
509         prev_cards(gs_i) = 0
510     NEXT gs_i
511
512 game_loop:
513     WAIT
514
515     IF poll_wait > 0 THEN
516         poll_wait = poll_wait - 1
517         GOTO input_check
518     END IF
519
520     IF has_move THEN
521         gs_path = 2
522     ELSE
523         gs_path = 1
524     END IF
525     GOSUB compose_url
526     #net_readlen = GAME_MAXLEN
527     GOSUB api_call
528     has_move = 0
529
530     ' A short read means FN_RX beyond #net_gotlen still holds stale bytes
531     ' from a previous, larger response -- render_game trusts fixed offsets
532     ' deep into the buffer, so rendering it would show corrupted-looking
533     ' data (a garbage pot/bet value, or a stray card glyph) instead of
534     ' just skipping this poll. GAME_MINLEN alone isn't tight enough: it's
535     ' only the 0-player floor, so a 7-8 player game's response could land
536     ' anywhere from 154 bytes up to its true ~385+ bytes and still pass
537     ' that check while genuinely truncated partway through the player
538     ' array. playerCount (offset 153) is read from before where any
539     ' truncation could have happened, so it's safe to use here to compute
540     ' the *exact* expected length for this specific response.
541     ' A malformed request (e.g. a stray character somewhere it shouldn't
542     ' be) can get rejected by the server with an HTTP error page instead
543     ' of a Game struct -- our mailbox transaction still reports "OK" (the
544     ' RP2040 got *a* reply, just not the one we wanted), so round/
545     ' playerCount are the cheapest smoke test: neither should ever be
546     ' implausible for a real response, and checking them here means a
547     ' bogus response gets treated as a failed poll instead of being
548     ' handed to render_game as if it were valid.
549     IF fn_ok = 1 AND #net_gotlen >= GAME_MINLEN THEN
550         IF (PEEK(FN_RX + GAME_ROUND) AND 255) > 5 OR (PEEK(FN_RX +
GAME_PLAYERCOUNT) AND 255) > 8 THEN fn_ok = 0
551     END IF
552
553     IF fn_ok = 1 AND #net_gotlen >= GAME_MINLEN THEN
554         #tmp_expect = GAME_PLAYERS + (PEEK(FN_RX + GAME_PLAYERCOUNT) AND 255) *
PLAYER_STRIDE
555         IF #net_gotlen < #tmp_expect THEN fn_ok = 0
556     END IF
557
558     IF fn_ok = 0 OR #net_gotlen < GAME_MINLEN THEN
559         poll_wait = 30
560         GOTO input_check
561     END IF
562
563     ' Render *before* checking for a new result message: this is the one
564     ' poll whose response actually carries the showdown's revealed hands
565     ' (folded/hidden "??" flips to the real cards only for this response --
566     ' by the next poll the server has usually already started a new hand
567     ' and reset them). Rendering first means the reveal lands on screen
568     ' before the message overlay below covers the bottom two rows; doing
569     ' it the other way around (as before) meant the reveal was drawn *at
570     ' the earliest* 4 seconds later, by which point it was already gone,
571     ' so it never actually appeared.
572     GOSUB render_game
573
574     ' The server sends a one-shot message (e.g. "Fry BOT won with Pair,
575     ' Sixes") in lastResult at the end of a round/hand, but it's normally
576     ' only shown in the single-line status row -- easy to miss entirely
577     ' since bots move fast and the round can already have advanced past
578     ' it by the next poll. Detect a new (changed, non-empty) message via
579     ' a cheap byte-sum "hash" and, when one shows up, black out the
580     ' bottom two rows to display it prominently and hold it there for a
581     ' few seconds before resuming play.
582     #tmp_hash = 0
583     FOR gs_i = 0 TO 19
584         #tmp_hash = #tmp_hash + (PEEK(FN_RX + GAME_LASTRESULT + gs_i) AND 255)
585     NEXT gs_i
586     IF first_poll THEN
587         first_poll = 0
588         #prev_result_hash = #tmp_hash ' arm the baseline silently; whatever's
589                                       ' already there predates us sitting down
590     ELSEIF (PEEK(FN_RX + GAME_LASTRESULT) AND 255) <> 0 AND #tmp_hash <>
#prev_result_hash THEN
591         #prev_result_hash = #tmp_hash
592         FOR gs_i = STATUS_ROW - ROWCELLS TO STATUS_ROW + ROWCELLS - 1
593             #BACKTAB(gs_i) = COL_STATUS
594         NEXT gs_i
595         #df_src = FN_RX + GAME_LASTRESULT : df_pos = STATUS_ROW - ROWCELLS :
df_len = 2 * ROWCELLS : #df_color = COL_STATUS
596         GOSUB draw_field
597         GOSUB sound_gamedone
598         force_redraw = 1 ' restore the felt background under row 10 once play
resumes
599         poll_wait = 240
600         GOTO input_check
601     END IF
602     poll_wait = 20
603
604     IF active_player = 0 AND (PEEK(FN_RX + GAME_VIEWING) AND 255) = 0 THEN
605         GOSUB move_ui
606         IF has_move THEN poll_wait = 0
```

```
    END IF

input_check:
    IF CONT1.KEY = 10 THEN GOSUB ingame_menu
    IF CONT1.KEY = 11 THEN GOSUB show_purses
    IF want_leave THEN
        want_leave = 0
        GOTO table_select
    END IF

    GOTO game_loop

' ============================================================================
' render_game: full redraw of the table from the Game struct in FN_RX.
' ============================================================================
' ----------------------------------------------------------------------------
' render_game: redraw the table from the Game struct in FN_RX.
'
' IntyBASIC/the STIC have no true page-flip (unlike the C clients' Apple2/
' C64 double buffer or CoCo/MSX's SINGLE_BUFFER differential-only mode) --
' BACKTAB is read live every frame, so a CLS visibly blanks the screen for
' a frame before the redraw lands. Since state only changes ~once per poll
' (not per frame), the fix that matters is simply not clearing the whole
' screen on every poll: a full CLS only happens on the first render after
' entering the game, or when the player count or hand round regresses
' (i.e. a new hand started) -- both mean the previous frame's layout is no
' longer valid. Every other poll only touches the specific cells whose
' values can actually change (pot digits, bet digits, name/card cells,
' status row), matching the CoCo/MSX approach since we're in the same
' no-real-double-buffer boat they were.
' ----------------------------------------------------------------------------
render_game: PROCEDURE

    round = PEEK(FN_RX + GAME_ROUND) AND 255
    tmp_pc = PEEK(FN_RX + GAME_PLAYERCOUNT) AND 255

    IF (tmp_pc <> prev_playercount) OR (round < prev_round) OR force_redraw THEN
        CLS
        GOSUB fill_bg
        ' Center block, rows 5-6: pot ("$" + value) and your own purse ("P"
        ' + value) stacked directly beneath it. Both stay within cols 8-12
        ' -- the one column range no seat's card art ever reaches at any
        ' player count (the nearest cards, seat 2/6's, start at col 0/14).
        PRINT AT 108 COLOR COL_NAME, "$"
        PRINT AT 128 COLOR COL_NAME, "P"
        #prev_pot = 65535   ' force the pot to redraw too on a full layout reset
        #prev_purse = 65535 ' likewise for purse
        force_redraw = 0
        ' A new hand means every seat's hand is starting over -- without
        ' this, a seat that had (say) 3 cards last hand would treat this
        ' hand's first 3 cards as "already dealt" and skip animating them.
        IF round < prev_round THEN
            FOR gs_i = 0 TO 7
                prev_cards(gs_i) = 0
            NEXT gs_i
        END IF
    END IF

    ' prev_playercount=255 is the boot sentinel (first render ever) -- skip
    ' the join/leave cue then, there's nothing to compare against yet.
    IF prev_playercount <> 255 THEN
        IF tmp_pc > prev_playercount THEN GOSUB sound_player_join
        IF tmp_pc < prev_playercount THEN GOSUB sound_player_left
    END IF
    IF round < prev_round THEN GOSUB sound_deal

    prev_playercount = tmp_pc
    prev_round = round

    ' Real hardware/accurate emulation only guarantees a BACKTAB write is
    ' tear-free if it lands within the vblank window; a full-table redraw
    ' (dozens of cells plus several PRINT calls, each involving a slow
    ' division loop for the digits) can run long enough to spill into
    ' active display, and a live screenshot can catch that half-written
    ' state -- seen as a stray non-digit glyph where only a digit could
    ' ever legitimately be. Skipping the PRINT entirely when the value
    ' hasn't changed since the last poll is what keeps the common-case
    ' redraw (most fields static from one poll to the next) small enough
    ' to actually fit in one vblank, rather than trying to chase tearing
    ' after the fact.
    #tmp_num = u16be(FN_RX + GAME_POT)
    IF #tmp_num <> #prev_pot THEN
        IF #tmp_num > #prev_pot AND #prev_pot <> 65535 THEN GOSUB sound_chip
        PRINT AT 109 COLOR COL_NAME, <.4>#tmp_num
        #prev_pot = #tmp_num
    END IF

    ' Your own purse (wire index 0 is always you, per the server's rotation
    ' -- see state.bas), directly under the pot now that collapsing POT to
    ' one line freed row 6 for it.
    #tmp_num = u16be(player_addr(0) + PL_PURSE)
    IF #tmp_num <> #prev_purse THEN
        PRINT AT 129 COLOR COL_NAME, <.4>#tmp_num
        #prev_purse = #tmp_num
    END IF


    gs_j = tmp_pc - 2
    IF gs_j < 0 THEN gs_j = 0
    IF gs_j > 6 THEN gs_j = 6

    FOR gs_i = 0 TO tmp_pc - 1
        sel_seat = PEEK(VARPTR seatmap(0) + gs_j * 8 + gs_i) AND 255
        IF sel_seat <> 255 THEN
            sel_i = PEEK(VARPTR seat_name_off(0) + sel_seat) AND 255
```

```
            #gs_c = COL_NAME
            IF gs_i = active_player THEN #gs_c = COL_HILITE

            #tmp_addr = player_addr(gs_i)
            #df_src = #tmp_addr + PL_NAME : df_pos = sel_i : df_len = 4 :
#df_color = #gs_c
            GOSUB draw_field

            #tmp_num = u16be(#tmp_addr + PL_BET)
            IF #tmp_num > 255 THEN #tmp_num = 255 ' clamp: prev_bet is an 8-bit
array
            IF #tmp_num <> prev_bet(sel_seat) THEN
                PRINT AT sel_i + 4 COLOR #gs_c, <2>#tmp_num
                prev_bet(sel_seat) = #tmp_num
            END IF

            ' Cards fill each hand contiguously from index 0 (never a gap),
            ' so the count currently dealt is just 1 + the highest nonzero
            ' index. Comparing that against what this seat showed last
            ' poll (prev_cards) is what tells a genuinely new card apart
            ' from one we've already drawn on every previous poll.
            deal_count = 0
            FOR mv_sel = 0 TO 4
                card = PEEK(#tmp_addr + PL_HAND + mv_sel * 2) AND 255
                IF card <> 0 THEN deal_count = mv_sel + 1
            NEXT mv_sel

            FOR mv_sel = 0 TO deal_count - 1
                card = PEEK(#tmp_addr + PL_HAND + mv_sel * 2) AND 255
                suit = PEEK(#tmp_addr + PL_HAND + mv_sel * 2 + 1) AND 255
                ' A card index this seat hasn't shown before is a fresh
                ' deal -- the click (and its own built-in pause) stands in
                ' for the "card landing on the table" beat, instead of
                ' every new card just popping in silently and instantly
                ' like the C clients' animated deal never happens here.
                IF mv_sel >= prev_cards(sel_seat) THEN GOSUB sound_deal
                ' card=0/suit=0 after conversion (an unrecognized wire
                ' char, e.g. "??" for a folded/hidden hand) is itself a
                ' valid "hidden" input to print_card -- draws the card-
                ' back glyph, which is what keeps a folded/hidden card
                ' from leaving stale rank art on screen instead of just
                ' going quiet.
                conv_in = card : GOSUB card_from_ascii : card = conv_out
                conv_in = suit : GOSUB suit_from_ascii : suit = conv_out
                p = sel_i + 20 + mv_sel
                GOSUB print_card
            NEXT mv_sel
            prev_cards(sel_seat) = deal_count

            ' One seat's worth of pokes (name + bet digits + up to 5 cards)
            ' is small enough to reliably land within a single vblank; the
            ' full ~8-seat redraw as one unbroken burst isn't, and that's
            ' what let a screenshot catch a genuinely half-written frame
            ' (garbage that looked like data corruption but wasn't -- nothing
            ' in FN_RX was wrong, the *display* was mid-update). Yielding a
            ' frame between seats bounds the tear to at most one seat's
            ' fields at a time instead of the whole table, at the cost of
            ' a full new-hand redraw taking up to ~8 extra frames (moot;
            ' that only happens once per hand, not every poll).
            WAIT
        END IF
    NEXT gs_i

    GOSUB draw_status
END

' ---------------------------------------------------------------------------
' show_purses: while KEYPAD ENTER is held, overwrite every seated player's
' name cells with their purse value instead. Reuses tmp_pc/seatmap/
' seat_name_off exactly as render_game's own seat loop does -- tmp_pc is a
' global left holding the last poll's player count, so this works between
' polls without needing a fresh network round-trip. Releasing the key just
' asks for a full redraw on the next poll (the same trick the win-message
' overlay uses) rather than restoring each name field by hand here.
' ---------------------------------------------------------------------------
show_purses: PROCEDURE
    gs_j = tmp_pc - 2
    IF gs_j < 0 THEN gs_j = 0
    IF gs_j > 6 THEN gs_j = 6

    FOR gs_i = 0 TO tmp_pc - 1
        sel_seat = PEEK(VARPTR seatmap(0) + gs_j * 8 + gs_i) AND 255
        IF sel_seat <> 255 THEN
            sel_i = PEEK(VARPTR seat_name_off(0) + sel_seat) AND 255
            #tmp_addr = player_addr(gs_i)
            #tmp_num = u16be(#tmp_addr + PL_PURSE)
            IF #tmp_num > 9999 THEN #tmp_num = 9999
            PRINT AT sel_i COLOR COL_HILITE, <.4>#tmp_num
        END IF
    NEXT gs_i

sp_wait:
    WAIT
    IF CONT1.KEY = 11 THEN GOTO sp_wait

    force_redraw = 1 ' restore names (and everything else) on the next poll
END

' ---------------------------------------------------------------------------
' card_from_ascii / suit_from_ascii: translate the wire's lowercase ASCII
' rank/suit chars (conv_in) into print_card's expected numeric codes
' (conv_out; 0 = unrecognized/blank).
' ---------------------------------------------------------------------------
card_from_ascii: PROCEDURE
    IF conv_in = 50 THEN
        conv_out = 2
```

```
    ELSEIF conv_in = 51 THEN
        conv_out = 3
    ELSEIF conv_in = 52 THEN
        conv_out = 4
    ELSEIF conv_in = 53 THEN
        conv_out = 5
    ELSEIF conv_in = 54 THEN
        conv_out = 6
    ELSEIF conv_in = 55 THEN
        conv_out = 7
    ELSEIF conv_in = 56 THEN
        conv_out = 8
    ELSEIF conv_in = 57 THEN
        conv_out = 9
    ELSEIF conv_in = 116 THEN
        conv_out = 10
    ELSEIF conv_in = 106 THEN
        conv_out = 11
    ELSEIF conv_in = 113 THEN
        conv_out = 12
    ELSEIF conv_in = 107 THEN
        conv_out = 13
    ELSEIF conv_in = 97 THEN
        conv_out = 14
    ELSE
        conv_out = 0
    END IF
END

suit_from_ascii: PROCEDURE
    IF conv_in = 100 THEN
        conv_out = DIAMONDS
    ELSEIF conv_in = 104 THEN
        conv_out = HEARTS
    ELSEIF conv_in = 99 THEN
        conv_out = CLUBS
    ELSEIF conv_in = 115 THEN
        conv_out = SPADES
    ELSE
        conv_out = 0
    END IF
END

' ------------------------------------------------------------------------
' draw_status: status row (row 11). Your turn is handled separately by
' move_ui (which overwrites this row); otherwise show "WAIT <name>" while
' someone else acts, or the truncated lastResult otherwise.
' ------------------------------------------------------------------------
draw_status: PROCEDURE
    IF active_player > 0 THEN
        PRINT AT STATUS_ROW COLOR COL_STATUS, "WAIT "
        #tmp_addr = player_addr(active_player)
        #df_src = #tmp_addr + PL_NAME : df_pos = STATUS_ROW + 5 : df_len = 4 :
#df_color = COL_STATUS
        GOSUB draw_field
        ' "WAIT <name>" only fills 9 of the row's 20 cells -- without a
        ' full CLS between polls, blank out the rest so a previous,
        ' longer status line (the lastResult branch below, or a move
        ' menu) can't leave stale characters past column 8.
        FOR gs_i = STATUS_ROW + 9 TO STATUS_ROW + ROWCELLS - 1
            #BACKTAB(gs_i) = COL_STATUS
        NEXT gs_i
    ELSE
        #df_src = FN_RX + GAME_LASTRESULT : df_pos = STATUS_ROW : df_len =
ROWCELLS : #df_color = COL_STATUS
        GOSUB draw_field
    END IF
END

' ========================================================================
' move_ui: your turn. List valid moves on the status row, disc left/right
' to choose, action button to submit. Sets has_move + mvcode_a/mvcode_b.
' ========================================================================
move_ui: PROCEDURE

    mv_count = PEEK(FN_RX + GAME_VALIDMOVECOUNT) AND 255
    IF mv_count > 5 THEN mv_count = 5
    IF mv_count = 0 THEN RETURN

    mv_sel = 0
    IF mv_count > 1 THEN mv_sel = 1 ' default to the second move (not Fold),
matching the C client
    GOSUB sound_myturn

    ' Without a per-poll CLS, the status row can be carrying over a
    ' previous, wider draw_status message (lastResult fills all 20
    ' cells) -- move_ui only pokes the exact columns its move names
    ' occupy, so blank the whole row once up front rather than leaving
    ' stale text past the last move.
    FOR gs_i = STATUS_ROW TO STATUS_ROW + ROWCELLS - 1
        #BACKTAB(gs_i) = COL_STATUS
    NEXT gs_i
    ' Stopwatch icon (cardbot's 7th/last glyph, screen code 276 -- the
    ' only one of that set that's a clock face) in the last 3 columns of
    ' the move row, poked directly rather than through print_card since
    ' this isn't a playing card.
    #BACKTAB(STATUS_ROW + 17) = 276 * 8 + COL_STATUS

    ' moveTime (server-computed, already net of a network round-trip
    ' grace period -- see gameLogic.go) is our countdown's starting
    ' point; we count it down locally frame-by-frame rather than
    ' re-polling the server every second.
    mv_timeleft = PEEK(FN_RX + GAME_MOVETIME) AND 255
    mv_framecount = 0
    PRINT AT STATUS_ROW + 18 COLOR COL_STATUS, <2>mv_timeleft

```

```
    inp_lock = 0
mu_loop:
    gs_j = 0
    FOR gs_i = 0 TO mv_count - 1
        mv_col(gs_i) = gs_j
        #gs_c = COL_STATUS
        IF gs_i = mv_sel THEN #gs_c = COL_HILITE
        #tmp_addr = move_addr(gs_i)
        #df_src = #tmp_addr + MOVE_NAME : df_pos = STATUS_ROW + gs_j : df_len =
4 : #df_color = #gs_c
        GOSUB draw_field
        gs_j = gs_j + 5
        IF gs_j > 12 THEN gs_j = 12 ' clamp so the last move name never reaches
the stopwatch/timer at cols 17-19
    NEXT gs_i

    WAIT

    mv_framecount = mv_framecount + 1
    IF mv_framecount >= 60 THEN
        mv_framecount = 0
        IF mv_timeleft > 0 THEN mv_timeleft = mv_timeleft - 1
        PRINT AT STATUS_ROW + 18 COLOR COL_STATUS, <2>mv_timeleft
        ' Timed out -- submit whatever's currently highlighted (the
        ' default move if the player never touched the disc, matching
        ' the C clients' timeout behavior of submitting the highlighted
        ' selection rather than always folding).
        IF mv_timeleft = 0 THEN GOTO mu_confirm
    END IF

    IF inp_lock > 0 THEN inp_lock = inp_lock - 1 : GOTO mu_loop

    IF CONT1.RIGHT THEN
        mv_sel = mv_sel + 1
        IF mv_sel >= mv_count THEN mv_sel = 0
        inp_lock = 8
        GOSUB sound_cursor
        GOTO mu_loop
    END IF
    IF CONT1.LEFT THEN
        IF mv_sel = 0 THEN mv_sel = mv_count
        mv_sel = mv_sel - 1
        inp_lock = 8
        GOSUB sound_cursor
        GOTO mu_loop
    END IF
    IF CONT1.KEY = 10 THEN RETURN ' bail to in-game menu without moving
    IF CONT1.BUTTON = 0 THEN GOTO mu_loop
    GOSUB sound_select

mu_confirm:
    #tmp_addr = move_addr(mv_sel)
    mvcode_a = PEEK(#tmp_addr + MOVE_CODE) AND 255
    mvcode_b = PEEK(#tmp_addr + MOVE_CODE + 1) AND 255
    has_move = 1
END

' ============================================================================
' ingame_menu: keypad CLEAR overlay. Disc up/down to choose, action button
' to confirm, matching every other screen's controls (Clear itself also
' cancels, for a quick way back in without touching the disc). Defaults
' to RESUME so an accidental Clear press can't quit a hand by surprise.
' ============================================================================
ingame_menu: PROCEDURE
    im_sel = 0
    inp_lock = 0
im_loop:
    PRINT AT 80 COLOR COL_STATUS, "                    "
    PRINT AT 100 COLOR COL_STATUS, "                    "
    PRINT AT 120 COLOR COL_STATUS, "                    "
    PRINT AT 86 COLOR COL_STATUS, "TABLE MENU"
    #gs_c = COL_STATUS
    IF im_sel = 0 THEN #gs_c = COL_HILITE
    PRINT AT 106 COLOR #gs_c, "RESUME"
    #gs_c = COL_STATUS
    IF im_sel = 1 THEN #gs_c = COL_HILITE
    PRINT AT 126 COLOR #gs_c, "QUIT TABLE"

    WAIT
    IF inp_lock > 0 THEN inp_lock = inp_lock - 1 : GOTO im_loop

    IF CONT1.UP OR CONT1.DOWN THEN
        im_sel = 1 - im_sel
        inp_lock = 10
        GOSUB sound_cursor
        GOTO im_loop
    END IF
    IF CONT1.KEY = 10 THEN GOTO im_done ' Clear again cancels straight back to
RESUME
    IF CONT1.BUTTON = 0 THEN GOTO im_loop
    GOSUB sound_select

    IF im_sel = 1 THEN
        gs_path = 3 : GOSUB compose_url
        #net_readlen = 8
        GOSUB api_call
        GOSUB clear_room_appkey
        want_leave = 1
    END IF
im_done:
    force_redraw = 1
END

' ============================================================================
' name_entry_screen: disc letter picker, max 8 chars. Disc up/down cycles
' the character under the cursor through A-Z, 0-9, space; left/right moves
```

```
' the cursor; the action button accepts (at least 1 non-space char).
' ============================================================================
name_entry_screen: PROCEDURE
    CLS
    GOSUB fill_bg
    PRINT AT 0 COLOR COL_STATUS, "ENTER YOUR NAME"
    FOR ne_i = 0 TO 7
        ne_buf(ne_i) = 36 ' space
    NEXT ne_i
    ne_cur = 0
    inp_lock = 0

ne_loop:
    FOR ne_i = 0 TO 7
        #gs_c = COL_NAME
        IF ne_i = ne_cur THEN #gs_c = COL_HILITE
        ne_j = ne_buf(ne_i)
        IF ne_j < 26 THEN
            gs_j = 65 + ne_j
        ELSEIF ne_j < 36 THEN
            gs_j = 48 + ne_j - 26
        ELSE
            gs_j = 95 ' underscore stands in for a visible blank
        END IF
        #BACKTAB(60 + 6 + ne_i) = (gs_j - 32) * 8 + #gs_c
    NEXT ne_i

    WAIT
    IF inp_lock > 0 THEN inp_lock = inp_lock - 1 : GOTO ne_loop

    IF CONT1.RIGHT THEN
        ne_cur = ne_cur + 1
        IF ne_cur > 7 THEN ne_cur = 0
        inp_lock = 8
        GOSUB sound_cursor
        GOTO ne_loop
    END IF
    IF CONT1.LEFT THEN
        IF ne_cur = 0 THEN ne_cur = 8
        ne_cur = ne_cur - 1
        inp_lock = 8
        GOSUB sound_cursor
        GOTO ne_loop
    END IF
    IF CONT1.UP THEN
        ne_buf(ne_cur) = ne_buf(ne_cur) + 1
        IF ne_buf(ne_cur) > 36 THEN ne_buf(ne_cur) = 0
        inp_lock = 6
        GOSUB sound_cursor
        GOTO ne_loop
    END IF
    IF CONT1.DOWN THEN
        IF ne_buf(ne_cur) = 0 THEN ne_buf(ne_cur) = 37
        ne_buf(ne_cur) = ne_buf(ne_cur) - 1
        inp_lock = 6
        GOSUB sound_cursor
        GOTO ne_loop
    END IF
    IF CONT1.BUTTON = 0 THEN GOTO ne_loop
    GOSUB sound_select

    ne_len = 8
    WHILE ne_len > 0 AND ne_buf(ne_len - 1) = 36
        ne_len = ne_len - 1
    WEND
    IF ne_len = 0 THEN GOTO ne_loop

    FOR ne_i = 0 TO ne_len - 1
        ne_j = ne_buf(ne_i)
        IF ne_j < 26 THEN
            gs_j = 65 + ne_j
        ELSE
            gs_j = 48 + ne_j - 26
        END IF
        POKE (SC_NAME + ne_i), gs_j
    NEXT ne_i
    POKE (SC_NAME + ne_len), 0

    ak_creator_lo = 1 : ak_creator_hi = 0 : ak_app = 1 : ak_key = 0 : ak_mode =
1
    GOSUB appkey_open
    IF fn_ok THEN
        #fn_src = SC_NAME : fn_len = ne_len
        GOSUB appkey_write
        ' A failed write used to be silent -- looked identical to success
        ' from here, even though the name would never come back on the
        ' next boot (e.g. if the backend has no SD/appkey storage mounted).
        IF fn_ok = 0 THEN
            PRINT AT 100 COLOR COL_STATUS, "NAME NOT SAVED FOR "
            PRINT AT 120 COLOR COL_STATUS, "NEXT TIME           "
            poll_wait = 90
            WHILE poll_wait > 0
                poll_wait = poll_wait - 1
                WAIT
            WEND
        END IF
        GOSUB appkey_close
    END IF
END

halt:
    WAIT
    GOTO halt
```

LISTING 3 state.bas — 5 Card Stud wire format

```
' state.bas -- binary Game/Tables wire format, read in place out of FN_RX.
'
' Per the plan: never copy the reply into IntyBASIC variables. These are
' just fixed offsets (matching server/util.go's appendFixedLengthString
' binary layout, requested with ?bin=1 -- uint16 fields are little-endian,
' the server's default; &be=1 was dropped after pot/bet started showing
' up as an exact multiple of 256, the signature of a byte-order mismatch
' on the server's binary.BigEndian path, which is exercised by only one
' other client in the whole family (CoCo/6809) versus everything else
' being little-endian x86/6502/Z80)
' plus small DEF FN accessors that PEEK straight out of FN_RX.
'
' /state, /move/XX, /leave -> Game struct, up to 418 bytes:
'   0   lastResult[81]
'  81   round (1)              0=waiting 1-4=streets 5=showdown
'  82   pot (u16 LE)
'  84   activePlayer (signed, $FF = -1)
'  85   moveTime (1, seconds)
'  86   viewing (1, 0/1)
'  87   validMoveCount (1)
'  88   validMoves[5] -- always 5 slots, 13 bytes each: move[3] + name[10]
' 153   playerCount (1)
' 154   players[N] -- N = playerCount, 33 bytes each:
'         name[9] status(1) bet(u16 LE) move[8] purse(u16 LE) hand[11]
'
' /tables -> Tables struct:
'   0   count (1)
'   then N x 36 bytes: table[9] name[21] players[6] (literal "cur / max")

    CONST GAME_LASTRESULT     = 0
    CONST GAME_ROUND          = 81
    CONST GAME_POT            = 82
    CONST GAME_ACTIVEPLAYER   = 84
    CONST GAME_MOVETIME       = 85
    CONST GAME_VIEWING        = 86
    CONST GAME_VALIDMOVECOUNT = 87
    CONST GAME_VALIDMOVES     = 88
    CONST GAME_PLAYERCOUNT    = 153
    CONST GAME_PLAYERS        = 154

    CONST MOVE_STRIDE  = 13
    CONST MOVE_CODE    = 0   ' 3 bytes
    CONST MOVE_NAME    = 3   ' 10 bytes

    CONST PLAYER_STRIDE = 33
    CONST PL_NAME   = 0      ' 9 bytes
    CONST PL_STATUS = 9      ' 1 byte: 0 waiting, 1 playing, 2 folded, 3 left
    CONST PL_BET    = 10     ' u16 LE
    CONST PL_MOVE   = 12     ' 8 bytes
    CONST PL_PURSE  = 20     ' u16 LE
    CONST PL_HAND   = 22     ' 11 bytes (5 cards x 2 chars + NUL)

    CONST TABLE_STRIDE = 36
    CONST TBL_ID      = 0    ' 9 bytes
    CONST TBL_NAME    = 9    ' 21 bytes
    CONST TBL_PLAYERS = 30   ' 6 bytes, literal "cur / max"

    CONST GAME_MAXLEN   = 418
    CONST GAME_MINLEN   = 154  ' fixed prefix through playerCount, 0 players
    CONST TABLES_MAXLEN = 361

' u16be: read a little-endian uint16 at RAM address `addr` (see header note
' on why this is LE despite the name -- kept as u16be to avoid renaming
' every call site for what's ultimately a one-line fix).
    DEF FN u16be(addr) = (PEEK(addr + 1) AND 255) * 256 + (PEEK(addr) AND 255)

' player_addr(i): address of player i's 33-byte record in FN_RX.
    DEF FN player_addr(i) = FN_RX + GAME_PLAYERS + i * PLAYER_STRIDE

' move_addr(i): address of valid-move slot i's 13-byte record in FN_RX.
    DEF FN move_addr(i) = FN_RX + GAME_VALIDMOVES + i * MOVE_STRIDE

' table_addr(i): address of table i's 36-byte record in FN_RX (i is 0-based,
' the record right after the leading count byte).
    DEF FN table_addr(i) = FN_RX + 1 + i * TABLE_STRIDE

' active_player: activePlayer as a signed value (-1..7), converting the
' wire's $FF sentinel. Called as plain `active_player` (no parens -- DEF FN
' with no arguments is defined and invoked without them).
    DEF FN active_player = ((PEEK(FN_RX + GAME_ACTIVEPLAYER) AND 255) = 255) *
-256 + (PEEK(FN_RX + GAME_ACTIVEPLAYER) AND 255)

' draw_field is in 5card.bas -- it renders a field straight from FN_RX (or
' any RAM address) onto the screen in one pass (uppercase, NUL/pad-stop,
' ASCII->card-code), so there's no need to stage a separate ASCII copy here.

```

LISTING 4 gfx.bas — 5 Card Stud card art

```
' gfx.bas -- card GRAM bitmaps and draw routines.
'
' Card faces and the print_card/foreground_color rendering logic are ported
' verbatim from carlsson's intv/5card-test-display.bas mock. cardtop holds
' 14 GRAM images (index 0 = card back/blank, 1-13 = ranks 2..A), cardbot
' holds 7 (index 0 = blank, 1-4 = suit bottoms for DIAMONDS/HEARTS/CLUBS/
' SPADES, plus 2 more finishing the set) -- see print_card for the exact
' index math. DEFINE loads these into GRAM slots 0-13 (screen codes 256-269)
' and 14-20 (screen codes 270-276) respectively.

    CONST DIAMONDS = 1
    CONST HEARTS = 2
    CONST CLUBS = 3
    CONST SPADES = 4

' ---------------------------------------------------------------------------
' gfx_init: load both GRAM sets. Must run once at startup, each DEFINE
' followed by a WAIT (GRAM loads take effect on the next video frame; a
' second DEFINE in the same frame would silently overwrite the first).
' ---------------------------------------------------------------------------
gfx_init: PROCEDURE
    DEFINE 0, 14, cardtop : WAIT
    DEFINE 14, 7, cardbot : WAIT
END

' ---------------------------------------------------------------------------
' print_card: draw one card at screen cell p (top half) / p+20 (bottom
' half). Inputs: p (top-left cell), card (1-13, 0 = hidden/back), suit
' (1-4, DIAMONDS/HEARTS/CLUBS/SPADES).
' ---------------------------------------------------------------------------
print_card: PROCEDURE
    #col = BG_WHITE
    GOSUB foreground_color
    #BACKTAB(p) = #col + (card + 256) * 8
    #BACKTAB(p + 20) = #col + (suit + 270) * 8
END

' ---------------------------------------------------------------------------
' foreground_color: pick #col's foreground bits for the card being drawn
' (red for D/H, black for C/S, blue for a hidden card), and adjusts `card`
' from its wire value (2-14) down to the 1-13 GRAM index.
' ---------------------------------------------------------------------------
foreground_color: PROCEDURE
    IF card > 0 THEN
        card = card - 1 ' compensate for suit runs 23456789TJQKA
        IF suit = DIAMONDS OR suit = HEARTS THEN
            #col = #col + FG_RED
        ELSE
            #col = #col + FG_BLACK
        END IF
    ELSE
        #col = #col + FG_BLUE
    END IF
END

cardtop:
  BITMAP "oooooooo"
    BITMAP "oo..o..o"
  BITMAP "o.o..o.."
  BITMAP "o..o..o."
  BITMAP "oo..o..o"
  BITMAP "o.o..o.."
  BITMAP "o..o..o."
  BITMAP "oo..o..o"

  BITMAP "oooooooo"
    BITMAP "o......."
  BITMAP "o.oooo.."
  BITMAP "o.....o."
  BITMAP "o..ooo.."
  BITMAP "o.o....."
  BITMAP "o.ooooo."
  BITMAP "o......."

  BITMAP "oooooooo"
    BITMAP "o......."
  BITMAP "o.oooo.."
  BITMAP "o.....o."
  BITMAP "o..ooo.."
  BITMAP "o.....o."
  BITMAP "o.oooo.."
  BITMAP "o......."

  BITMAP "oooooooo"
    BITMAP "o......."
  BITMAP "o.o...o."
  BITMAP "o.o...o."
  BITMAP "o.ooooo."
  BITMAP "o.....o."
  BITMAP "o.....o."
  BITMAP "o......."

  BITMAP "oooooooo"
    BITMAP "o......."
  BITMAP "o.ooooo."
  BITMAP "o.o....."
  BITMAP "o.oooo.."
  BITMAP "o.....o."
  BITMAP "o.oooo.."
  BITMAP "o......."
```

```
101
102   BITMAP "oooooooo"
103     BITMAP "o......."
104   BITMAP "o...oo.."
105   BITMAP "o..o...."
106   BITMAP "o.oooo.."
107   BITMAP "o.o...o."
108   BITMAP "o..ooo.."
109   BITMAP "o......."
110
111   BITMAP "oooooooo"
112     BITMAP "o......."
113   BITMAP "o.ooooo."
114   BITMAP "o.....o."
115   BITMAP "o....o.."
116   BITMAP "o...o..."
117   BITMAP "o...o..."
118   BITMAP "o......."
119
120   BITMAP "oooooooo"
121     BITMAP "o......."
122   BITMAP "o..ooo.."
123   BITMAP "o.o...o."
124   BITMAP "o..ooo.."
125   BITMAP "o.o...o."
126   BITMAP "o..ooo.."
127   BITMAP "o......."
128
129   BITMAP "oooooooo"
130     BITMAP "o......."
131   BITMAP "o..ooo.."
132   BITMAP "o.o...o."
133   BITMAP "o..oooo."
134   BITMAP "o.....o."
135   BITMAP "o..ooo.."
136   BITMAP "o......."
137
138   BITMAP "oooooooo"
139     BITMAP "o......."
140   BITMAP "o.o..o.."
141   BITMAP "o.o.o.o."
142   BITMAP "o.o.o.o."
143   BITMAP "o.o.o.o."
144   BITMAP "o.o..o.."
145   BITMAP "o......."
146
147   BITMAP "oooooooo"
148     BITMAP "o......."
149   BITMAP "o.....o."
150   BITMAP "o.....o."
151   BITMAP "o.....o."
152   BITMAP "o.o...o."
153   BITMAP "o..ooo.."
154   BITMAP "o......."
155
156   BITMAP "oooooooo"
157     BITMAP "o......."
158   BITMAP "o..ooo.."
159   BITMAP "o.o...o."
160   BITMAP "o.o.o.o."
161   BITMAP "o.o..o.."
162   BITMAP "o..oo.o."
163     BITMAP "o......."
164
165   BITMAP "oooooooo"
166     BITMAP "o......."
167   BITMAP "o.o...o."
168   BITMAP "o.o..o.."
169   BITMAP "o.ooo..."
170   BITMAP "o.o..o.."
171   BITMAP "o.o...o."
172     BITMAP "o......."
173
174   BITMAP "oooooooo"
175   BITMAP "o......."
176   BITMAP "o..ooo.."
177   BITMAP "o.o...o."
178   BITMAP "o.ooooo."
179   BITMAP "o.o...o."
180   BITMAP "o.o...o."
181   BITMAP "o......."
182
183 cardbot:
184   BITMAP "o.o..o.."
185     BITMAP "o..o..o."
186   BITMAP "oo..o..o"
187   BITMAP "o.o..o.."
188   BITMAP "o..o..o."
189   BITMAP "oo..o..o"
190   BITMAP "o.o..o.."
191   BITMAP "oooooooo"
192
193   BITMAP "o......."
194     BITMAP "o...o..."
195   BITMAP "o..ooo.."
196   BITMAP "o.ooooo."
197   BITMAP "o..ooo.."
198   BITMAP "o...o..."
199   BITMAP "o......."
200   BITMAP "oooooooo"
201
202   BITMAP "o......."
203     BITMAP "o..o.o.."
204   BITMAP "o.ooooo."
205   BITMAP "o.ooooo."
206   BITMAP "o..ooo.."
```

```
  BITMAP "o...o..."
  BITMAP "o......."
  BITMAP "oooooooo"

  BITMAP "o......."
    BITMAP "o..ooo.."
  BITMAP "o.o.o.o."
  BITMAP "o.ooooo."
  BITMAP "o.o.o.o."
  BITMAP "o...o..."
  BITMAP "o......."
  BITMAP "oooooooo"

  BITMAP "o......."
    BITMAP "o..ooo.."
  BITMAP "o.ooooo."
  BITMAP "o.ooooo."
  BITMAP "o...o..."
  BITMAP "o...o..."
  BITMAP "o......."
  BITMAP "oooooooo"

  BITMAP "o......."
  BITMAP "o......."
  BITMAP "o......."
  BITMAP "o......."
  BITMAP "o......."
  BITMAP "o......."
  BITMAP "o......."
  BITMAP "o......."

  BITMAP ".ooooo.."
  BITMAP "ooo.ooo."
  BITMAP "ooo.ooo."
  BITMAP "ooo...o."
  BITMAP "ooooooo."
  BITMAP "ooooooo."
  BITMAP ".ooooo.."
  BITMAP "........"
```

LISTING 5 sound.bas — 5 Card Stud effects

```
' sound.bas -- sound effects, modeled on the C clients' platform-specific
' sound.c implementations (src/msx/sound.c and src/plus4/sound.c gave the
' clearest Hz-based reference; other platforms use raw PSG period/noise
' values that amount to the same tones). Every effect there is built from
' plain square-wave tones (SOUND channel 0-2, no envelope/noise), which is
' exactly IntyBASIC's SOUND statement, so this is a direct transcription
' rather than a reinterpretation.
'
' All effects play on PSG channel 0 -- the game is turn-based and effects
' are triggered at distinct, non-overlapping UI events, so there's no need
' for polyphony (and SOUND's channel argument must be a compile-time
' constant anyway, which would rule out a single generic multi-channel
' helper).
'
```

```
' SOUND's frequency argument is a 12-bit PSG divisor, computed as
' round(3579545/32/hz) for NTSC (the manual's documented formula;
' IntyBASIC exposes no runtime PAL/NTSC query outside the MUSIC/PLAY
' engine, and this project has only ever been built/tested NTSC). These
' are precomputed here rather than divided at runtime: 3579545/32 alone
' is ~111861, which doesn't fit IntyBASIC's 16-bit variables (max 65535),
' so it only works as a compile-time-folded literal division (a fixed Hz
' baked into the expression) -- not as CONST-divided-by-a-runtime-variable,
' which is what a single reusable "play Hz X" helper would need. Since
' every effect here uses fixed, known-in-advance frequencies anyway,
' precomputing sidesteps the overflow entirely and is cheaper besides.
'   50Hz->2237  60Hz->1864  70Hz->1598  80Hz->1398  100Hz->1119
'  150Hz->746  300Hz->373  311Hz->360  330Hz->339  340Hz->329
'  350Hz->320  392Hz->285  415Hz->270  430Hz->260  500Hz->224
    CONST SND_VOL = 12

    DIM #snd_val, snd_gate, snd_post, snd_i

' ------------------------------------------------------------------------
' play_tone: one square-wave note on channel 0. #snd_val = precomputed PSG
' divisor (see table above), snd_gate = frames the tone sounds, snd_post =
' frames of silence after (both "vblank" counts, taken directly from the
' reference implementations since they're already frame units at ~60Hz).
' ------------------------------------------------------------------------
play_tone: PROCEDURE
    SOUND 0, #snd_val, SND_VOL
    FOR snd_i = 1 TO snd_gate
        WAIT
    NEXT snd_i
    SOUND 0, 0, 0
    FOR snd_i = 1 TO snd_post
        WAIT
    NEXT snd_i
END

' sound_join: sit down at a table (3-tone rising figure: 430,340,500 Hz).
sound_join: PROCEDURE
    #snd_val = 260 : snd_gate = 5 : snd_post = 8 : GOSUB play_tone
    #snd_val = 329 : snd_gate = 5 : snd_post = 0 : GOSUB play_tone
    #snd_val = 224 : snd_gate = 5 : snd_post = 0 : GOSUB play_tone
END

' sound_myturn: it's your turn to move (double beep, 430,430 Hz).
sound_myturn: PROCEDURE
    #snd_val = 260 : snd_gate = 4 : snd_post = 2 : GOSUB play_tone
    #snd_val = 260 : snd_gate = 4 : snd_post = 2 : GOSUB play_tone
END

' sound_gamedone: round/showdown result just landed (4-tone rising
' fanfare: 311,330,392,415 Hz).
sound_gamedone: PROCEDURE
    #snd_val = 360 : snd_gate = 10 : snd_post = 0 : GOSUB play_tone
    #snd_val = 339 : snd_gate = 20 : snd_post = 0 : GOSUB play_tone
    #snd_val = 285 : snd_gate = 10 : snd_post = 0 : GOSUB play_tone
    #snd_val = 270 : snd_gate = 20 : snd_post = 0 : GOSUB play_tone
END

' sound_deal: a new hand's cards have arrived (150 Hz click). The C clients
' click once per card as each is dealt; we don't track per-card deal
' state (cards just render as soon as they're in the fetched hand), so
' this plays once per new deal instead of per card.
sound_deal: PROCEDURE
    #snd_val = 746 : snd_gate = 1 : snd_post = 5 : GOSUB play_tone
END

' sound_player_join: another player sits down mid-game (5-tone rising
' sweep: 50,60,70,80 Hz).
sound_player_join: PROCEDURE
    #snd_val = 2237 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
    #snd_val = 1864 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
    #snd_val = 1598 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
    #snd_val = 1398 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
END

' sound_player_left: a player leaves mid-game (falling sweep, mirror of join).
sound_player_left: PROCEDURE
    #snd_val = 1398 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
    #snd_val = 1598 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
    #snd_val = 1864 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
    #snd_val = 2237 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
END

' sound_select: a move or table selection was confirmed (2-tone rising
' chirp: 300,350 Hz).
sound_select: PROCEDURE
    #snd_val = 373 : snd_gate = 3 : snd_post = 1 : GOSUB play_tone
    #snd_val = 320 : snd_gate = 3 : snd_post = 0 : GOSUB play_tone
END

' sound_cursor: menu/move cursor moved one position (300 Hz).
sound_cursor: PROCEDURE
    #snd_val = 373 : snd_gate = 2 : snd_post = 0 : GOSUB play_tone
END

' sound_chip: pot value increased (a bet/call/raise landed, 50 Hz). The C
' clients play one ascending tone per player as chips sweep into the pot
' during a dedicated collection animation; we don't animate that (pot just
' updates), so this plays their first-player tone once per pot increase
' as a stand-in "chip clink."
sound_chip: PROCEDURE
    #snd_val = 2237 : snd_gate = 2 : snd_post = 2 : GOSUB play_tone
END

```

LISTING 6 battleship.bas — Battleship client (Chapter 9)

```
' battleship.bas -- FujiNet Battleship for Intellivision, entry point +
' state machine. Follows the pattern established by fujinet-5cardstud/intv/
' 5card.bas (name entry -> table select -> game loop -> in-game menu), but
' the board itself is rendered with board.bas's colored-squares kernel
' instead of GRAM card art, since four 10x10 grids need to be on screen at
' once (Battleship's core multiplayer rule is that one shot hits every
' opposing board simultaneously).
'
' GOTO boot_start jumps over every INCLUDE below before any of it runs --
' see fujinet.bas/5card.bas for why: falling into a PROCEDURE or DATA block
' by straight-line execution corrupts the return stack.
    GOTO boot_start

    INCLUDE "constants.bas"
    INCLUDE "fujinet.bas"
    INCLUDE "state.bas"
    INCLUDE "board.bas"
    INCLUDE "sound.bas"

    CONST ROWCELLS = 20
    CONST STATUS_ROW = 220     ' row 11 (screenpos(0,11))
    CONST PANEL_COL = 11       ' right-hand chrome panel starts at column 11

    CONST COL_TEXT   = CS_WHITE
    CONST COL_HILITE = CS_YELLOW

' Scratch RAM, ours, below fujinet.bas's own SC_* buffers (which stop at
' SC_QUERY $9150 + 48 = $9180) and well below the $9C00 mailbox.
    CONST SC_TABLE      = $9180 ' selected table id, 9 bytes (8 + NUL)
    CONST SC_MYSHIPS    = $9190 ' our 5 placed ship values (cell+100*dir), 1
byte each
    CONST SC_OCCUPY     = $91A0 ' 100 bytes, placement-time occupancy map (0/1)
    CONST SC_PREVFIELD  = $9200 ' 4 x 100 bytes, shadow gamefields for diffing

' ---------------------------------------------------------------------------
' URL literals (ASCII DATA). url_path selects the endpoint:
'   0 tables   1 state   2 ready   3 place   4 attack   5 leave
' ---------------------------------------------------------------------------
lit_n_colon: DATA 78,58
lit_https: DATA 104,116,116,112,115,58,47,47,98,97,116,116,108,101,115,104,105,
112,46,99,97,114,114,45,100,101,115,105,103,110,115,46,99,111,109,47
lit_tables: DATA 116,97,98,108,101,115
lit_qbin: DATA 63,98,105,110,61,49
lit_state: DATA 115,116,97,116,101
lit_ready: DATA 114,101,97,100,121
lit_place: DATA 112,108,97,99,101,47
lit_attack: DATA 97,116,116,97,99,107,47
lit_leave: DATA 108,101,97,118,101
lit_qtable: DATA 63,116,97,98,108,101,61
lit_aplayer: DATA 38,112,108,97,121,101,114,61
lit_abinv2: DATA 38,98,105,110,61,49,38,118,61,50 (2)
lit_comma: DATA 44

    CONST LEN_HTTPS = 36

    ' Lobby appkey: creator 1 / app 1, key = this game's registered slot.
    ' See fujinet-battleship/src/misc.h AK_LOBBY_KEY_SERVER.
    CONST AK_LOBBY_KEY_SERVER = 5

' ---------------------------------------------------------------------------
' Globals
' ---------------------------------------------------------------------------
    DIM gs_i, gs_j, gs_char, #gs_c
    DIM ne_i, ne_j, ne_cur, ne_len
    DIM ne_buf(8)
    DIM ak_try
    DIM tbl_count, tbl_sel
    DIM inp_lock
    DIM want_leave, im_sel
    DIM url_path
    DIM #tmp_addr, #tmp_field

    DIM df_pos, df_len, #df_color, df_i, df_c, df_stop
    DIM #df_src

    ' split_room_url locals (see procedure below)
    DIM sp_i, sp_found, sp_ok, sp_j, sp_k, sp_c, sp_valid, sp_m

    DIM cu_i
    DIM pn_val, pn_h, pn_t, pn_o, pn_started

    DIM cur_status, cur_pc, cur_pstatus, prev_status, ships_drawn, has_action
    DIM atk_pos, has_targeted
    DIM prev_seen_pc, prev_result_status, prev_result_pos, rb_lastattack

    DIM pc_x, pc_y, pc_dir, pc_size, pc_i, pc_ok, pc_moved
    DIM pc_prevx, pc_prevy, pc_prevdir, pc_cx, pc_cy, pc_mx, pc_my
    DIM ship_idx, ph_shipnum

    DIM rb_i, rb_j, rb_c, rb_state, rb_pstatus, rb_ch
    DIM #rb_field
    DIM rb_ship, rb_sx, rb_sy, rb_sdir, rb_ssize
    DIM ds_bq, ds_base, ds_color

    DIM tg_x, tg_y, tg_newx, tg_newy, tg_moved, tg_timeleft, tg_framecount
    DIM tc_q, tc_state, tc_color, tc_field

' ---------------------------------------------------------------------------
' draw_field: render df_len ASCII bytes from #df_src onto the screen at
```

```
' BACKTAB offset df_pos, in color #df_color. MODE 0 gives full GROM (cards
' 0-255), so unlike 5cardstud's MODE 1 client, lowercase is available --
' server prompts/names arrive lowercased and are shown as-is. Stops at a
' NUL and pads the rest of the field with spaces; clamps anything outside
' the printable range to a space rather than drawing garbage.
' ------------------------------------------------------------------------
draw_field: PROCEDURE
    df_stop = 0
    FOR df_i = 0 TO df_len - 1
        df_c = PEEK(#df_src + df_i) AND 255
        IF df_c = 0 THEN df_stop = 1
        IF df_stop THEN df_c = 32
        IF df_c < 32 OR df_c > 127 THEN df_c = 32
        #BACKTAB(df_pos + df_i) = (df_c - 32) * 8 + #df_color
    NEXT df_i
END

' ------------------------------------------------------------------------
' fn_putnum: append the decimal (no leading zeros) representation of pn_val
' (0-999) into FN_TX at the current #fn_txlen. IntyBASIC has no built-in
' itoa; placement values run 0-199 and attack positions 0-99, so three
' digits is generous headroom.
' ------------------------------------------------------------------------
fn_putnum: PROCEDURE
    pn_h = pn_val / 100
    pn_t = (pn_val / 10) % 10
    pn_o = pn_val % 10
    pn_started = 0
    IF pn_h > 0 THEN
        POKE (FN_TX + #fn_txlen), pn_h + 48 : #fn_txlen = #fn_txlen + 1
        pn_started = 1
    END IF
    IF pn_t > 0 OR pn_started THEN
        POKE (FN_TX + #fn_txlen), pn_t + 48 : #fn_txlen = #fn_txlen + 1
    END IF
    POKE (FN_TX + #fn_txlen), pn_o + 48 : #fn_txlen = #fn_txlen + 1
END

' ------------------------------------------------------------------------
' compose_url: build "N:<endpoint><path...><suffix>" directly into FN_TX.
' For url_path=3 (place), SC_MYSHIPS's 5 bytes are comma-joined. For
' url_path=4 (attack), atk_pos is appended. Always requests &bin=1&v=2 --
' v=1 omits the winner's ships and forces activePlayer=-1 at game over,
' both of which render_gameover needs.
' ------------------------------------------------------------------------
compose_url: PROCEDURE
    #fn_txlen = 0
    #fn_src = VARPTR lit_n_colon(0) : fn_len = 2 : GOSUB fn_putstr
    #fn_src = SC_ENDPT : ls_max = 65 : GOSUB fn_strlen : GOSUB fn_putstr

    IF url_path = 0 THEN
        #fn_src = VARPTR lit_tables(0) : fn_len = 6 : GOSUB fn_putstr
        #fn_src = VARPTR lit_qbin(0) : fn_len = 6 : GOSUB fn_putstr
    ELSE
        IF url_path = 1 THEN
            #fn_src = VARPTR lit_state(0) : fn_len = 5 : GOSUB fn_putstr
        ELSEIF url_path = 2 THEN
            #fn_src = VARPTR lit_ready(0) : fn_len = 5 : GOSUB fn_putstr
        ELSEIF url_path = 3 THEN
            #fn_src = VARPTR lit_place(0) : fn_len = 6 : GOSUB fn_putstr
            FOR cu_i = 0 TO 4
                pn_val = PEEK(SC_MYSHIPS + cu_i) AND 255
                GOSUB fn_putnum
                IF cu_i < 4 THEN
                    #fn_src = VARPTR lit_comma(0) : fn_len = 1 : GOSUB fn_putstr
                END IF
            NEXT cu_i
        ELSEIF url_path = 4 THEN
            #fn_src = VARPTR lit_attack(0) : fn_len = 7 : GOSUB fn_putstr
            pn_val = atk_pos
            GOSUB fn_putnum
        ELSE
            #fn_src = VARPTR lit_leave(0) : fn_len = 5 : GOSUB fn_putstr
        END IF

        #fn_src = VARPTR lit_qtable(0) : fn_len = 7 : GOSUB fn_putstr
        #fn_src = SC_TABLE : ls_max = 9 : GOSUB fn_strlen : GOSUB fn_putstr
        #fn_src = VARPTR lit_aplayer(0) : fn_len = 8 : GOSUB fn_putstr
        #fn_src = SC_NAME : ls_max = 9 : GOSUB fn_strlen : GOSUB fn_putstr
        #fn_src = VARPTR lit_abinv2(0) : fn_len = 10 : GOSUB fn_putstr
    END IF
END

' ------------------------------------------------------------------------
' split_room_url: given a lobby-supplied "https://host/?table=xyz" value
' already sitting in SC_ENDPT with its length in fn_len (as left by
' appkey_read), split it into the bare endpoint (SC_ENDPT, truncated at
' the '?') and the table id (SC_TABLE). Mirrors the C clients'
' welcomeActionVerifyServerDetails() '?' scan. Leaves SC_TABLE empty (a
' leading NUL) if there's no '?', the query isn't "table=...", or the id
' contains anything other than A-Z/a-z/0-9 -- it goes straight into a
' rebuilt query string unescaped, same reasoning as the username check
' above.
' ------------------------------------------------------------------------
split_room_url: PROCEDURE
    POKE SC_TABLE, 0

    sp_found = 255 ' sentinel: not found
    sp_i = 0
    WHILE sp_i < fn_len
        IF (PEEK(SC_ENDPT + sp_i) AND 255) = 63 THEN ' '?'
            sp_found = sp_i
            EXIT WHILE
        END IF
        sp_i = sp_i + 1
    WEND
```

```
    IF sp_found = 255 THEN RETURN

    ' Truncate the endpoint at the '?' regardless of what follows.
    POKE (SC_ENDPT + sp_found), 0

    ' Require "table=" (6 bytes) right after the '?'.
    sp_ok = 0
    IF sp_found + 7 <= fn_len THEN
        sp_ok = 1
        IF (PEEK(SC_ENDPT + sp_found + 1) AND 255) <> 116 THEN sp_ok = 0 ' t
        IF (PEEK(SC_ENDPT + sp_found + 2) AND 255) <> 97  THEN sp_ok = 0 ' a
        IF (PEEK(SC_ENDPT + sp_found + 3) AND 255) <> 98  THEN sp_ok = 0 ' b
        IF (PEEK(SC_ENDPT + sp_found + 4) AND 255) <> 108 THEN sp_ok = 0 ' l
        IF (PEEK(SC_ENDPT + sp_found + 5) AND 255) <> 101 THEN sp_ok = 0 ' e
        IF (PEEK(SC_ENDPT + sp_found + 6) AND 255) <> 61  THEN sp_ok = 0 ' =
    END IF
    IF sp_ok = 0 THEN RETURN

    ' Copy the id: up to 8 bytes, stopping at NUL or '&'.
    sp_j = sp_found + 7
    sp_k = 0
    WHILE sp_k < 8 AND sp_j < fn_len
        sp_c = PEEK(SC_ENDPT + sp_j) AND 255
        IF sp_c = 0 OR sp_c = 38 THEN EXIT WHILE ' NUL or '&'
        POKE (SC_TABLE + sp_k), sp_c
        sp_k = sp_k + 1
        sp_j = sp_j + 1
    WEND
    POKE (SC_TABLE + sp_k), 0

    ' Validate: A-Z/a-z/0-9 only.
    sp_valid = 1
    FOR sp_m = 0 TO sp_k - 1
        sp_c = PEEK(SC_TABLE + sp_m) AND 255
        IF (sp_c < 65 OR sp_c > 90) AND (sp_c < 97 OR sp_c > 122) AND (sp_c < 48
OR sp_c > 57) THEN sp_valid = 0
    NEXT sp_m
    IF sp_valid = 0 THEN POKE SC_TABLE, 0
END

' ------------------------------------------------------------------------
' write_room_appkey: persist SC_ENDPT + "?table=" + SC_TABLE back to the
' lobby server appkey slot, so a reboot without going back through the
' lobby rejoins the same room (mirrors the C clients' "Update server app
' key in case of reboot"). Builds the payload directly in FN_TX and writes
' it from there -- appkey_write's byte-by-byte copy from #fn_src into
' FN_TX is a safe no-op when #fn_src is FN_TX itself.
' ------------------------------------------------------------------------
write_room_appkey: PROCEDURE
    #fn_txlen = 0
    #fn_src = SC_ENDPT : ls_max = 65 : GOSUB fn_strlen : GOSUB fn_putstr
    #fn_src = VARPTR lit_qtable(0) : fn_len = 7 : GOSUB fn_putstr
    #fn_src = SC_TABLE : ls_max = 9 : GOSUB fn_strlen : GOSUB fn_putstr

    ak_creator_lo = 1 : ak_creator_hi = 0 : ak_app = 1
    ak_key = AK_LOBBY_KEY_SERVER : ak_mode = 1
    GOSUB appkey_open
    IF fn_ok THEN
        fn_len = #fn_txlen : #fn_src = FN_TX
        GOSUB appkey_write
        GOSUB appkey_close
    END IF
END

' ------------------------------------------------------------------------
' clear_room_appkey: blank the lobby server appkey slot on a deliberate
' leave, so a later reboot lands on table select instead of silently
' rejoining a table the player just quit.
' ------------------------------------------------------------------------
clear_room_appkey: PROCEDURE
    ak_creator_lo = 1 : ak_creator_hi = 0 : ak_app = 1
    ak_key = AK_LOBBY_KEY_SERVER : ak_mode = 1
    GOSUB appkey_open
    IF fn_ok THEN
        fn_len = 0 : #fn_src = FN_TX
        GOSUB appkey_write
        GOSUB appkey_close
    END IF
    POKE SC_TABLE, 0
END

' ========================================================================
' Boot
' ========================================================================
boot_start:
    MODE 0, CS_BLACK, CS_BLACK, CS_BLACK, CS_BLACK : WAIT
    CLS
    PRINT AT 0 COLOR COL_TEXT, "BATTLESHIP"
    PRINT AT 40, "CONNECTING TO FUJINET"
    GOSUB fn_wait_mailbox
    IF fn_ok = 0 THEN
        PRINT AT 40, "NO CARTRIDGE MAILBOX"
        GOTO halt
    END IF

' ------------------------------------------------------------------------
' Name: try the shared lobby appkey (creator 1 / app 1 / key 0) first, with
' one retry (a cold RP2040/ESP32 link right after the mailbox comes up can
' time out the very first transaction); fall back to the disc letter picker
' if the read fails, comes back empty, or holds anything outside A-Z0-9 --
' this slot is shared by every FujiNet client, and a stray character here
' goes straight into the query string unescaped.
' ------------------------------------------------------------------------
    gs_i = 0
    FOR ak_try = 0 TO 1
        ak_creator_lo = 1 : ak_creator_hi = 0 : ak_app = 1 : ak_key = 0 :
```

```
ak_mode = 0
        GOSUB appkey_open
        IF fn_ok THEN EXIT FOR
    NEXT ak_try
    IF fn_ok THEN
        #fn_src = SC_NAME : ls_max = 9 : GOSUB appkey_read
        GOSUB appkey_close
        IF fn_len > 0 THEN
            gs_i = 1
            FOR gs_j = 0 TO fn_len - 1
                gs_char = PEEK(SC_NAME + gs_j) AND 255
                IF gs_char >= 97 AND gs_char <= 122 THEN gs_char = gs_char - 32
                IF (gs_char < 65 OR gs_char > 90) AND (gs_char < 48 OR gs_char >
57) THEN gs_i = 0
                NEXT gs_j
            END IF
    END IF
    IF gs_i = 0 THEN GOSUB name_entry_screen

' ------------------------------------------------------------------------
' Room: check the lobby server appkey (creator 1 / app 1 / key
' AK_LOBBY_KEY_SERVER). If the lobby wrote a room there, split it into
' SC_ENDPT/SC_TABLE and skip straight past table select. Seed the
' compiled-in default endpoint first so SC_ENDPT is always valid even if
' the appkey is empty or the read fails -- compose_url relies on it from
' here on for every request, not just /tables.
' ------------------------------------------------------------------------
    FOR gs_i = 0 TO LEN_HTTPS - 1
        POKE (SC_ENDPT + gs_i), PEEK(VARPTR lit_https(0) + gs_i) AND 255
    NEXT gs_i
    POKE (SC_ENDPT + LEN_HTTPS), 0
    POKE SC_TABLE, 0

    FOR ak_try = 0 TO 1
        ak_creator_lo = 1 : ak_creator_hi = 0 : ak_app = 1
        ak_key = AK_LOBBY_KEY_SERVER : ak_mode = 0
        GOSUB appkey_open
        IF fn_ok THEN EXIT FOR
    NEXT ak_try
    IF fn_ok THEN
        #fn_src = SC_ENDPT : ls_max = 65 : GOSUB appkey_read
        GOSUB appkey_close
        IF fn_len > 0 THEN
            GOSUB split_room_url
        ELSE
            ' appkey_read always NUL-terminates at fn_len, even when
            ' empty, which would otherwise wipe out the default just
            ' seeded above -- restore it.
            FOR gs_i = 0 TO LEN_HTTPS - 1
                POKE (SC_ENDPT + gs_i), PEEK(VARPTR lit_https(0) + gs_i) AND 255
            NEXT gs_i
            POKE (SC_ENDPT + LEN_HTTPS), 0
        END IF
    END IF

    IF (PEEK(SC_TABLE) AND 255) <> 0 THEN GOTO table_joined

' ========================================================================
' Table select (re-entered after leaving a table)
' ========================================================================
table_select:
    CLS
    PRINT AT 0 COLOR COL_TEXT, "CHOOSE A TABLE"
    PRINT AT 40, "REFRESHING..."

    url_path = 0 : GOSUB compose_url
    #net_readlen = TABLES_MAXLEN
    GOSUB api_call

    IF fn_ok = 1 THEN
        #tmp_field = 1 + (PEEK(FN_RX) AND 255) * TABLE_STRIDE
        IF #net_gotlen < #tmp_field THEN fn_ok = 0
    END IF

    IF fn_ok = 0 THEN
        PRINT AT 40, "SERVER UNREACHABLE  "
        inp_lock = 90
        WHILE inp_lock > 0
            inp_lock = inp_lock - 1
            WAIT
        WEND
        GOTO table_select
    END IF

    tbl_count = PEEK(FN_RX) AND 255
    IF tbl_count > 8 THEN tbl_count = 8
    IF tbl_count = 0 THEN
        PRINT AT 40, "NO TABLES AVAILABLE "
        inp_lock = 90
        WHILE inp_lock > 0
            inp_lock = inp_lock - 1
            WAIT
        WEND
        GOTO table_select
    END IF

    CLS
    PRINT AT 0 COLOR COL_TEXT, "CHOOSE A TABLE"
    FOR gs_i = 0 TO tbl_count - 1
        #tmp_addr = table_addr(gs_i)
        #df_src = #tmp_addr + TBL_NAME : df_pos = 20 + gs_i * 20 + 1 : df_len =
14 : #df_color = COL_TEXT
        GOSUB draw_field
        #df_src = #tmp_addr + TBL_PLAYERS : df_pos = 20 + gs_i * 20 + 15 :
df_len = 5 : #df_color = COL_TEXT
        GOSUB draw_field
```

```
    NEXT gs_i

    tbl_sel = 0
    inp_lock = 0
ts_input:
    WAIT
    FOR gs_i = 0 TO tbl_count - 1
        #gs_c = COL_TEXT
        IF gs_i = tbl_sel THEN #gs_c = COL_HILITE
        #BACKTAB(20 + gs_i * 20) = (62 - 32) * 8 + #gs_c ' '>' cursor glyph
    NEXT gs_i

    IF inp_lock > 0 THEN inp_lock = inp_lock - 1 : GOTO ts_input

    IF CONT1.DOWN THEN
        tbl_sel = tbl_sel + 1
        IF tbl_sel >= tbl_count THEN tbl_sel = 0
        inp_lock = 8
        GOSUB sound_cursor
        GOTO ts_input
    END IF
    IF CONT1.UP THEN
        IF tbl_sel = 0 THEN tbl_sel = tbl_count
        tbl_sel = tbl_sel - 1
        inp_lock = 8
        GOSUB sound_cursor
        GOTO ts_input
    END IF
    IF CONT1.BUTTON = 0 THEN GOTO ts_input
    GOSUB sound_select

    #tmp_addr = table_addr(tbl_sel)
    FOR gs_i = 0 TO 8
        POKE (SC_TABLE + gs_i), PEEK(#tmp_addr + TBL_ID + gs_i) AND 255
    NEXT gs_i

    ' Update the lobby server appkey so a reboot without going back
    ' through the lobby rejoins this table.
    GOSUB write_room_appkey

table_joined:
' ============================================================================
' In a table: poll /state (or submit a pending ready/place/attack action in
' its place) and dispatch on status. A full CLS + board_init happens only
' on the transition out of the lobby (or on first entry) -- every other
' poll updates in place, matching the differential-render approach 5 Card
' Stud settled on for the same reason (IntyBASIC/the STIC have no true
' page-flip; a per-poll CLS would visibly blank the screen every ~1s).
' ============================================================================
    prev_status = 255 ' sentinel: forces the initial CLS/board_init
    prev_seen_pc = 255 ' sentinel: skip the join/leave cue on the first poll
    prev_result_status = 255
    prev_result_pos = 255
    poll_wait = 0
    want_leave = 0
    has_action = 0
    GOSUB sound_join

tl_loop:
    WAIT

    IF poll_wait > 0 THEN
        poll_wait = poll_wait - 1
        GOTO tl_input
    END IF

    IF has_action = 1 THEN
        url_path = 2
    ELSEIF has_action = 2 THEN
        url_path = 3
    ELSEIF has_action = 3 THEN
        url_path = 4
    ELSE
        url_path = 1
    END IF
    has_action = 0
    GOSUB compose_url
    #net_readlen = GAME_MAXLEN
    GOSUB api_call
    GOSUB validate_state

    IF fn_ok = 0 THEN
        poll_wait = 30
        GOTO tl_input
    END IF

    cur_status = state_status
    cur_pc = state_playercount
    cur_pstatus = state_playerstatus

    IF prev_seen_pc <> 255 THEN
        IF cur_pc > prev_seen_pc THEN GOSUB sound_player_join
        IF cur_pc < prev_seen_pc THEN GOSUB sound_player_left
    END IF
    prev_seen_pc = cur_pc

    IF cur_status = STATUS_LOBBY THEN
        IF prev_status <> STATUS_LOBBY THEN CLS
        GOSUB render_lobby
        poll_wait = 45
    ELSEIF cur_status = STATUS_GAMEOVER THEN
        IF prev_status = STATUS_LOBBY OR prev_status = 255 THEN
            CLS
            GOSUB board_init
        END IF
        IF prev_status <> STATUS_GAMEOVER THEN GOSUB render_gameover
```

```
        poll_wait = 60
    ELSE
        ' STATUS_PLACE_SHIPS, GAMESTART, MISS, HIT, SUNK.
        IF prev_status = STATUS_LOBBY OR prev_status = 255 THEN
            CLS
            GOSUB board_init
            ships_drawn = 0
        END IF

        IF cur_status = STATUS_PLACE_SHIPS THEN
            IF cur_pstatus = PSTATUS_PLACE_SHIPS THEN
                GOSUB handle_placement
                has_action = 2
                poll_wait = 0
            ELSE
                #df_src = FN_RX + GAME_PROMPT : df_pos = STATUS_ROW : df_len =
ROWCELLS : #df_color = COL_TEXT
                GOSUB draw_field
                poll_wait = 30
            END IF
        ELSE
            GOSUB render_game_board
            GOSUB render_panel
            #df_src = FN_RX + GAME_PROMPT : df_pos = STATUS_ROW : df_len =
ROWCELLS : #df_color = COL_TEXT
            GOSUB draw_field

            ' status (MISS/HIT/SUNK) reflects the *last* attack's result and
            ' can legitimately repeat across several polls if no one has
            ' fired since -- pair it with lastAttackPos so the sound plays
            ' exactly once per real event, not once per poll it's visible.
            IF cur_status = STATUS_MISS OR cur_status = STATUS_HIT OR cur_status
= STATUS_SUNK THEN 6
                rb_lastattack = PEEK(FN_RX + GAME_LASTATTACK) AND 255
                IF cur_status <> prev_result_status OR rb_lastattack <>
prev_result_pos THEN
                    IF cur_status = STATUS_MISS THEN
                        GOSUB sound_miss
                    ELSEIF cur_status = STATUS_HIT THEN
                        GOSUB sound_hit
                    ELSE
                        GOSUB sound_sink
                    END IF
                    prev_result_status = cur_status
                    prev_result_pos = rb_lastattack
                END IF
            END IF

            IF active_player = 0 AND cur_pstatus = PSTATUS_PLAYING THEN
                GOSUB targeting
                IF has_targeted THEN
                    GOSUB sound_myturn
                    has_action = 3
                    poll_wait = 0
                ELSE
                    poll_wait = 20
                END IF
            ELSE
                poll_wait = 20
            END IF
        END IF
    END IF

    prev_status = cur_status

tl_input:
    IF CONT1.KEY = 10 THEN GOSUB ingame_menu
    IF want_leave THEN
        want_leave = 0
        GOTO table_select
    END IF
    GOTO tl_loop

' ============================================================================
' render_lobby: server name, ready list, prompt. Trigger toggles /ready.
' ============================================================================
render_lobby: PROCEDURE
    #df_src = FN_RX + LOBBY_SERVERNAME : df_pos = 20 : df_len = 20 : #df_color =
COL_TEXT
    GOSUB draw_field
    FOR gs_i = 0 TO cur_pc - 1
        #tmp_addr = lobby_player_addr(gs_i)
        #df_src = #tmp_addr + LP_NAME : df_pos = 60 + gs_i * 20 : df_len = 9 :
#df_color = COL_TEXT
        GOSUB draw_field
        #gs_c = 46 ' '.'
        IF (PEEK(#tmp_addr + LP_READY) AND 255) <> 0 THEN #gs_c = 42 ' '*'
        #BACKTAB(60 + gs_i * 20 + 10) = (#gs_c - 32) * 8 + COL_TEXT
    NEXT gs_i
    #df_src = FN_RX + GAME_PROMPT : df_pos = STATUS_ROW : df_len = ROWCELLS :
#df_color = COL_TEXT
    GOSUB draw_field

    IF CONT1.BUTTON THEN
        GOSUB sound_select
        url_path = 2 : GOSUB compose_url
        #net_readlen = GAME_MAXLEN
        GOSUB api_call
        poll_wait = 15
    END IF
END

' ============================================================================
' render_gameover: reveal the winner's ships (myShips[5..9]) on their
' quadrant, show the prompt. Held on screen (poll_wait=60) until the server
```

```
615 ' resets the table back to the lobby.
616 ' ============================================================================
617 render_gameover: PROCEDURE
618     GOSUB sound_gamedone
619     IF active_player >= 0 AND active_player <= 3 THEN
620         ds_bq = active_player : ds_base = GAME_MYSHIPS + 5 : ds_color =
BCOL_SUNKSHIP
621         GOSUB draw_ship_set
622     END IF
623     #df_src = FN_RX + GAME_PROMPT : df_pos = STATUS_ROW : df_len = ROWCELLS :
#df_color = COL_HILITE
624     GOSUB draw_field
625 END
626
627 ' ----------------------------------------------------------------------------
628 ' draw_ship_set: paint the 5 ships encoded at FN_RX+ds_base (each byte
629 ' cell+100*dir, state.bas's myShips layout) onto quadrant ds_bq in colour
630 ' ds_color. Shared by render_game_board (own ships, ds_base=GAME_MYSHIPS)
631 ' and render_gameover (winner's ships, ds_base=GAME_MYSHIPS+5).
632 ' ----------------------------------------------------------------------------
633 draw_ship_set: PROCEDURE
634     FOR rb_i = 0 TO 4
635         rb_ship = PEEK(FN_RX + ds_base + rb_i) AND 255
636         IF rb_ship > 0 THEN
637             rb_sx = (rb_ship % 100) % 10
638             rb_sy = (rb_ship % 100) / 10
639             rb_sdir = rb_ship / 100
640             rb_ssize = ship_size(rb_i)
641             FOR rb_j = 0 TO rb_ssize - 1
642                 IF rb_sdir = 0 THEN
643                     bc_x = rb_sx + rb_j : bc_y = rb_sy
644                 ELSE
645                     bc_x = rb_sx : bc_y = rb_sy + rb_j
646                 END IF
647                 bc_bq = ds_bq : bc_color = ds_color
648                 GOSUB board_cell
649             NEXT rb_j
650         END IF
651     NEXT rb_i
652 END
653
654 ' ============================================================================
655 ' render_panel: right-hand chrome (columns 11-19). Three rows per player
656 ' (name, ships-left, blank separator) x 4 players fits rows 0-10 exactly.
657 ' Redrawn in full every poll -- cheap (36 characters max) and simpler than
658 ' diffing four short text fields.
659 ' ============================================================================
660 render_panel: PROCEDURE
661     FOR rb_i = 0 TO 3
662         #gs_c = COL_TEXT
663         IF rb_i < cur_pc THEN
664             IF rb_i = active_player THEN #gs_c = COL_HILITE
665             #df_src = player_addr(rb_i) + PL_NAME : df_pos =
screenpos(PANEL_COL, rb_i * 3) : df_len = 9 : #df_color = #gs_c
666             GOSUB draw_field
667             FOR rb_j = 0 TO 4
668                 rb_state = PEEK(player_addr(rb_i) + PL_SHIPSLEFT + rb_j) AND 255
669                 rb_ch = 46 ' '.'  -- sunk
670                 IF rb_state <> 0 THEN rb_ch = 35 ' '#' -- afloat
671                 #BACKTAB(screenpos(PANEL_COL + rb_j, rb_i * 3 + 1)) = (rb_ch -
32) * 8 + #gs_c
672             NEXT rb_j
673         ELSE
674             FOR rb_j = 0 TO 8
675                 #BACKTAB(screenpos(PANEL_COL + rb_j, rb_i * 3)) = COL_TEXT
676                 #BACKTAB(screenpos(PANEL_COL + rb_j, rb_i * 3 + 1)) = COL_TEXT
677             NEXT rb_j
678         END IF
679     NEXT rb_i
680 END
681
682 ' ============================================================================
683 ' render_game_board: diff each seated player's 100-byte gamefield against
684 ' its SC_PREVFIELD shadow and repaint only the cells that changed (board
685 ' cells are always painted water by board_init, and SC_PREVFIELD is seeded
686 ' to a sentinel below whenever a fresh board_init just ran, so the first
687 ' call after entering GAMESTART paints every real cell exactly once). Own
688 ' ship overlay is drawn once from the wire's myShips bytes, authoritative
689 ' over whatever handle_placement guessed client-side (covers reconnects).
690 ' ============================================================================
691 render_game_board: PROCEDURE
692     IF ships_drawn = 0 THEN
693         FOR rb_i = 0 TO 99
694             POKE (SC_PREVFIELD + 0 * 100 + rb_i), 255
695             POKE (SC_PREVFIELD + 1 * 100 + rb_i), 255
696             POKE (SC_PREVFIELD + 2 * 100 + rb_i), 255
697             POKE (SC_PREVFIELD + 3 * 100 + rb_i), 255
698         NEXT rb_i
699     END IF
700
701     FOR rb_i = 0 TO cur_pc - 1
702         rb_pstatus = PEEK(player_addr(rb_i) + PL_STATUS) AND 255
703         IF rb_pstatus <> PSTATUS_PLACE_SHIPS THEN
704             #rb_field = player_addr(rb_i) + PL_GAMEFIELD
705             FOR rb_c = 0 TO 99
706                 rb_state = PEEK(#rb_field + rb_c) AND 255
707                 IF rb_state <> (PEEK(SC_PREVFIELD + rb_i * 100 + rb_c) AND 255)
THEN
708                     bc_bq = rb_i
709                     bc_x = rb_c % 10
710                     bc_y = rb_c / 10
711                     IF rb_state = 1 THEN
712                         bc_color = BCOL_HIT
713                     ELSEIF rb_state = 2 THEN
714                         bc_color = BCOL_MISS
```

```
                    ELSE
                        bc_color = BCOL_WATER
                    END IF
                    GOSUB board_cell
                    POKE (SC_PREVFIELD + rb_i * 100 + rb_c), rb_state
                END IF
            NEXT rb_c
        END IF
    NEXT rb_i

    IF ships_drawn = 0 THEN
        FOR rb_i = 0 TO 4
            rb_ship = PEEK(FN_RX + GAME_MYSHIPS + rb_i) AND 255
            rb_sx = (rb_ship % 100) % 10
            rb_sy = (rb_ship % 100) / 10
            rb_sdir = rb_ship / 100
            rb_ssize = ship_size(rb_i)
            FOR rb_j = 0 TO rb_ssize - 1
                IF rb_sdir = 0 THEN
                    bc_x = rb_sx + rb_j : bc_y = rb_sy
                ELSE
                    bc_x = rb_sx : bc_y = rb_sy + rb_j
                END IF
                bc_bq = 0 : bc_color = BCOL_SHIP
                GOSUB board_cell
            NEXT rb_j
        NEXT rb_i
        ships_drawn = 1
    END IF
END

' ============================================================================
' targeting: your turn. Cursor moves in lockstep over every live enemy
' quadrant at once -- the server's core multiplayer rule is that a single
' shot fires at that coordinate on every other playing board simultaneously,
' so there's only ever one cursor position to choose, not one per opponent.
' Sets has_targeted + atk_pos on a confirmed attack.
' ============================================================================
targeting: PROCEDURE
    tg_x = 0 : tg_y = 0
    has_targeted = 0
    inp_lock = 0
    GOSUB targeting_draw_cursor

    ' moveTime (server-computed, already net of a network round-trip grace
    ' period -- see gameLogic.go) is the countdown's starting point; count
    ' it down locally frame-by-frame rather than re-polling every second.
    ' Shown in the prompt row's last 2 columns, same corner 5cardstud uses
    ' for its stopwatch.
    tg_timeleft = PEEK(FN_RX + GAME_MOVETIME) AND 255
    tg_framecount = 0
    ' moveTime can run up to 250 (state.bas), so the field needs 3 digits --
    ' a 2-digit field silently produces garbage glyphs rather than truncating
    ' (PRNUM16.z's digit loop assumes higher place-values were already
    ' stripped by wider field positions it never visits). Space-padded
    ' (<.3>, PRNUM16.b) rather than zero-padded (<3>, PRNUM16.z): .z's entry
    ' point never initializes the register it uses to pick blank-vs-zero
    ' padding, so on a value needing fewer than 3 digits it can print
    ' whatever garbage that register held as the leading "digit" instead of
    ' an actual 0. .b's entry point explicitly clears it first.
    PRINT AT STATUS_ROW + 17 COLOR COL_TEXT, <.3>tg_timeleft

tg_loop:
    WAIT

    tg_framecount = tg_framecount + 1
    IF tg_framecount >= 60 THEN
        tg_framecount = 0
        IF tg_timeleft > 0 THEN tg_timeleft = tg_timeleft - 1
        PRINT AT STATUS_ROW + 17 COLOR COL_TEXT, <.3>tg_timeleft
        GOSUB sound_tick
        ' The server enforces its own 45s move limit and skips a turn that
        ' runs out (gameLogic.go's PLAYER_TIME_LIMIT) -- without this bail,
        ' a player who never acts leaves this loop blocked on WAIT forever,
        ' so the client would never poll again to discover its turn had
        ' already moved on.
        IF tg_timeleft = 0 THEN
            GOSUB targeting_erase_cursor
            RETURN
        END IF
    END IF

    IF inp_lock > 0 THEN inp_lock = inp_lock - 1 : GOTO tg_loop

    ' Compute the candidate new position into tg_newx/tg_newy without
    ' touching tg_x/tg_y yet -- targeting_erase_cursor reads tg_x/tg_y to
    ' know which cell to restore, so it must still see the *old* position
    ' when it runs, not the one we're about to move to.
    tg_moved = 0
    tg_newx = tg_x : tg_newy = tg_y
    IF CONT1.RIGHT AND tg_newx < 9 THEN tg_newx = tg_newx + 1 : tg_moved = 1
    IF CONT1.LEFT AND tg_newx > 0 THEN tg_newx = tg_newx - 1 : tg_moved = 1
    IF CONT1.DOWN AND tg_newy < 9 THEN tg_newy = tg_newy + 1 : tg_moved = 1
    IF CONT1.UP AND tg_newy > 0 THEN tg_newy = tg_newy - 1 : tg_moved = 1

    IF tg_moved THEN
        GOSUB targeting_erase_cursor
        tg_x = tg_newx : tg_y = tg_newy
        GOSUB targeting_draw_cursor
        inp_lock = 6
        GOSUB sound_cursor
        GOTO tg_loop
    END IF

    IF CONT1.KEY = 10 THEN
```

```
        GOSUB targeting_erase_cursor
        RETURN
    END IF
    IF CONT1.BUTTON = 0 THEN GOTO tg_loop

    GOSUB sound_attack
    GOSUB targeting_erase_cursor
    atk_pos = tg_y * 10 + tg_x
    has_targeted = 1
END

targeting_draw_cursor: PROCEDURE
    FOR tc_q = 1 TO 3
        IF tc_q < cur_pc THEN
            tc_state = PEEK(player_addr(tc_q) + PL_STATUS) AND 255
            IF tc_state = PSTATUS_PLAYING THEN
                bc_bq = tc_q : bc_x = tg_x : bc_y = tg_y : bc_color =
BCOL_CURSOR
                GOSUB board_cell
            END IF
        END IF
    NEXT tc_q
END

targeting_erase_cursor: PROCEDURE
    FOR tc_q = 1 TO 3
        IF tc_q < cur_pc THEN
            tc_state = PEEK(player_addr(tc_q) + PL_STATUS) AND 255
            IF tc_state = PSTATUS_PLAYING THEN
                tc_field = PEEK(SC_PREVFIELD + tc_q * 100 + tg_y * 10 + tg_x)
AND 255
                tc_color = BCOL_WATER
                IF tc_field = 1 THEN tc_color = BCOL_HIT
                IF tc_field = 2 THEN tc_color = BCOL_MISS
                bc_bq = tc_q : bc_x = tg_x : bc_y = tg_y : bc_color = tc_color
                GOSUB board_cell
            END IF
        END IF
    NEXT tc_q
END

' The compiled program overflows IntyBASIC's default $5000-$6FFF (8192-word)
' window -- the compiler happily keeps allocating past $6FFF, but $7000 is
' not in the manual's list of ranges "usable without additional programming"
' on modern flash cart/homebrew PCBs (that list jumps from $6FFF to $A000).
' Relocating everything from here to halt: into $C100-$FFFF (also on that
' list) via an explicit ASM ORG keeps the whole ROM inside documented-safe
' territory. This is a code segment, not just data (contrary to the
' manual's "easier... if you put only data in these areas" suggestion),
' but cross-segment GOSUB/GOTO compiles to ordinary absolute CP-1610
' jumps/calls, so that's only a bookkeeping caveat, not a correctness one.
    ASM ORG $D000

' ============================================================================
' handle_placement: place all 5 ships (sizes 5,4,3,3,2, fixed order). Each
' ship starts at a random legal position, then the disc moves it, B1
' rotates it, and the action button confirms (rejected locally if it
' overlaps an already-confirmed ship -- no need to round-trip to the server
' to find that out). Fills SC_MYSHIPS for compose_url's url_path=3.
' ============================================================================
handle_placement: PROCEDURE 4

    ' q0 is already water -- board_init just painted all four boards before
    ' this was reached (see tl_loop's STATUS_LOBBY-or-255 transition). Only
    ' the occupancy map needs clearing here.
    FOR rb_i = 0 TO 99
        POKE (SC_OCCUPY + rb_i), 0
    NEXT rb_i

    FOR ship_idx = 0 TO 4
        pc_size = ship_size(ship_idx)

        DO
            pc_dir = RANDOM(2)
            IF pc_dir = 0 THEN
                pc_x = RANDOM(11 - pc_size)
                pc_y = RANDOM(10)
            ELSE
                pc_x = RANDOM(10)
                pc_y = RANDOM(11 - pc_size)
            END IF
            GOSUB place_fits
        LOOP WHILE pc_ok = 0

        pc_prevx = pc_x : pc_prevy = pc_y : pc_prevdir = pc_dir
        GOSUB place_draw

        ph_shipnum = ship_idx + 1
        ' "/5 MOVE B1=ROT" is exactly 14 chars, filling columns 6-19 of the
        ' 20-column STATUS_ROW with no overflow -- the original "/5  DISC=
        ' MOVE B1=ROT" (20 chars) ran 6 columns past the end of the row and
        ' off the end of #BACKTAB, corrupting the rest of the screen.
        PRINT AT STATUS_ROW COLOR COL_TEXT, "SHIP "
        PRINT AT STATUS_ROW + 5, <>ph_shipnum
        PRINT AT STATUS_ROW + 6, "/5 MOVE B1=ROT"

        inp_lock = 0
ph_loop:
        WAIT
        IF inp_lock > 0 THEN inp_lock = inp_lock - 1 : GOTO ph_loop

        pc_mx = 9 : pc_my = 9
        IF pc_dir = 0 THEN pc_mx = 10 - pc_size ELSE pc_my = 10 - pc_size

        pc_moved = 0
        IF CONT1.RIGHT AND pc_x < pc_mx THEN pc_x = pc_x + 1 : pc_moved = 1
```

```
        IF CONT1.LEFT AND pc_x > 0 THEN pc_x = pc_x - 1 : pc_moved = 1
        IF CONT1.DOWN AND pc_y < pc_my THEN pc_y = pc_y + 1 : pc_moved = 1
        IF CONT1.UP AND pc_y > 0 THEN pc_y = pc_y - 1 : pc_moved = 1

        IF CONT1.B1 THEN
            pc_dir = 1 - pc_dir
            IF pc_dir = 0 AND pc_x > 10 - pc_size THEN pc_x = 10 - pc_size
            IF pc_dir = 1 AND pc_y > 10 - pc_size THEN pc_y = 10 - pc_size
            pc_moved = 1
        END IF

        IF pc_moved THEN
            GOSUB place_erase
            pc_prevx = pc_x : pc_prevy = pc_y : pc_prevdir = pc_dir
            GOSUB place_draw
            inp_lock = 6
            GOSUB sound_cursor
            GOTO ph_loop
        END IF

        IF CONT1.BUTTON = 0 THEN GOTO ph_loop

        GOSUB place_fits
        IF pc_ok = 0 THEN
            GOSUB sound_invalid
            GOTO ph_loop
        END IF

        GOSUB place_mark
        GOSUB sound_place_ship
        FOR pc_i = 0 TO pc_size - 1
            IF pc_dir = 0 THEN
                bc_x = pc_x + pc_i : bc_y = pc_y
            ELSE
                bc_x = pc_x : bc_y = pc_y + pc_i
            END IF
            bc_bq = 0 : bc_color = BCOL_SHIP
            GOSUB board_cell
        NEXT pc_i

        POKE (SC_MYSHIPS + ship_idx), pc_x + pc_y * 10 + 100 * pc_dir
    NEXT ship_idx
END

place_fits: PROCEDURE
    pc_ok = 1
    IF pc_dir = 0 THEN
        IF pc_x + pc_size > 10 THEN pc_ok = 0
    ELSE
        IF pc_y + pc_size > 10 THEN pc_ok = 0
    END IF
    IF pc_ok THEN
        FOR pc_i = 0 TO pc_size - 1
            IF pc_dir = 0 THEN
                pc_cx = pc_x + pc_i : pc_cy = pc_y
            ELSE
                pc_cx = pc_x : pc_cy = pc_y + pc_i
            END IF
            IF (PEEK(SC_OCCUPY + pc_cy * 10 + pc_cx) AND 255) <> 0 THEN pc_ok =
0
        NEXT pc_i
    END IF
END

place_mark: PROCEDURE
    FOR pc_i = 0 TO pc_size - 1
        IF pc_dir = 0 THEN
            pc_cx = pc_x + pc_i : pc_cy = pc_y
        ELSE
            pc_cx = pc_x : pc_cy = pc_y + pc_i
        END IF
        POKE (SC_OCCUPY + pc_cy * 10 + pc_cx), 1
    NEXT pc_i
END

place_erase: PROCEDURE
    FOR pc_i = 0 TO pc_size - 1
        IF pc_prevdir = 0 THEN
            pc_cx = pc_prevx + pc_i : pc_cy = pc_prevy
        ELSE
            pc_cx = pc_prevx : pc_cy = pc_prevy + pc_i
        END IF
        bc_bq = 0 : bc_x = pc_cx : bc_y = pc_cy : bc_color = BCOL_WATER
        GOSUB board_cell
    NEXT pc_i
END

place_draw: PROCEDURE
    FOR pc_i = 0 TO pc_size - 1
        IF pc_dir = 0 THEN
            pc_cx = pc_x + pc_i : pc_cy = pc_y
        ELSE
            pc_cx = pc_x : pc_cy = pc_y + pc_i
        END IF
        bc_bq = 0 : bc_x = pc_cx : bc_y = pc_cy : bc_color = BCOL_CURSOR
        GOSUB board_cell
    NEXT pc_i
END

' ============================================================================
' ingame_menu: keypad CLEAR overlay, drawn over the prompt row and the two
' rows above it. Disc up/down to choose, action button to confirm, Clear
' again cancels straight back to RESUME.
' ============================================================================
ingame_menu: PROCEDURE
    im_sel = 0
```

```
    inp_lock = 0
im_loop:
    PRINT AT STATUS_ROW - 40 COLOR COL_TEXT, "                    "
    PRINT AT STATUS_ROW - 20 COLOR COL_TEXT, "                    "
    PRINT AT STATUS_ROW COLOR COL_TEXT, "                      "
    PRINT AT STATUS_ROW - 40 COLOR COL_TEXT, "TABLE MENU"
    #gs_c = COL_TEXT
    IF im_sel = 0 THEN #gs_c = COL_HILITE
    PRINT AT STATUS_ROW - 20 COLOR #gs_c, "RESUME"
    #gs_c = COL_TEXT
    IF im_sel = 1 THEN #gs_c = COL_HILITE
    PRINT AT STATUS_ROW COLOR #gs_c, "QUIT TABLE"

    WAIT
    IF inp_lock > 0 THEN inp_lock = inp_lock - 1 : GOTO im_loop

    IF CONT1.UP OR CONT1.DOWN THEN
        im_sel = 1 - im_sel
        inp_lock = 10
        GOSUB sound_cursor
        GOTO im_loop
    END IF
    IF CONT1.KEY = 10 THEN GOTO im_done
    IF CONT1.BUTTON = 0 THEN GOTO im_loop
    GOSUB sound_select

    IF im_sel = 1 THEN
        url_path = 5 : GOSUB compose_url
        #net_readlen = 8
        GOSUB api_call
        GOSUB clear_room_appkey
        want_leave = 1
    END IF
im_done:
    prev_status = 255 ' force a full CLS/board_init redraw on return
END

' ============================================================================
' name_entry_screen: disc letter picker, max 8 chars. Disc up/down cycles
' the character under the cursor through A-Z, 0-9, space; left/right moves
' the cursor; the action button accepts (at least 1 non-space char).
' Adapted from fujinet-5cardstud/intv/5card.bas, verbatim apart from colors.
' ============================================================================
name_entry_screen: PROCEDURE
    CLS
    PRINT AT 0 COLOR COL_TEXT, "ENTER YOUR NAME"
    FOR ne_i = 0 TO 7
        ne_buf(ne_i) = 36 ' space
    NEXT ne_i
    ne_cur = 0
    inp_lock = 0

ne_loop:
    FOR ne_i = 0 TO 7
        #gs_c = COL_TEXT
        IF ne_i = ne_cur THEN #gs_c = COL_HILITE
        ne_j = ne_buf(ne_i)
        IF ne_j < 26 THEN
            gs_j = 65 + ne_j
        ELSEIF ne_j < 36 THEN
            gs_j = 48 + ne_j - 26
        ELSE
            gs_j = 95 ' underscore stands in for a visible blank
        END IF
        #BACKTAB(60 + 6 + ne_i) = (gs_j - 32) * 8 + #gs_c
    NEXT ne_i

    WAIT
    IF inp_lock > 0 THEN inp_lock = inp_lock - 1 : GOTO ne_loop

    IF CONT1.RIGHT THEN
        ne_cur = ne_cur + 1
        IF ne_cur > 7 THEN ne_cur = 0
        inp_lock = 8
        GOSUB sound_cursor
        GOTO ne_loop
    END IF
    IF CONT1.LEFT THEN
        IF ne_cur = 0 THEN ne_cur = 8
        ne_cur = ne_cur - 1
        inp_lock = 8
        GOSUB sound_cursor
        GOTO ne_loop
    END IF
    IF CONT1.UP THEN
        ne_buf(ne_cur) = ne_buf(ne_cur) + 1
        IF ne_buf(ne_cur) > 36 THEN ne_buf(ne_cur) = 0
        inp_lock = 6
        GOSUB sound_cursor
        GOTO ne_loop
    END IF
    IF CONT1.DOWN THEN
        IF ne_buf(ne_cur) = 0 THEN ne_buf(ne_cur) = 37
        ne_buf(ne_cur) = ne_buf(ne_cur) - 1
        inp_lock = 6
        GOSUB sound_cursor
        GOTO ne_loop
    END IF
    IF CONT1.BUTTON = 0 THEN GOTO ne_loop
    GOSUB sound_select

    ne_len = 8
    WHILE ne_len > 0 AND ne_buf(ne_len - 1) = 36
        ne_len = ne_len - 1
    WEND
    IF ne_len = 0 THEN GOTO ne_loop
```

```
1134
1135    FOR ne_i = 0 TO ne_len - 1
1136        ne_j = ne_buf(ne_i)
1137        IF ne_j < 26 THEN
1138            gs_j = 65 + ne_j
1139        ELSE
1140            gs_j = 48 + ne_j - 26
1141        END IF
1142        POKE (SC_NAME + ne_i), gs_j
1143    NEXT ne_i
1144    POKE (SC_NAME + ne_len), 0
1145
1146    ak_creator_lo = 1 : ak_creator_hi = 0 : ak_app = 1 : ak_key = 0 : ak_mode =
1
1147    GOSUB appkey_open
1148    IF fn_ok THEN
1149        #fn_src = SC_NAME : fn_len = ne_len
1150        GOSUB appkey_write
1151        IF fn_ok = 0 THEN
1152            PRINT AT 100 COLOR COL_TEXT, "NAME NOT SAVED FOR "
1153            PRINT AT 120 COLOR COL_TEXT, "NEXT TIME           "
1154            inp_lock = 90
1155            WHILE inp_lock > 0
1156                inp_lock = inp_lock - 1
1157                WAIT
1158            WEND
1159        END IF
1160        GOSUB appkey_close
1161    END IF
1162 END
1163
1164 halt:
1165     WAIT
1166     GOTO halt
1167
```

LISTING 7 state.bas — Battleship wire format

```
1  ' state.bas -- binary GameState/Tables wire format, read in place out of FN_RX.
2  '
3  ' Per the pattern established by fujinet-5cardstud/intv/state.bas: never copy
4  ' the reply into IntyBASIC variables -- these are just fixed offsets (matching
5  ' server/util.go's serializeResults binary layout, requested with
6  ' &bin=1&v=2) plus small DEF FN accessors that PEEK straight out of FN_RX.
7  ' v=2 is not optional here: v=1 omits the winner's ships and forces
8  ' activePlayer=-1 at game over, both of which the gameover screen needs.
9  '
10 ' Header (always present), 38 bytes:
11 '   0   playerCount (1)
12 '   1   prompt[33]           lowercased, NUL-padded
13 '  34   status (1)           0 lobby, 1 place-ships, 10 gamestart,
14 '                             11 miss, 12 hit, 13 sunk, 99 gameover
15 '  35   playerStatus (1)     your status: 0 playing, 1 defeated, 2 viewing,
16 '                             3 ready, 10 place-ships
17 '  36   activePlayer (signed, $FF = -1; 0 = your turn)
18 '  37   moveTime (1, seconds, 0-250)
19 '
20 ' If status = 0 (STATUS_LOBBY), the header is followed by:
21 '  38   serverName[21]
22 '  59   players[N] -- N = playerCount, 10 bytes each: name[9] ready(1)
23 '
24 ' Otherwise (in a table, placing/playing/over):
25 '  38   lastAttackPos (1)     0-99
26 '  39   myShips[10]           [0..4] your ships, [5..9] winner's ships
```

```
'                           (v=2; only nonzero at game over), each byte is
'                           cell + 100*dir (dir 0=horizontal, 1=vertical)
'  49   players[N] -- N = playerCount, 115 bytes each:
'         name[9] status(1) gamefield[100] shipsLeft[5]
'         (gamefield/shipsLeft are present only once ships are placed and
'          the game has started -- see GAME_MINLEN/expected-length notes
'          below; entries for players who haven't placed are shorter, but
'          in practice the server only starts sending player records once
'          GAMESTART, by which point every playing player has placed.)
'
' /tables -> Tables struct:
'   0   count (1)
'   then N x 36 bytes: table[9] name[21] players[6] (literal "cur / max")

    CONST GAME_PLAYERCOUNT  = 0
    CONST GAME_PROMPT       = 1     ' 33 bytes
    CONST GAME_STATUS       = 34
    CONST GAME_PLAYERSTATUS = 35
    CONST GAME_ACTIVEPLAYER = 36
    CONST GAME_MOVETIME     = 37

    CONST LOBBY_SERVERNAME  = 38    ' 21 bytes
    CONST LOBBY_PLAYERS     = 59
    CONST LOBBY_PLAYER_STRIDE = 10
    CONST LP_NAME  = 0               ' 9 bytes
    CONST LP_READY = 9               ' 1 byte

    CONST GAME_LASTATTACK   = 38
    CONST GAME_MYSHIPS      = 39    ' 10 bytes
    CONST GAME_PLAYERS      = 49

    CONST PLAYER_STRIDE = 115
    CONST PL_NAME      = 0          ' 9 bytes
    CONST PL_STATUS    = 9          ' 1 byte
    CONST PL_GAMEFIELD = 10         ' 100 bytes
    CONST PL_SHIPSLEFT = 110        ' 5 bytes

    CONST TABLE_STRIDE = 36
    CONST TBL_ID      = 0           ' 9 bytes
    CONST TBL_NAME    = 9           ' 21 bytes
    CONST TBL_PLAYERS = 30          ' 6 bytes, literal "cur / max"

    CONST GAME_MAXLEN    = 509
    CONST GAME_MINLEN    = 38       ' fixed header, before branching on status
    CONST TABLES_MAXLEN  = 361

    ' Status values.
    CONST STATUS_LOBBY       = 0
    CONST STATUS_PLACE_SHIPS = 1
    CONST STATUS_GAMESTART   = 10
    CONST STATUS_MISS        = 11
    CONST STATUS_HIT         = 12
    CONST STATUS_SUNK        = 13
    CONST STATUS_GAMEOVER    = 99

    ' Player status values.
    CONST PSTATUS_PLAYING     = 0
    CONST PSTATUS_DEFEATED    = 1
    CONST PSTATUS_VIEWING     = 2
    CONST PSTATUS_READY       = 3
    CONST PSTATUS_PLACE_SHIPS = 10

    ' Ship sizes, fixed order (index also selects the shipsLeft/myShips slot).
    ship_size: DATA 5, 4, 3, 3, 2

' player_addr(i): address of player i's 115-byte in-game record in FN_RX.
    DEF FN player_addr(i) = FN_RX + GAME_PLAYERS + i * PLAYER_STRIDE

' lobby_player_addr(i): address of player i's 10-byte lobby record in FN_RX.
    DEF FN lobby_player_addr(i) = FN_RX + LOBBY_PLAYERS + i *
LOBBY_PLAYER_STRIDE

' table_addr(i): address of table i's 36-byte record in FN_RX (i is 0-based,
' the record right after the leading count byte).
    DEF FN table_addr(i) = FN_RX + 1 + i * TABLE_STRIDE

' active_player: activePlayer as a signed value (-1..3), converting the
' wire's $FF sentinel. Called as plain `active_player` (no parens -- DEF FN
' with no arguments is defined and invoked without them).
    DEF FN active_player = ((PEEK(FN_RX + GAME_ACTIVEPLAYER) AND 255) = 255) *
-256 + (PEEK(FN_RX + GAME_ACTIVEPLAYER) AND 255)

' state_status / state_playercount / state_playerstatus: unsigned byte reads,
' broken out as DEF FNs purely so call sites read as intent, not offsets.
    DEF FN state_status = PEEK(FN_RX + GAME_STATUS) AND 255
    DEF FN state_playercount = PEEK(FN_RX + GAME_PLAYERCOUNT) AND 255
    DEF FN state_playerstatus = PEEK(FN_RX + GAME_PLAYERSTATUS) AND 255

' ------------------------------------------------------------------------
' validate_state: layered sanity check on a /state, /ready, /place, /attack,
' or /view reply already sitting in FN_RX (#net_gotlen bytes). FN_RX is never
' cleared between calls, so a short read leaves stale bytes from a previous,
' larger response -- fixed-offset reads would happily render that as
' garbage. Sets fn_ok = 0 (in addition to whatever net/transact left it) if
' any gate fails; callers must check fn_ok after calling this, not just after
' api_call.
' ------------------------------------------------------------------------
DIM vs_status, vs_pc, #vs_expect
validate_state: PROCEDURE

    IF fn_ok = 0 OR #net_gotlen < GAME_MINLEN THEN
        fn_ok = 0
        RETURN
    END IF

    vs_status = state_status
```

```
    vs_pc = state_playercount

    IF vs_pc > 4 THEN
        fn_ok = 0
        RETURN
    END IF

    IF vs_status <> STATUS_LOBBY AND vs_status <> STATUS_PLACE_SHIPS AND
vs_status <> STATUS_GAMESTART AND vs_status <> STATUS_MISS AND vs_status <>
STATUS_HIT AND vs_status <> STATUS_SUNK AND vs_status <> STATUS_GAMEOVER THEN
        fn_ok = 0
        RETURN
    END IF

    IF vs_status = STATUS_LOBBY THEN
        #vs_expect = LOBBY_PLAYERS + vs_pc * LOBBY_PLAYER_STRIDE
    ELSEIF vs_status = STATUS_PLACE_SHIPS THEN
        ' No players[] array yet -- see the GAME_MINLEN/expected-length note
        ' above header comment: the server only starts sending player
        ' records once GAMESTART. Header + lastAttackPos + myShips only.
        #vs_expect = GAME_PLAYERS
    ELSE
        #vs_expect = GAME_PLAYERS + vs_pc * PLAYER_STRIDE
    END IF

    IF #net_gotlen < #vs_expect THEN fn_ok = 0
END
```

LISTING 8 board.bas — the colored-squares kernel

```
' board.bas -- colored-squares (40x24 4x4-block) render kernel for the four
' 10x10 Battleship boards. Requires MODE 0 (Color Stack) -- colored-squares
' cards do not exist in MODE 1 (constants.bas / IntyBASIC manual).
'
' A BACKTAB word with bit 12 = 1 and bit 11 = 0 is not a GROM/GRAM card at
' all: it is four independently-coloured 4x4 pixel quadrants (constants.bas
' calls them "pixels" 0-3, laid out TL/TR/BL/BR). Colours 0-6 are the STIC
' primaries; 7 = transparent (shows the current colour-stack colour). This
' file never uses 7 -- every cell always gets an explicit colour, so unlike
' Carlsson's original battleships-display-example.bas there is no need to
' cycle the colour stack for decoration; MODE 0's four stack slots are left
' at their compiled-in defaults and ignored.
'
' Board layout (screen is 20 cards x 12 rows; each board is 5x5 cards):
'
'   col   0-4    5    6-10       11-19
'   row0  q1  | div |  q2   |  right-hand chrome panel
'    -4             |       |  (player names, ships-left, clock --
'   row5  ---- divider row ----    owned by the caller, not this file)
'   row6  q0  | div |  q3   |
'    -10            |       |
'   row11 ......... 20-char prompt / status line .........
'
' Quadrant numbering matches the C clients' graphics.h convention: 0-3
' starting bottom-left, going clockwise; the local player is always
' quadrant 0.

    board_col: DATA 0, 0, 6, 6      ' q0..q3 origin column (card coords)
    board_row: DATA 6, 0, 0, 6      ' q0..q3 origin row

' cs_mask(sub): AND-mask that clears sub-square `sub`'s bits (0=TL,1=TR,
' 2=BL,3=BR) plus bit 11 (must always be 0 on a colored-squares card) and
' bit 12 (the enable bit) -- cs_plot ADDs CS_COLOUR_SQUARES_ENABLE back in
' unconditionally afterward, so the mask must clear it here rather than
' preserve it. An earlier version of this table preserved bit 12: on a card
' that was already a colored square (true for every cell after its first
' write), that left the ADD doubling an already-set bit 12, which carries
' into bit 13 and clears bit 12 -- turning the card back into a plain GROM/
' GRAM text card and rendering as a stray letter instead of a color.
' $D1FF (BR) additionally clears bit 13 (the BR quadrant's relocated high
```

```
' colour bit -- see cs_val below).
    cs_mask: DATA $E7F8, $E7C7, $E63F, $C1FF

' cs_val(sub*8 + color): pre-shifted bit pattern for sub-square `sub` (0-3)
' showing STIC colour `color` (0-6 primary, 7 = transparent/background).
' Pulled directly from constants.bas's CS_PIX0_*..CS_PIX3_* -- CS_PIX3_*
' already has the bit11->bit13 relocation baked in for colours 4-6 (a
' straight *512 shift for colour>=4 would land in bit 11, illegal on a
' colored-squares card; the STIC expects the BR quadrant's high colour bit
' at bit 13 instead).
    cs_val: DATA CS_PIX0_BLACK, CS_PIX0_BLUE, CS_PIX0_RED, CS_PIX0_TAN,
CS_PIX0_DARKGREEN, CS_PIX0_GREEN, CS_PIX0_YELLOW, CS_PIX0_BACKGROUND
    DATA CS_PIX1_BLACK, CS_PIX1_BLUE, CS_PIX1_RED, CS_PIX1_TAN,
CS_PIX1_DARKGREEN, CS_PIX1_GREEN, CS_PIX1_YELLOW, CS_PIX1_BACKGROUND
    DATA CS_PIX2_BLACK, CS_PIX2_BLUE, CS_PIX2_RED, CS_PIX2_TAN,
CS_PIX2_DARKGREEN, CS_PIX2_GREEN, CS_PIX2_YELLOW, CS_PIX2_BACKGROUND
    DATA CS_PIX3_BLACK, CS_PIX3_BLUE, CS_PIX3_RED, CS_PIX3_TAN,
CS_PIX3_DARKGREEN, CS_PIX3_GREEN, CS_PIX3_YELLOW, CS_PIX3_BACKGROUND

' Cell-state colours. Chosen so "empty/unknown" reads calmly (blue water)
' and hit/miss/ship read at a glance; see board_draw below for the mapping
' from the server's gamefield byte values (state.bas: 0 unknown, 1 hit,
' 2 miss) to these.
    CONST BCOL_MISS    = 0   ' black
    CONST BCOL_WATER   = 1   ' blue  -- unknown/untouched cell
    CONST BCOL_HIT     = 2   ' red
    CONST BCOL_SUNKSHIP = 3  ' tan   -- winner's revealed ships at game over
    CONST BCOL_SHIP    = 5   ' green -- own ship, own board only
    CONST BCOL_CURSOR  = 6   ' yellow
    CONST BCOL_DIVIDER = 0   ' black

' ------------------------------------------------------------------------
' cs_fill: paint an entire card (all four sub-squares) with one colour.
' Inputs: cs_col, cs_row (card coords), cs_color.
' ------------------------------------------------------------------------
DIM cs_col, cs_row, cs_q, cs_color
DIM #cs_word, cs_addr
cs_fill: PROCEDURE
    #cs_word = CS_COLOUR_SQUARES_ENABLE + cs_val(cs_color) + cs_val(8 +
cs_color) + cs_val(16 + cs_color) + cs_val(24 + cs_color)
    cs_addr = screenpos(cs_col, cs_row)
    #BACKTAB(cs_addr) = #cs_word
END

' ------------------------------------------------------------------------
' cs_plot: read-modify-write a single sub-square, preserving the other
' three. Inputs: cs_col, cs_row (card coords), cs_q (0-3), cs_color.
' ------------------------------------------------------------------------
cs_plot: PROCEDURE
    cs_addr = screenpos(cs_col, cs_row)
    #BACKTAB(cs_addr) = (#BACKTAB(cs_addr) AND cs_mask(cs_q)) +
CS_COLOUR_SQUARES_ENABLE + cs_val(cs_q * 8 + cs_color)
END

' ------------------------------------------------------------------------
' board_init: paint all four boards to water and the dividers to black.
' Call once after MODE 0 / CLS, before the first board_draw.
' ------------------------------------------------------------------------
DIM bi_bq, bi_cc, bi_cr
board_init: PROCEDURE
    FOR bi_bq = 0 TO 3
        FOR bi_cr = 0 TO 4
            FOR bi_cc = 0 TO 4
                cs_col = board_col(bi_bq) + bi_cc
                cs_row = board_row(bi_bq) + bi_cr
                cs_color = BCOL_WATER
                GOSUB cs_fill
            NEXT bi_cc
        NEXT bi_cr
        WAIT
    NEXT bi_bq

    ' Divider: column 5 for all 11 board rows, row 5 for all 11 board
    ' columns except column 5 itself (already painted by the first loop).
    FOR bi_cr = 0 TO 10
        cs_col = 5 : cs_row = bi_cr : cs_color = BCOL_DIVIDER : GOSUB cs_fill
    NEXT bi_cr
    FOR bi_cc = 0 TO 10
        IF bi_cc <> 5 THEN
            cs_col = bi_cc : cs_row = 5 : cs_color = BCOL_DIVIDER : GOSUB
cs_fill
        END IF
    NEXT bi_cc
    WAIT
END

' ------------------------------------------------------------------------
' board_draw: full 25-card repaint of quadrant bd_bq's 10x10 grid from a
' 100-byte gamefield buffer (state.bas values: 0 unknown, 1 hit, 2 miss) at
' RAM address #bd_field. One WAIT per board -- 25 single-write cards per
' frame stays well inside vblank; the lesson from 5cardstud's differential
' renderer (5card.bas) is that an unbroken multi-region burst does not.
' Ship overlay (own board only) is layered on afterward by the caller via
' board_cell, since the gamefield alone doesn't carry ship positions.
' ------------------------------------------------------------------------
DIM bd_bq, #bd_field
DIM bd_cc, bd_cr, bd_sub, bd_cell, bd_state, bd_color
DIM #bd_word, bd_addr
board_draw: PROCEDURE
    FOR bd_cr = 0 TO 4
        FOR bd_cc = 0 TO 4
            #bd_word = CS_COLOUR_SQUARES_ENABLE
            FOR bd_sub = 0 TO 3
                ' sub 0=TL(x even,y even) 1=TR(x odd,y even)
                ' sub 2=BL(x even,y odd)  3=BR(x odd,y odd)
```

```
                bd_cell = (bd_cr * 2 + bd_sub / 2) * 10 + (bd_cc * 2 + (bd_sub
AND 1))
                bd_state = PEEK(#bd_field + bd_cell) AND 255
                IF bd_state = 1 THEN
                    bd_color = BCOL_HIT
                ELSEIF bd_state = 2 THEN
                    bd_color = BCOL_MISS
                ELSE
                    bd_color = BCOL_WATER
                END IF
                #bd_word = #bd_word + cs_val(bd_sub * 8 + bd_color)
            NEXT bd_sub
            bd_addr = screenpos(board_col(bd_bq) + bd_cc, board_row(bd_bq) +
bd_cr)
            #BACKTAB(bd_addr) = #bd_word
        NEXT bd_cc
        WAIT
    NEXT bd_cr
END

' ------------------------------------------------------------------------
' board_cell: update a single cell (x,y both 0-9) on quadrant bc_bq to
' bc_color. Used for incremental attack-result updates, own-ship overlay
' during/after placement, and the targeting cursor (caller is responsible
' for restoring the underlying colour when the cursor moves off a cell --
' this file is stateless and does not remember what was drawn).
' ------------------------------------------------------------------------
DIM bc_bq, bc_x, bc_y, bc_color
board_cell: PROCEDURE
    cs_col = board_col(bc_bq) + bc_x / 2
    cs_row = board_row(bc_bq) + bc_y / 2
    cs_q = (bc_y AND 1) * 2 + (bc_x AND 1)
    cs_color = bc_color
    GOSUB cs_plot
END
```

LISTING 9 sound.bas — Battleship effects

```
' sound.bas -- sound effects, adapted from fujinet-5cardstud/intv/sound.bas.
' The transport primitive (play_tone) and several UI-generic effects are
' kept verbatim; the game-specific effects are replaced with Battleship's
' own set, matching the C clients' platform-specific/sound.h event names
' (soundCursor/Select/MyTurn/JoinGame/... plus Attack/Hit/Sink/Miss/Invalid
' which 5 Card Stud has no equivalent of).
'
' All effects play on PSG channel 0 -- turn-based game, non-overlapping UI
' events, no need for polyphony (and SOUND's channel argument must be a
' compile-time constant anyway, which rules out a generic multi-channel
' helper).
'
' SOUND's frequency argument is a 12-bit PSG divisor, computed as
' round(3579545/32/hz) for NTSC. These are precomputed rather than divided
' at runtime: 3579545/32 alone is ~111861, which overflows IntyBASIC's
' 16-bit variables (max 65535), so it only works as a compile-time-folded
' literal division -- not as CONST-divided-by-a-runtime-variable. Every
' effect here uses a fixed, known-in-advance frequency, so precomputing
' sidesteps the overflow entirely.
'   50Hz->2237  60Hz->1864  70Hz->1598  80Hz->1398  100Hz->1119
'  150Hz->746  200Hz->559   300Hz->373  311Hz->360   330Hz->339
'  340Hz->329  350Hz->320   392Hz->285  415Hz->270    430Hz->260
'  500Hz->224  800Hz->140
    CONST SND_VOL = 12
```

```
25
26      DIM #snd_val, snd_gate, snd_post, snd_i
27
28  ' ------------------------------------------------------------------------
29  ' play_tone: one square-wave note on channel 0. #snd_val = precomputed PSG
30  ' divisor (see table above), snd_gate = frames the tone sounds, snd_post =
31  ' frames of silence after.
32  ' ------------------------------------------------------------------------
33  play_tone: PROCEDURE
34      SOUND 0, #snd_val, SND_VOL
35      FOR snd_i = 1 TO snd_gate
36          WAIT
37      NEXT snd_i
38      SOUND 0, 0, 0
39      FOR snd_i = 1 TO snd_post
40          WAIT
41      NEXT snd_i
42  END
43
44  ' sound_join: sit down at a table (3-tone rising figure: 430,340,500 Hz).
45  sound_join: PROCEDURE
46      #snd_val = 260 : snd_gate = 5 : snd_post = 8 : GOSUB play_tone
47      #snd_val = 329 : snd_gate = 5 : snd_post = 0 : GOSUB play_tone
48      #snd_val = 224 : snd_gate = 5 : snd_post = 0 : GOSUB play_tone
49  END
50
51  ' sound_myturn: it's your turn to fire (double beep, 430,430 Hz).
52  sound_myturn: PROCEDURE
53      #snd_val = 260 : snd_gate = 4 : snd_post = 2 : GOSUB play_tone
54      #snd_val = 260 : snd_gate = 4 : snd_post = 2 : GOSUB play_tone
55  END
56
57  ' sound_gamedone: game over, win fanfare (4-tone rising: 311,330,392,415 Hz).
58  sound_gamedone: PROCEDURE
59      #snd_val = 360 : snd_gate = 10 : snd_post = 0 : GOSUB play_tone
60      #snd_val = 339 : snd_gate = 20 : snd_post = 0 : GOSUB play_tone
61      #snd_val = 285 : snd_gate = 10 : snd_post = 0 : GOSUB play_tone
62      #snd_val = 270 : snd_gate = 20 : snd_post = 0 : GOSUB play_tone
63  END
64
65  ' sound_player_join: another player sits down mid-lobby (5-tone rising
66  ' sweep: 50,60,70,80 Hz).
67  sound_player_join: PROCEDURE
68      #snd_val = 2237 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
69      #snd_val = 1864 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
70      #snd_val = 1598 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
71      #snd_val = 1398 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
72  END
73
74  ' sound_player_left: a player leaves mid-game (falling sweep, mirror of join).
75  sound_player_left: PROCEDURE
76      #snd_val = 1398 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
77      #snd_val = 1598 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
78      #snd_val = 1864 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
79      #snd_val = 2237 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
80  END
81
82  ' sound_select: a menu/table/ready choice was confirmed (2-tone rising
83  ' chirp: 300,350 Hz).
84  sound_select: PROCEDURE
85      #snd_val = 373 : snd_gate = 3 : snd_post = 1 : GOSUB play_tone
86      #snd_val = 320 : snd_gate = 3 : snd_post = 0 : GOSUB play_tone
87  END
88
89  ' sound_cursor: cursor/ship moved one position (300 Hz).
90  sound_cursor: PROCEDURE
91      #snd_val = 373 : snd_gate = 2 : snd_post = 0 : GOSUB play_tone
92  END
93
94  ' sound_tick: countdown clock tick during targeting (short 300 Hz click).
95  sound_tick: PROCEDURE
96      #snd_val = 373 : snd_gate = 1 : snd_post = 0 : GOSUB play_tone
97  END
98
99  ' sound_place_ship: a ship was confirmed placed (short rising chirp,
100 ' 300,500 Hz).
101 sound_place_ship: PROCEDURE
102     #snd_val = 373 : snd_gate = 2 : snd_post = 0 : GOSUB play_tone
103     #snd_val = 224 : snd_gate = 3 : snd_post = 2 : GOSUB play_tone
104 END
105
106 ' sound_attack: a shot was fired (falling swoosh, 500,300 Hz).
107 sound_attack: PROCEDURE
108     #snd_val = 224 : snd_gate = 3 : snd_post = 0 : GOSUB play_tone
109     #snd_val = 373 : snd_gate = 3 : snd_post = 0 : GOSUB play_tone
110 END
111
112 ' sound_miss: shot landed in water (single splash-like click, 800 Hz).
113 sound_miss: PROCEDURE
114     #snd_val = 140 : snd_gate = 4 : snd_post = 4 : GOSUB play_tone
115 END
116
117 ' sound_hit: shot struck a ship (low descending buzz, 200,100 Hz).
118 sound_hit: PROCEDURE
119     #snd_val = 559 : snd_gate = 4 : snd_post = 0 : GOSUB play_tone
120     #snd_val = 1119 : snd_gate = 6 : snd_post = 4 : GOSUB play_tone
121 END
122
123 ' sound_sink: a ship was sunk (4-tone falling fanfare, mirror of
124 ' sound_gamedone: 415,392,330,311 Hz).
125 sound_sink: PROCEDURE
126     #snd_val = 270 : snd_gate = 10 : snd_post = 0 : GOSUB play_tone
127     #snd_val = 285 : snd_gate = 10 : snd_post = 0 : GOSUB play_tone
128     #snd_val = 339 : snd_gate = 10 : snd_post = 0 : GOSUB play_tone
129     #snd_val = 360 : snd_gate = 15 : snd_post = 0 : GOSUB play_tone
130 END
```

```
131
132 ' sound_invalid: rejected input -- ship placement collision, spent attack
133 ' cell, out-of-turn action (single low buzz, 100 Hz, held).
134 sound_invalid: PROCEDURE
135     #snd_val = 1119 : snd_gate = 10 : snd_post = 5 : GOSUB play_tone
136 END
137
```

LISTING 10 fujitzee.bas — Fujitzee client (Chapter 10)

```
  1 ' fujitzee.bas -- FujiNet Fujitzee (Yahtzee-style) for Intellivision, entry
  2 ' point + state machine. Follows the pattern established by
  3 ' fujinet-5cardstud/intv/5card.bas and fujinet-battleship/intv/battleship.bas
  4 ' (name entry -> table select -> game loop -> in-game menu), adapted for a
  5 ' scorecard game: the 20x12 screen can't show all 6 players' 15-category
  6 ' cards at once, so one player's card is shown at a time (card.bas), paged
  7 ' with the disc when it isn't your turn, alongside a permanent 6-seat strip.
  8 '
  9 ' GOTO boot_start jumps over every INCLUDE below before any of it runs --
 10 ' see fujinet.bas for why: falling into a PROCEDURE or DATA block by
 11 ' straight-line execution corrupts the return stack.
 12     GOTO boot_start
 13
 14     INCLUDE "constants.bas"
 15     INCLUDE "fujinet.bas"
 16     INCLUDE "state.bas"
 17     INCLUDE "dice.bas"
 18     INCLUDE "card.bas"
 19     INCLUDE "sound.bas"
 20
 21     CONST ROWCELLS = 20
 22     CONST STATUS_ROW = screenpos(0, 11)
 23
 24 ' ---------------------------------------------------------------------------
 25 ' URL literals (ASCII DATA). url_path selects the endpoint:
 26 '   0 tables   1 state   2 ready   3 roll/<mask>   4 score/<idx>   5 leave
 27 ' ---------------------------------------------------------------------------
 28 lit_n_colon: DATA 78,58
 29 lit_https: DATA 104,116,116,112,115,58,47,47,102,117,106,105,116,122,101,101,46,
99,97,114,114,45,100,101,115,105,103,110,115,46,99,111,109,47
 30 lit_tables: DATA 116,97,98,108,101,115
 31 lit_qbin: DATA 63,98,105,110,61,49
 32 lit_state: DATA 115,116,97,116,101
 33 lit_ready: DATA 114,101,97,100,121
 34 lit_roll: DATA 114,111,108,108,47
 35 lit_score: DATA 115,99,111,114,101,47
 36 lit_leave: DATA 108,101,97,118,101
 37 lit_qtable: DATA 63,116,97,98,108,101,61
 38 lit_aplayer: DATA 38,112,108,97,121,101,114,61
 39 lit_abin: DATA 38,98,105,110,61,49
 40
 41     CONST LEN_HTTPS = 34
 42
 43     ' Lobby appkey: creator 1 / app 1, key = this game's registered slot.
 44     ' See fujinet-fujitzee/src/misc.h AK_LOBBY_KEY_SERVER.
 45     CONST AK_LOBBY_KEY_SERVER = 3
 46
 47 ' ---------------------------------------------------------------------------
 48 ' Globals
 49 ' ---------------------------------------------------------------------------
 50     DIM gs_i, gs_j, #gs_char, #gs_c
 51     DIM ne_i, ne_j, ne_cur, ne_len
 52     DIM ne_buf(8)
 53     DIM ak_try
 54     DIM #tbl_count, tbl_sel
 55     DIM inp_lock
 56     DIM want_leave, im_sel
 57     DIM url_path
 58     DIM #tmp_addr
 59
 60     ' split_room_url locals (see procedure below)
 61     DIM sp_i, sp_found, sp_ok, sp_j, sp_k, sp_c, sp_valid, sp_m
 62
 63     DIM #cur_round, #cur_pc, prev_round, prev_active, prev_seen_pc, my_turn
 64     DIM poll_wait, has_action
 65     DIM turn_changed, roll_changed, #prev_dice_rollsleft, shadow_seeded
 66     DIM view_lock, ui_mode, dice_cur, crs_idx, score_idx
 67
 68     DIM pn_val, pn_h, pn_t, pn_o, pn_started
 69     DIM #ti_timeleft, ti_framecount, ti_allkept, pt_i, score_is_fujitzee,
at_bottom_row
 70
 71 ' ---------------------------------------------------------------------------
 72 ' fn_putnum: append the decimal (no leading zeros) representation of pn_val
 73 ' (0-999) into FN_TX at the current #fn_txlen. IntyBASIC has no built-in
 74 ' itoa; score indices only run 0-14, but a 3-digit routine costs nothing
 75 ' extra and matches fujinet-battleship/intv/battleship.bas's version.
 76 ' ---------------------------------------------------------------------------
 77 fn_putnum: PROCEDURE
 78     pn_h = pn_val / 100
 79     pn_t = (pn_val / 10) % 10
 80     pn_o = pn_val % 10
```

```
    pn_started = 0
    IF pn_h > 0 THEN
        POKE (FN_TX + #fn_txlen), pn_h + 48 : #fn_txlen = #fn_txlen + 1
        pn_started = 1
    END IF
    IF pn_t > 0 OR pn_started THEN
        POKE (FN_TX + #fn_txlen), pn_t + 48 : #fn_txlen = #fn_txlen + 1
    END IF
    POKE (FN_TX + #fn_txlen), pn_o + 48 : #fn_txlen = #fn_txlen + 1
END

' ------------------------------------------------------------------------
' draw_field: render df_len ASCII bytes from #df_src onto the screen at
' BACKTAB offset df_pos, in color #df_color. MODE 1 only exposes GROM cards
' 0-63 (ASCII 32-95, no lowercase), so unlike Battleship's MODE 0 client
' this uppercases -- server strings (names, prompts) arrive lowercased.
' Stops at a NUL and pads the rest of the field with spaces; clamps
' anything outside the printable range to a space rather than drawing
' garbage. Ported from fujinet-5cardstud/intv/5card.bas's version.
' ------------------------------------------------------------------------
DIM df_i, #df_c, df_stop
draw_field: PROCEDURE
    df_stop = 0
    FOR df_i = 0 TO df_len - 1
        #df_c = (PEEK(#df_src + df_i) AND 255)
        IF #df_c = 0 THEN df_stop = 1
        IF df_stop THEN
            #df_c = 32
        ELSEIF #df_c >= 97 AND #df_c <= 122 THEN
            #df_c = #df_c - 32
        END IF
        IF #df_c < 32 OR #df_c > 95 THEN #df_c = 32
        #BACKTAB(df_pos + df_i) = (#df_c - 32) * 8 + #df_color
    NEXT df_i
END

' ------------------------------------------------------------------------
' cls_blue: CLS, then fill the whole 240-cell BACKTAB with a blank blue
' card. CLS alone leaves every cell at card 0 / color 0 (black), so
' without this every screen would still show a black field behind
' whatever text gets drawn on top of it. Every CLS in this file goes
' through here instead, matching fujinet-5cardstud/intv/5card.bas's
' fill_bg pattern.
' ------------------------------------------------------------------------
DIM cb_i
cls_blue: PROCEDURE
    CLS
    FOR cb_i = 0 TO 239
        #BACKTAB(cb_i) = COL_TEXT
    NEXT cb_i
END

' ------------------------------------------------------------------------
' compose_url: build "N:<endpoint><path...><suffix>" directly into FN_TX.
' url_path=3 (roll) appends SC_KEEP's 5 raw ascii mask bytes directly (it's
' already in wire format, no NUL). url_path=4 (score) appends score_idx via
' fn_putnum.
' ------------------------------------------------------------------------
compose_url: PROCEDURE
    #fn_txlen = 0
    #fn_src = VARPTR lit_n_colon(0) : fn_len = 2 : GOSUB fn_putstr
    #fn_src = SC_ENDPT : ls_max = 65 : GOSUB fn_strlen : GOSUB fn_putstr

    IF url_path = 0 THEN
        #fn_src = VARPTR lit_tables(0) : fn_len = 6 : GOSUB fn_putstr
        #fn_src = VARPTR lit_qbin(0) : fn_len = 6 : GOSUB fn_putstr
    ELSE
        IF url_path = 1 THEN
            #fn_src = VARPTR lit_state(0) : fn_len = 5 : GOSUB fn_putstr
        ELSEIF url_path = 2 THEN
            #fn_src = VARPTR lit_ready(0) : fn_len = 5 : GOSUB fn_putstr
        ELSEIF url_path = 3 THEN
            #fn_src = VARPTR lit_roll(0) : fn_len = 5 : GOSUB fn_putstr
            #fn_src = SC_KEEP : fn_len = 5 : GOSUB fn_putstr
        ELSEIF url_path = 4 THEN
            #fn_src = VARPTR lit_score(0) : fn_len = 6 : GOSUB fn_putstr
            pn_val = score_idx
            GOSUB fn_putnum
        ELSE
            #fn_src = VARPTR lit_leave(0) : fn_len = 5 : GOSUB fn_putstr
        END IF

        #fn_src = VARPTR lit_qtable(0) : fn_len = 7 : GOSUB fn_putstr
        #fn_src = SC_TABLE : ls_max = 9 : GOSUB fn_strlen : GOSUB fn_putstr
        #fn_src = VARPTR lit_aplayer(0) : fn_len = 8 : GOSUB fn_putstr
        #fn_src = SC_NAME : ls_max = 9 : GOSUB fn_strlen : GOSUB fn_putstr
        #fn_src = VARPTR lit_abin(0) : fn_len = 6 : GOSUB fn_putstr
    END IF
END

' ========================================================================
' Boot
' ========================================================================
boot_start:
    MODE 1
    GOSUB cls_blue
    GOSUB dice_init
    PRINT AT 0 COLOR COL_TEXT, "FUJITZEE"
    PRINT AT 40, "CONNECTING TO FUJINET"
    GOSUB fn_wait_mailbox
    IF fn_ok = 0 THEN
        PRINT AT 40, "NO CARTRIDGE MAILBOX"
        GOTO halt
    END IF

' ------------------------------------------------------------------------
```

```
' Name: try the shared lobby appkey (creator 1 / app 1 / key 0) first, with
' one retry (a cold RP2040/ESP32 link right after the mailbox comes up can
' time out the very first transaction); fall back to the disc letter picker
' if the read fails, comes back empty, or holds anything outside A-Z0-9 --
' this slot is shared by every FujiNet client, and a stray character here
' goes straight into the query string unescaped.
' ------------------------------------------------------------------------
    gs_i = 0
    FOR ak_try = 0 TO 1
        ak_creator_lo = 1 : ak_creator_hi = 0 : ak_app = 1 : ak_key = 0 :
ak_mode = 0
        GOSUB appkey_open
        IF fn_ok THEN EXIT FOR
    NEXT ak_try
    IF fn_ok THEN
        #fn_src = SC_NAME : ls_max = 9 : GOSUB appkey_read
        GOSUB appkey_close
        IF fn_len > 0 THEN
            gs_i = 1
            FOR gs_j = 0 TO fn_len - 1
                #gs_char = (PEEK(SC_NAME + gs_j) AND 255)
                IF #gs_char >= 97 AND #gs_char <= 122 THEN #gs_char = #gs_char -
32
                IF (#gs_char < 65 OR #gs_char > 90) AND (#gs_char < 48 OR
#gs_char > 57) THEN gs_i = 0
                NEXT gs_j
        END IF
    END IF
    IF gs_i = 0 THEN GOSUB name_entry_screen

' ------------------------------------------------------------------------
' Room: check the lobby server appkey (creator 1 / app 1 / key
' AK_LOBBY_KEY_SERVER). If the lobby wrote a room there, split it into
' SC_ENDPT/SC_TABLE and skip straight past table select. Seed the
' compiled-in default endpoint first so SC_ENDPT is always valid even if
' the appkey is empty or the read fails -- compose_url relies on it from
' here on for every request, not just /tables.
' ------------------------------------------------------------------------
    FOR gs_i = 0 TO LEN_HTTPS - 1
        POKE (SC_ENDPT + gs_i), PEEK(VARPTR lit_https(0) + gs_i) AND 255
    NEXT gs_i
    POKE (SC_ENDPT + LEN_HTTPS), 0
    POKE SC_TABLE, 0

    FOR ak_try = 0 TO 1
        ak_creator_lo = 1 : ak_creator_hi = 0 : ak_app = 1
        ak_key = AK_LOBBY_KEY_SERVER : ak_mode = 0
        GOSUB appkey_open
        IF fn_ok THEN EXIT FOR
    NEXT ak_try
    IF fn_ok THEN
        #fn_src = SC_ENDPT : ls_max = 65 : GOSUB appkey_read
        GOSUB appkey_close
        IF fn_len > 0 THEN
            GOSUB split_room_url
        ELSE
            ' appkey_read always NUL-terminates at fn_len, even when
            ' empty, which would otherwise wipe out the default just
            ' seeded above -- restore it.
            FOR gs_i = 0 TO LEN_HTTPS - 1
                POKE (SC_ENDPT + gs_i), PEEK(VARPTR lit_https(0) + gs_i) AND 255
            NEXT gs_i
            POKE (SC_ENDPT + LEN_HTTPS), 0
        END IF
    END IF

    IF (PEEK(SC_TABLE) AND 255) <> 0 THEN GOTO table_joined

' ========================================================================
' Table select (re-entered after leaving a table)
' ========================================================================
table_select:
    GOSUB cls_blue
    PRINT AT 0 COLOR COL_TEXT, "CHOOSE A TABLE"
    PRINT AT 40, "REFRESHING..."

    url_path = 0 : GOSUB compose_url
    #net_readlen = TABLES_MAXLEN
    GOSUB api_call

    IF fn_ok = 1 THEN
        #tmp_addr = 1 + (PEEK(FN_RX) AND 255) * TABLE_STRIDE
        IF #net_gotlen < #tmp_addr THEN fn_ok = 0
    END IF

    IF fn_ok = 0 THEN
        PRINT AT 40, "SERVER UNREACHABLE  "
        inp_lock = 90
        WHILE inp_lock > 0
            inp_lock = inp_lock - 1
            WAIT
        WEND
        GOTO table_select
    END IF

    #tbl_count = (PEEK(FN_RX) AND 255)
    IF #tbl_count > 8 THEN #tbl_count = 8
    IF #tbl_count = 0 THEN
        PRINT AT 40, "NO TABLES AVAILABLE "
        inp_lock = 90
        WHILE inp_lock > 0
            inp_lock = inp_lock - 1
            WAIT
        WEND
        GOTO table_select
    END IF
```

```
290
291     GOSUB cls_blue
292     PRINT AT 0 COLOR COL_TEXT, "CHOOSE A TABLE"
293     FOR gs_i = 0 TO #tbl_count - 1
294         #tmp_addr = table_addr(gs_i)
295         #df_src = #tmp_addr + TBL_NAME : df_pos = 20 + gs_i * 20 + 1 : df_len =
 14 : #df_color = COL_TEXT
296         GOSUB draw_field
297         #df_src = #tmp_addr + TBL_PLAYERS : df_pos = 20 + gs_i * 20 + 15 :
df_len = 5 : #df_color = COL_TEXT
298         GOSUB draw_field
299     NEXT gs_i
300
301     tbl_sel = 0
302     inp_lock = 0
303 ts_input:
304     WAIT
305     FOR gs_i = 0 TO #tbl_count - 1
306         #gs_c = COL_TEXT
307         IF gs_i = tbl_sel THEN #gs_c = COL_HILITE
308         #BACKTAB(20 + gs_i * 20) = (62 - 32) * 8 + #gs_c ' '>' cursor glyph
309     NEXT gs_i
310
311     IF inp_lock > 0 THEN inp_lock = inp_lock - 1 : GOTO ts_input
312
313     IF CONT1.DOWN THEN
314         tbl_sel = tbl_sel + 1
315         IF tbl_sel >= #tbl_count THEN tbl_sel = 0
316         inp_lock = 8
317         GOSUB sound_cursor
318         GOTO ts_input
319     END IF
320     IF CONT1.UP THEN
321         IF tbl_sel = 0 THEN tbl_sel = #tbl_count
322         tbl_sel = tbl_sel - 1
323         inp_lock = 8
324         GOSUB sound_cursor
325         GOTO ts_input
326     END IF
327     IF CONT1.BUTTON = 0 THEN GOTO ts_input
328     GOSUB sound_select
329
330     #tmp_addr = table_addr(tbl_sel)
331     FOR gs_i = 0 TO 8
332         POKE (SC_TABLE + gs_i), PEEK(#tmp_addr + TBL_ID + gs_i) AND 255
333     NEXT gs_i
334
335     ' Update the lobby server appkey so a reboot without going back
336     ' through the lobby rejoins this table.
337     GOSUB write_room_appkey
338
339 table_joined:
340 ' ============================================================================
341 ' In a table: poll /state (or submit a pending ready/roll/score action in
342 ' its place) and dispatch on round. A full CLS happens only on the
343 ' transition into/out of play (lobby<->play<->gameover); every other poll
344 ' updates cells in place -- the STIC has no page-flip, so a per-poll CLS
345 ' would visibly blank the screen every cycle.
346 ' ============================================================================
347     prev_round = 255  ' sentinel: forces the initial CLS/card_init
348     prev_active = 255 ' sentinel: forces the initial turn-start reset+sound
349     prev_seen_pc = 255 ' sentinel: skip the join/leave cue on the first poll
350     poll_wait = 0
351     want_leave = 0
352     has_action = 0
353     view_idx = 0 : view_lock = 0
354     ui_mode = 0 : crs_idx = 0 : dice_cur = 0
355     GOSUB sound_join
356
357 game_loop:
358     WAIT
359
360     IF poll_wait > 0 THEN
361         poll_wait = poll_wait - 1
362         GOTO gl_input
363     END IF
364
365     IF has_action = 1 THEN
366         url_path = 2
367     ELSEIF has_action = 2 THEN
368         url_path = 3
369     ELSEIF has_action = 3 THEN
370         url_path = 4
371     ELSE
372         url_path = 1
373     END IF
374     has_action = 0
375     GOSUB compose_url
376     #net_readlen = GAME_MAXLEN
377     GOSUB api_call
378     GOSUB validate_state
379
380     IF fn_ok = 0 THEN
381         poll_wait = 30
382         GOTO gl_input
383     END IF
384
385     #cur_round = state_round
386     #cur_pc = state_playercount
387     IF view_idx >= #cur_pc THEN view_idx = 0
388
389     IF prev_seen_pc <> 255 THEN
390         IF #cur_pc > prev_seen_pc THEN GOSUB sound_player_join
391         IF #cur_pc < prev_seen_pc THEN GOSUB sound_player_left
392     END IF
393     prev_seen_pc = #cur_pc
```

```
394
395     IF #cur_round = ROUND_LOBBY THEN
396         IF prev_round <> ROUND_LOBBY THEN GOSUB cls_blue
397         GOSUB render_lobby
398         poll_wait = 45
399     ELSEIF #cur_round = ROUND_GAMEOVER THEN
400         IF prev_round <> ROUND_GAMEOVER THEN
401             GOSUB cls_blue
402             GOSUB render_gameover
403         END IF
404         poll_wait = 60
405     ELSE
406         IF prev_round = ROUND_LOBBY OR prev_round = 255 OR prev_round =
ROUND_GAMEOVER THEN
407             GOSUB cls_blue
408             GOSUB card_init
409             view_idx = 0 : ui_mode = 0 : crs_idx = 0 : dice_cur = 0
410             #prev_dice_rollsleft = 255 ' sentinel: don't animate the first paint
411             shadow_seeded = 0 ' sentinel: don't reveal-check against garbage
412             FOR pt_i = 0 TO 4
413                 POKE (SC_KEEP + pt_i), 49
414             NEXT pt_i
415         END IF
416
417         ' Whenever the active player changes (a new turn begins, yours or
418         ' someone else's), snap the viewed card to follow them, like the
419         ' other clients -- no need to manually page to see who's rolling.
420         ' Manual disc-paging in gl_input still works to peek elsewhere
421         ' while waiting; it's just overridden at the next turn change.
422         turn_changed = 0
423         IF active_player <> prev_active THEN turn_changed = 1
424
425         ' Someone other than you just finished their turn (scored, or
426         ' timed out and got auto-scored) -- flash whatever they picked
427         ' before the view below flips over to whoever's active now. Skips
428         ' your own turn ending: you already saw that happen live, driven
429         ' by turn_input.
430         IF shadow_seeded THEN
431             IF turn_changed THEN
432                 IF prev_active <> 0 THEN
433                     IF prev_active >= 0 THEN
434                         IF prev_active < #cur_pc THEN GOSUB check_score_reveal
435                     END IF
436                 END IF
437             END IF
438         END IF
439
440         ' Deliberately nested single-condition IFs, not
441         ' "IF active_player = 0 AND state_viewing = 0 THEN": IntyBASIC
442         ' v1.4.2 miscompiles a chain of two "X = const" comparisons joined
443         ' by AND when one side's value comes from an expression that
444         ' itself contains "AND 255" (state_viewing's DEF FN body, or
445         ' active_player's sign-extension formula) -- the fused codegen
446         ' ANDs in a stray always-zero register instead of the second
447         ' comparison, so the whole condition is always false. Confirmed
448         ' by reading the generated .lst: it never called turn_input.
449         my_turn = 0
450         IF active_player = 0 THEN
451             IF state_viewing = 0 THEN my_turn = 1
452         END IF
453
454         IF turn_changed THEN
455             IF active_player >= 0 THEN
456                 IF active_player < #cur_pc THEN view_idx = active_player
457             END IF
458             IF my_turn THEN
459                 GOSUB sound_myturn
460                 ui_mode = 0 : crs_idx = 0 : dice_cur = 0
461                 FOR pt_i = 0 TO 4
462                     POKE (SC_KEEP + pt_i), 49
463                 NEXT pt_i
464             END IF
465         END IF
466
467         ' Snapshot whoever's active now, so whenever *their* turn ends we
468         ' have a valid "before" state for check_score_reveal to diff
469         ' against -- done every poll (cheap: 15 words), not just on
470         ' turn_changed, since it must reflect their latest scoring right
471         ' up until the moment they stop being active.
472         IF active_player >= 0 THEN
473             IF active_player < #cur_pc THEN
474                 GOSUB seed_score_shadow
475                 shadow_seeded = 1
476             END IF
477         END IF
478
479         ' A roll landed for whoever's turn it is -- rollsLeft changed
480         ' since the last poll, or the turn itself just changed (covers
481         ' the server's automatic opening roll). Flicker the dice the
482         ' wire's keepRoll says were rerolled before drawing the settled
483         ' result; see dice.bas's animate_roll for why this same check
484         ' animates both your own rolls and every other player's.
485         roll_changed = 0
486         IF #prev_dice_rollsleft <> 255 THEN
487             IF state_rollsleft <> #prev_dice_rollsleft THEN roll_changed = 1
488             IF turn_changed THEN roll_changed = 1
489             IF roll_changed THEN GOSUB animate_roll
490         END IF
491         #prev_dice_rollsleft = state_rollsleft
492
493         ' Out of rolls -- jump to SCORE mode automatically, like the
494         ' other clients (gamelogic.c defaults cursorPos onto the
495         ' highest-scoring open category the moment rollsLeft hits 0).
496         ' Gated on roll_changed so this fires once, not every poll while
497         ' sitting at 0 (which would otherwise fight any manual navigation
```

```
        ' the player had already done).
        IF roll_changed THEN
            IF my_turn THEN
                IF state_rollsleft = 0 THEN
                    ui_mode = 1
                    GOSUB pick_best_score
                END IF
            END IF
        END IF

        GOSUB render_playscreen

        IF my_turn THEN
            GOSUB turn_input
            IF has_action THEN poll_wait = 0 ELSE poll_wait = 20
        ELSE
            poll_wait = 20
        END IF
    END IF
    prev_active = active_player
    prev_round = #cur_round

gl_input:
    ' Page the viewed player's card with the disc when it isn't your turn
    ' (during your own turn, turn_input owns the disc for die/category
    ' selection instead). No network round-trip needed -- every seated
    ' player's record already arrived in the same poll. Reuses my_turn
    ' (computed once per successful poll, above) rather than re-testing
    ' active_player/state_viewing here -- see the comment at that
    ' computation for why chaining those DEF FNs into a multi-condition
    ' AND on one line miscompiles on this compiler.
    IF #cur_round >= 1 THEN
        IF #cur_round <= ROUND_FINAL THEN
            IF my_turn = 0 THEN
                IF view_lock > 0 THEN view_lock = view_lock - 1
                IF CONT1.RIGHT AND view_lock = 0 THEN
                    view_idx = view_idx + 1
                    IF view_idx >= #cur_pc THEN view_idx = 0
                    view_lock = 8
                    GOSUB sound_cursor
                    GOSUB render_header
                    GOSUB render_card_for_view
                ELSEIF CONT1.LEFT AND view_lock = 0 THEN
                    IF view_idx = 0 THEN view_idx = #cur_pc
                    view_idx = view_idx - 1
                    view_lock = 8
                    GOSUB sound_cursor
                    GOSUB render_header
                    GOSUB render_card_for_view
                END IF
            END IF
        END IF
    END IF

    IF CONT1.KEY = 10 THEN GOSUB ingame_menu
    IF want_leave THEN
        want_leave = 0
        GOTO table_select
    END IF
    GOTO game_loop

' ============================================================================
' render_lobby: server name, ready list, prompt. Trigger toggles /ready.
' Uses the same 42-byte player records as play -- scores[0] doubles as the
' ready flag while round=0 (SCORE_READY/SCORE_UNREADY/SCORE_VIEWING).
' ============================================================================
DIM #lb_pc, lb_ready
render_lobby: PROCEDURE
    #df_src = FN_RX + GAME_SERVERNAME : df_pos = screenpos(0, 0) : df_len = 20 :
#df_color = COL_TEXT
    GOSUB draw_field

    #lb_pc = state_playercount
    IF #lb_pc > 8 THEN #lb_pc = 8
    FOR gs_i = 0 TO #lb_pc - 1
        #tmp_addr = player_addr(gs_i)
        pc_idx = gs_i
        GOSUB player_color
        #df_src = #tmp_addr + PL_NAME : df_pos = screenpos(0, gs_i + 2) : df_len
= 9 : #df_color = #pc_color
        GOSUB draw_field
        #gs_c = 46 ' '.'
        lb_ready = score_at(#tmp_addr, 0)
        IF lb_ready = SCORE_READY THEN #gs_c = 42 ' '*'
        #BACKTAB(screenpos(10, gs_i + 2)) = (#gs_c - 32) * 8 + COL_TEXT
    NEXT gs_i

    #df_src = FN_RX + GAME_PROMPT : df_pos = STATUS_ROW : df_len = ROWCELLS :
#df_color = COL_TEXT
    GOSUB draw_field

    IF CONT1.BUTTON THEN
        GOSUB sound_select
        url_path = 2 : GOSUB compose_url
        #net_readlen = GAME_MAXLEN
        GOSUB api_call
        poll_wait = 15
    END IF
END

' ============================================================================
' render_gameover: standings (name + final total, same running-total math
' that's live during play, so it doesn't depend on the server's own
' scores[15] convention) plus the prompt naming the winner. Held on screen
' (poll_wait=60) until the server resets the table back to the lobby.
' ============================================================================
```

```
601 render_gameover: PROCEDURE
602     GOSUB sound_gamedone
603     prl_top = 1
604     GOSUB render_player_list
605     #df_src = FN_RX + GAME_PROMPT : df_pos = STATUS_ROW : df_len = ROWCELLS :
#df_color = COL_HILITE
606     GOSUB draw_field
607 END
608
609 ' ============================================================================
610 ' render_header: viewed player's name and the round counter.
611 ' ============================================================================
612 render_header: PROCEDURE
613     #tmp_addr = player_addr(view_idx)
614     pc_idx = view_idx
615     GOSUB player_color
616     #df_src = #tmp_addr + PL_NAME : df_pos = screenpos(0, 0) : df_len = 9 :
#df_color = #pc_color
617     GOSUB draw_field
618     PRINT AT screenpos(10, 0) COLOR COL_ROUND, "R"
619     PRINT AT screenpos(12, 0) COLOR COL_ROUND, <.2>#cur_round
620     PRINT AT screenpos(14, 0) COLOR COL_ROUND, "/13"
621     PRINT AT screenpos(18, 0) COLOR COL_TEXT, "<>"
622 END
623
624 ' ============================================================================
625 ' render_card_for_view: point card.bas's render_card at the currently
626 ' viewed player, enabling the green "potential score" hint only on your own
627 ' card during your own turn.
628 ' ============================================================================
629 render_card_for_view: PROCEDURE
630     #rc_addr = player_addr(view_idx)
631     rc_own = 0
632     ' Nested single-condition IFs, not a chained "A = 0 AND B = 0 AND
633     ' C = 0" -- see the comment at game_loop's turn check for why that
634     ' shape miscompiles to always-false on this compiler.
635     IF view_idx = 0 THEN
636         IF active_player = 0 THEN
637             IF state_viewing = 0 THEN rc_own = 1
638         END IF
639     END IF
640     rc_mode = ui_mode
641     IF crs_idx < 6 THEN rc_selcat = crs_idx ELSE rc_selcat = crs_idx + 2
642     rc_reveal = 255
643     GOSUB render_card
644 END
645
646 ' ============================================================================
647 ' seed_score_shadow: snapshot the current active player's scores[0..14]
648 ' into SC_PREVSCORES. Called every poll for whoever's active right now, so
649 ' that whenever *their* turn eventually ends, check_score_reveal has a
650 ' valid "before" state to diff against.
651 ' ============================================================================
652 DIM #rv_addr, rv_i, #rv_new, #rv_old, rv_found, rv_cat, rv_frame
653 seed_score_shadow: PROCEDURE
654     #rv_addr = player_addr(active_player)
655     FOR rv_i = 0 TO 14
656         #rv_new = score_at(#rv_addr, rv_i)
657         POKE (SC_PREVSCORES + rv_i * 2), #rv_new AND 255
658         POKE (SC_PREVSCORES + rv_i * 2 + 1), (#rv_new / 256) AND 255
659     NEXT rv_i
660 END
661
662 ' ============================================================================
663 ' check_score_reveal: called right when turn_changed detects someone other
664 ' than you just finished their turn. Diffs their scores[] against the
665 ' shadow captured the last time seed_score_shadow saw them become active,
666 ' to find which single category they just filled in (skipping indices 6/7,
667 ' the computed upper-total/bonus rows -- a player never directly picks
668 ' those, they only change as a side effect of an upper-section pick, which
669 ' the 0-5 scan already catches first). Shows their card with that cell
670 ' highlighted for about 3/4 of a second before the normal turn-changed
671 ' flow (which reassigns view_idx to whoever's active now) takes over. A
672 ' no-op if nothing actually changed -- e.g. the departing "player" is a
673 ' sentinel (-1, nobody), or the shadow was never seeded for them.
674 ' ============================================================================
675 check_score_reveal: PROCEDURE
676     #rv_addr = player_addr(prev_active)
677     rv_found = 0
678     FOR rv_i = 0 TO 14
679         IF rv_i <> 6 THEN
680             IF rv_i <> 7 THEN
681                 #rv_new = score_at(#rv_addr, rv_i)
682                 #rv_old = (PEEK(SC_PREVSCORES + rv_i * 2 + 1) AND 255) * 256 +
(PEEK(SC_PREVSCORES + rv_i * 2) AND 255)
683                 IF rv_found = 0 THEN
684                     IF #rv_new <> #rv_old THEN
685                         IF #rv_new <> SCORE_UNSET THEN
686                             IF #rv_new <> SCORE_VIEWING THEN
687                                 rv_found = 1 : rv_cat = rv_i
688                             END IF
689                         END IF
690                     END IF
691                 END IF
692             END IF
693         END IF
694     NEXT rv_i
695
696     IF rv_found THEN
697         pc_idx = prev_active
698         GOSUB player_color
699         #df_src = #rv_addr + PL_NAME : df_pos = screenpos(0, 0) : df_len = 9 :
#df_color = #pc_color
700         GOSUB draw_field
701         #rc_addr = #rv_addr
```

```
        rc_own = 0 : rc_mode = 0 : rc_selcat = 0 : rc_reveal = rv_cat
        GOSUB render_card
        GOSUB sound_score
        FOR rv_frame = 1 TO 45
            WAIT
        NEXT rv_frame
    END IF
END

' ================================================================================
' render_playscreen: full redraw of the round 1-13 screen from whatever is
' currently sitting in FN_RX -- header, card, seat strip, dice, prompt.
' Split with WAITs between regions (render_card already breaks its own two
' columns) so one poll's redraw can't tear a frame; matches board.bas's
' per-quadrant WAIT and 5card.bas's per-seat WAIT, for the same reason
' (STIC has no page-flip; a big unbroken BACKTAB write can spill past
' vblank into active display).
' ================================================================================
render_playscreen: PROCEDURE
    GOSUB render_header
    GOSUB render_card_for_view
    WAIT
    GOSUB render_seatstrip
    dr_mode = ui_mode : dr_cursor = dice_cur
    GOSUB draw_dice_row
    #df_src = FN_RX + GAME_PROMPT : df_pos = STATUS_ROW : df_len = ROWCELLS :
#df_color = COL_TEXT
    GOSUB draw_field
END

' ================================================================================
' pick_best_score: point crs_idx at whichever open, selectable category
' scores the most with the current (final) roll. Used when rollsLeft hits
' 0 and SCORE mode is entered automatically -- matches the reference C
' clients defaulting cursorPos onto the highest score once no rolls
' remain (gamelogic.c). Categories 0-5 map to crs_idx 0-5 directly;
' 8-14 map to crs_idx 6-12 (see render_card's/turn_input's own comments
' for that mapping). Falls back to crs_idx=0 if nothing is open at all
' (shouldn't happen mid-round, but leaves a sane cursor position either
' way).
' ================================================================================
DIM pbs_found, #pbs_best, pbs_i, #pbs_v
pick_best_score: PROCEDURE (2)

    pbs_found = 0
    #pbs_best = 0
    crs_idx = 0
    FOR pbs_i = 0 TO 5
        #pbs_v = valid_at(pbs_i)
        IF #pbs_v <> 255 THEN
            IF pbs_found = 0 THEN
                pbs_found = 1 : #pbs_best = #pbs_v : crs_idx = pbs_i
            ELSEIF #pbs_v > #pbs_best THEN
                #pbs_best = #pbs_v : crs_idx = pbs_i
            END IF
        END IF
    NEXT pbs_i
    FOR pbs_i = 8 TO 14
        #pbs_v = valid_at(pbs_i)
        IF #pbs_v <> 255 THEN
            IF pbs_found = 0 THEN
                pbs_found = 1 : #pbs_best = #pbs_v : crs_idx = pbs_i - 2
            ELSEIF #pbs_v > #pbs_best THEN
                #pbs_best = #pbs_v : crs_idx = pbs_i - 2
            END IF
        END IF
    NEXT pbs_i
END

' ================================================================================
' turn_input: your turn. Blocks in its own WAIT loop (like Battleship's
' targeting) handling DICE mode (pick a die, toggle its reroll flag, submit
' a roll) and SCORE mode (pick a category, score it), switched with the
' bottom-left button. Sets has_action + score_idx/SC_KEEP and returns to
' let the outer poll loop perform the actual network call, exactly like
' Battleship's has_targeted/atk_pos handoff.
' ================================================================================
turn_input: PROCEDURE (1)

    inp_lock = 0
    has_action = 0
    #ti_timeleft = (PEEK(FN_RX + GAME_MOVETIME) AND 255)
    ti_framecount = 0
    ' moveTime can run up to 250 (state.bas comment), so the field needs 3
    ' digits -- see battleship.bas's targeting for why <.3> (space-padded,
    ' PRNUM16.b) is used instead of <3> (zero-padded, PRNUM16.z): .z never
    ' initializes the register that picks blank-vs-zero padding, so a
    ' shorter value can print garbage in the unused leading digit(s).
    PRINT AT STATUS_ROW + 17 COLOR COL_TEXT, <.3>#ti_timeleft

ti_loop:
    WAIT

    ti_framecount = ti_framecount + 1
    IF ti_framecount >= 60 THEN
        ti_framecount = 0
        IF #ti_timeleft > 0 THEN #ti_timeleft = #ti_timeleft - 1
        PRINT AT STATUS_ROW + 17 COLOR COL_TEXT, <.3>#ti_timeleft
        GOSUB sound_tick
        ' The server enforces its own move time limit and force-scores a
        ' player who runs out (gameLogic.go's forceHumanMove) -- without
        ' this bail, a player who never acts would stay stuck in this loop
        ' forever and never poll again to discover the turn moved on.
        IF #ti_timeleft = 0 THEN RETURN
    END IF

```

```
    IF inp_lock > 0 THEN inp_lock = inp_lock - 1 : GOTO ti_loop

    IF CONT1.KEY = 10 THEN
        GOSUB ingame_menu
        IF want_leave THEN RETURN
        GOTO ti_loop
    END IF
    IF CONT1.KEY = 11 THEN
        GOSUB show_standings
        GOSUB render_card_for_view
        GOTO ti_loop
    END IF

    IF ui_mode = 0 THEN
        ' DICE mode. B1/B2/UP are checked *before* BUTTON deliberately:
        ' the Intellivision controller's three action-button signals share
        ' overlapping bits (constants.bas's BUTTON_MASK = BUTTON_1 OR
        ' BUTTON_2 OR BUTTON_3), so CONT1.BUTTON reads non-zero for *any*
        ' of the three side buttons, not just the top one. Checking it
        ' first meant every button press -- including B1 and B2 -- fell
        ' into the "toggle hold" branch and B1/B2 were never reached.
        ' Battleship's handle_placement uses this same before-BUTTON
        ' ordering for CONT1.B1 for the identical reason.
        ' dice_cur ranges 0-5: 0 is the on-screen ROLL tile (drawn at
        ' col 0 by draw_dice_row), 1-5 are dice 0-4 -- matching the
        ' reference clients' own cursorPos convention (0=roll, 1-5=dice),
        ' rather than a dedicated controller button for rolling.
        IF CONT1.RIGHT THEN
            dice_cur = (dice_cur + 1) % 6
            inp_lock = 8 : GOSUB sound_cursor
            dr_mode = ui_mode : dr_cursor = dice_cur : GOSUB draw_dice_row
            GOTO ti_loop
        END IF
        IF CONT1.LEFT THEN
            dice_cur = (dice_cur + 5) % 6
            inp_lock = 8 : GOSUB sound_cursor
            dr_mode = ui_mode : dr_cursor = dice_cur : GOSUB draw_dice_row
            GOTO ti_loop
        END IF
        IF CONT1.UP OR CONT1.B1 THEN
            ' Move from the dice row up into the scorecard -- your current
            ' roll's potential scores are already shown in green there
            ' (render_card_for_view/draw_cat_cell), so this is how you
            ' "bank" a roll instead of rolling again.
            ui_mode = 1
            inp_lock = 10 : GOSUB sound_select
            dr_mode = ui_mode : dr_cursor = dice_cur : GOSUB draw_dice_row
            GOSUB render_card_for_view
            GOTO ti_loop
        END IF
        IF CONT1.BUTTON THEN
            IF dice_cur = 0 THEN
                ' On the ROLL tile -- submit. Rejected (sound_invalid,
                ' stay put) if there are no rolls left, or if every die is
                ' marked KEEP (mask='0') so nothing would actually
                ' reroll. The untouched default (every die still
                ' '1'=reroll) is itself a valid "reroll all 5" submission.
                IF state_rollsleft = 0 THEN
                    GOSUB sound_invalid
                    GOTO ti_loop
                END IF
                ti_allkept = 1
                FOR pt_i = 0 TO 4
                    IF (PEEK(SC_KEEP + pt_i) AND 255) = 49 THEN ti_allkept = 0
                NEXT pt_i
                IF ti_allkept THEN
                    GOSUB sound_invalid
                    GOTO ti_loop
                END IF
                GOSUB sound_roll
                has_action = 2
                RETURN
            ELSE
                ' On a die -- toggle it between its default (reroll, '1')
                ' and kept ('0').
                pt_i = dice_cur - 1
                IF (PEEK(SC_KEEP + pt_i) AND 255) = 49 THEN
                    POKE (SC_KEEP + pt_i), 48
                ELSE
                    POKE (SC_KEEP + pt_i), 49
                END IF
                GOSUB sound_hold
                dr_mode = ui_mode : dr_cursor = dice_cur : GOSUB draw_dice_row
                inp_lock = 8
                GOTO ti_loop
            END IF
        END IF
    ELSE
        ' SCORE mode.
        IF CONT1.DOWN THEN
            ' Pressing down from the bottom row of either column drops
            ' back to DICE mode instead of wrapping to the top -- unless
            ' there are no rolls left, in which case DICE mode has
            ' nothing to do (matches the B1 guard just above/below).
            at_bottom_row = 0
            IF crs_idx = 5 THEN at_bottom_row = 1
            IF crs_idx = 12 THEN at_bottom_row = 1
            IF at_bottom_row THEN
                IF state_rollsleft <> 0 THEN
                    ui_mode = 0
                    inp_lock = 10 : GOSUB sound_select
                    dr_mode = ui_mode : dr_cursor = dice_cur : GOSUB
draw_dice_row
                    GOSUB render_card_for_view
                    GOTO ti_loop
                END IF
```

```
            END IF
            IF crs_idx < 6 THEN crs_idx = (crs_idx + 1) % 6 ELSE crs_idx = 6 +
((crs_idx - 6 + 1) % 7)
            inp_lock = 8 : GOSUB sound_cursor : GOSUB render_card_for_view
            GOTO ti_loop
        END IF
        IF CONT1.UP THEN
            IF crs_idx < 6 THEN crs_idx = (crs_idx + 5) % 6 ELSE crs_idx = 6 +
((crs_idx - 6 + 6) % 7)
            inp_lock = 8 : GOSUB sound_cursor : GOSUB render_card_for_view
            GOTO ti_loop
        END IF
        IF CONT1.RIGHT AND crs_idx < 6 THEN
            pt_i = crs_idx : IF pt_i > 6 THEN pt_i = 6
            crs_idx = 6 + pt_i
            inp_lock = 8 : GOSUB sound_cursor : GOSUB render_card_for_view
            GOTO ti_loop
        END IF
        IF CONT1.LEFT AND crs_idx >= 6 THEN
            pt_i = crs_idx - 6 : IF pt_i > 5 THEN pt_i = 5
            crs_idx = pt_i
            inp_lock = 8 : GOSUB sound_cursor : GOSUB render_card_for_view
            GOTO ti_loop
        END IF
        IF CONT1.B1 THEN
            ' No rolls left means DICE mode has nothing to do (B2 there
            ' would just reject every attempt) -- stay in SCORE mode
            ' instead of bouncing back to a dead end.
            IF state_rollsleft = 0 THEN
                GOSUB sound_invalid
                GOTO ti_loop
            END IF
            ui_mode = 0
            inp_lock = 10 : GOSUB sound_select
            dr_mode = ui_mode : dr_cursor = dice_cur : GOSUB draw_dice_row
            GOSUB render_card_for_view
            GOTO ti_loop
        END IF
        IF CONT1.BUTTON THEN
            IF crs_idx < 6 THEN score_idx = crs_idx ELSE score_idx = crs_idx + 2
3


            IF valid_at(score_idx) = 255 THEN
                GOSUB sound_invalid
                GOTO ti_loop
            END IF
            score_is_fujitzee = 0
            IF score_idx = SCORE_FUJITZEE THEN
                IF valid_at(score_idx) > 0 THEN score_is_fujitzee = 1
            END IF
            IF score_is_fujitzee THEN
                GOSUB sound_fujitzee
            ELSE
                GOSUB sound_score
            END IF
            has_action = 3
            RETURN
        END IF
    END IF
    GOTO ti_loop
END

' The compiled program overflows IntyBASIC's default $5000-$6FFF
' (8192-word) window -- the compiler happily keeps allocating past $6FFF,
' but $7000 is not in the manual's list of ranges "usable without
' additional programming" on modern flash cart/homebrew PCBs (that list
' jumps from $6FFF to $A000). This ORG was originally placed just before
' halt:, at the very end of the file -- which put it *after*
' name_entry_screen, so name_entry_screen's own tail end (its "NAME NOT
' SAVED" error path) still spilled about 20 words into that unsafe $7000
' gap (confirmed in the .lst: code was landing at $7000-$7013). Moving the
' ORG here, ahead of both ingame_menu and name_entry_screen, pushes that
' whole "cold" tail (menu, name entry) into $D000+ instead, closing the
' gap entirely. This matters beyond spec-purity: FujiNet GO's boot loader
' hung on mount with the previous placement, consistent with $7000 not
' being safely mapped as ROM on at least one real target this ships to,
' even though jzIntv's emulation is permissive enough not to visibly
' choke on it. Relocating everything from here to halt: into $D000-$FFFF
' keeps the whole ROM inside the address ranges the manual documents as
' safe. Cross-segment GOSUB/GOTO compiles to ordinary absolute CP-1610
' jumps/calls, so this is a bookkeeping split, not a correctness one.
    ASM ORG $D000

' ------------------------------------------------------------------------
' split_room_url / write_room_appkey / clear_room_appkey: lobby room-
' appkey handling for the intv<->lobby handoff. Placed here (past the
' ORG $D000 split above) rather than back up near compose_url where they
' were first written -- they're only ever called from boot, table-select,
' and ingame_menu, none of which are the hot per-frame game_loop path, so
' like ingame_menu/name_entry_screen below they belong in the "cold" tail.
' Adding them before the split pushed the $5000-$6FFF segment over its
' 8192-word budget and spilled ~200 words into the unsafe $7000-$7FFF
' gap, which is exactly the "hung on mount" failure mode described above
' -- moving them here is the fix. Cross-segment GOSUB/GOTO from boot_start
' and table_select back up before the split compiles to ordinary absolute
' jumps, so this is bookkeeping only, not a correctness change.
' ------------------------------------------------------------------------
' split_room_url: given a lobby-supplied "https://host/?table=xyz" value
' already sitting in SC_ENDPT with its length in fn_len (as left by
' appkey_read), split it into the bare endpoint (SC_ENDPT, truncated at
' the '?') and the table id (SC_TABLE). Mirrors the C clients'
' welcomeActionVerifyServerDetails() '?' scan. Leaves SC_TABLE empty (a
' leading NUL) if there's no '?', the query isn't "table=...", or the id
' contains anything other than A-Z/a-z/0-9 -- it goes straight into a
' rebuilt query string unescaped, same reasoning as the username check
' above.
```

```
split_room_url: PROCEDURE
    POKE SC_TABLE, 0

    sp_found = 255 ' sentinel: not found
    sp_i = 0
    WHILE sp_i < fn_len
        IF (PEEK(SC_ENDPT + sp_i) AND 255) = 63 THEN ' '?'
            sp_found = sp_i
            EXIT WHILE
        END IF
        sp_i = sp_i + 1
    WEND
    IF sp_found = 255 THEN RETURN

    ' Truncate the endpoint at the '?' regardless of what follows.
    POKE (SC_ENDPT + sp_found), 0

    ' Require "table=" (6 bytes) right after the '?'.
    sp_ok = 0
    IF sp_found + 7 <= fn_len THEN
        sp_ok = 1
        IF (PEEK(SC_ENDPT + sp_found + 1) AND 255) <> 116 THEN sp_ok = 0 ' t
        IF (PEEK(SC_ENDPT + sp_found + 2) AND 255) <> 97  THEN sp_ok = 0 ' a
        IF (PEEK(SC_ENDPT + sp_found + 3) AND 255) <> 98  THEN sp_ok = 0 ' b
        IF (PEEK(SC_ENDPT + sp_found + 4) AND 255) <> 108 THEN sp_ok = 0 ' l
        IF (PEEK(SC_ENDPT + sp_found + 5) AND 255) <> 101 THEN sp_ok = 0 ' e
        IF (PEEK(SC_ENDPT + sp_found + 6) AND 255) <> 61  THEN sp_ok = 0 ' =
    END IF
    IF sp_ok = 0 THEN RETURN

    ' Copy the id: up to 8 bytes, stopping at NUL or '&'.
    sp_j = sp_found + 7
    sp_k = 0
    WHILE sp_k < 8 AND sp_j < fn_len
        sp_c = PEEK(SC_ENDPT + sp_j) AND 255
        IF sp_c = 0 OR sp_c = 38 THEN EXIT WHILE ' NUL or '&'
        POKE (SC_TABLE + sp_k), sp_c
        sp_k = sp_k + 1
        sp_j = sp_j + 1
    WEND
    POKE (SC_TABLE + sp_k), 0

    ' Validate: A-Z/a-z/0-9 only.
    sp_valid = 1
    FOR sp_m = 0 TO sp_k - 1
        sp_c = PEEK(SC_TABLE + sp_m) AND 255
        IF (sp_c < 65 OR sp_c > 90) AND (sp_c < 97 OR sp_c > 122) AND (sp_c < 48
OR sp_c > 57) THEN sp_valid = 0
    NEXT sp_m
    IF sp_valid = 0 THEN POKE SC_TABLE, 0
END

' write_room_appkey: persist SC_ENDPT + "?table=" + SC_TABLE back to the
' lobby server appkey slot, so a reboot without going back through the
' lobby rejoins the same room (mirrors the C clients' "Update server app
' key in case of reboot"). Builds the payload directly in FN_TX and writes
' it from there -- appkey_write's byte-by-byte copy from #fn_src into
' FN_TX is a safe no-op when #fn_src is FN_TX itself.
write_room_appkey: PROCEDURE
    #fn_txlen = 0
    #fn_src = SC_ENDPT : ls_max = 65 : GOSUB fn_strlen : GOSUB fn_putstr
    #fn_src = VARPTR lit_qtable(0) : fn_len = 7 : GOSUB fn_putstr
    #fn_src = SC_TABLE : ls_max = 9 : GOSUB fn_strlen : GOSUB fn_putstr

    ak_creator_lo = 1 : ak_creator_hi = 0 : ak_app = 1
    ak_key = AK_LOBBY_KEY_SERVER : ak_mode = 1
    GOSUB appkey_open
    IF fn_ok THEN
        fn_len = #fn_txlen : #fn_src = FN_TX
        GOSUB appkey_write
        GOSUB appkey_close
    END IF
END

' clear_room_appkey: blank the lobby server appkey slot on a deliberate
' leave, so a later reboot lands on table select instead of silently
' rejoining a table the player just quit.
clear_room_appkey: PROCEDURE
    ak_creator_lo = 1 : ak_creator_hi = 0 : ak_app = 1
    ak_key = AK_LOBBY_KEY_SERVER : ak_mode = 1
    GOSUB appkey_open
    IF fn_ok THEN
        fn_len = 0 : #fn_src = FN_TX
        GOSUB appkey_write
        GOSUB appkey_close
    END IF
    POKE SC_TABLE, 0
END

' ============================================================================
' ingame_menu: keypad CLEAR overlay, drawn over the prompt row and the two
' rows above it. Disc up/down to choose, action button to confirm, Clear
' again cancels straight back to RESUME.
' ============================================================================
ingame_menu: PROCEDURE
    im_sel = 0
    inp_lock = 0
    ' Wait for the CLEAR press that opened the menu to release before
    ' accepting any input. Without this, the same still-held key
    ' immediately satisfies "IF CONT1.KEY = 10 THEN GOTO im_done" below on
    ' the very first frame, closing the menu the instant it opens -- the
    ' manual's own advice ("it's suggested to wait for CONT1.KEY to
    ' contain 12 before waiting for a key") for exactly this reason.
im_wait_release:
    WAIT
    IF CONT1.KEY <> 12 THEN GOTO im_wait_release
```

```

im_loop:
    PRINT AT STATUS_ROW - 40 COLOR COL_TEXT, "                    "
    PRINT AT STATUS_ROW - 20 COLOR COL_TEXT, "                    "
    PRINT AT STATUS_ROW COLOR COL_TEXT, "                      "
    PRINT AT STATUS_ROW - 40 COLOR COL_TEXT, "TABLE MENU"
    #gs_c = COL_TEXT
    IF im_sel = 0 THEN #gs_c = COL_HILITE
    PRINT AT STATUS_ROW - 20 COLOR #gs_c, "RESUME"
    #gs_c = COL_TEXT
    IF im_sel = 1 THEN #gs_c = COL_HILITE
    PRINT AT STATUS_ROW COLOR #gs_c, "QUIT TABLE"

    WAIT
    IF inp_lock > 0 THEN inp_lock = inp_lock - 1 : GOTO im_loop

    IF CONT1.UP OR CONT1.DOWN THEN
        im_sel = 1 - im_sel
        inp_lock = 10
        GOSUB sound_cursor
        GOTO im_loop
    END IF
    IF CONT1.KEY = 10 THEN GOTO im_done
    IF CONT1.BUTTON = 0 THEN GOTO im_loop
    GOSUB sound_select

    IF im_sel = 1 THEN
        url_path = 5 : GOSUB compose_url
        #net_readlen = 8
        GOSUB api_call
        GOSUB clear_room_appkey
        want_leave = 1
    END IF
im_done:
    prev_round = 255 ' safety net, in case some other path still relies on it
    ' Redraw the whole screen immediately from whatever's already cached
    ' in FN_RX -- not just next poll. ingame_menu is called both from
    ' gl_input (outer loop) and from turn_input's own blocking ti_loop;
    ' in the latter case, control doesn't return to game_loop's own
    ' CLS/redraw dispatch until the current turn ends, which could be a
    ' long time away, so the menu's overlay (rows 9-11) would otherwise
    ' just sit there behind whatever turn_input draws around it next.
    IF want_leave = 0 THEN
        GOSUB cls_blue
        IF #cur_round = ROUND_LOBBY THEN
            GOSUB render_lobby
        ELSEIF #cur_round = ROUND_GAMEOVER THEN
            GOSUB render_gameover
        ELSE
            GOSUB card_init
            GOSUB render_playscreen
        END IF
    END IF
END

' ============================================================================
' name_entry_screen: disc letter picker, max 8 chars. Disc up/down cycles
' the character under the cursor through A-Z, 0-9, space; left/right moves
' the cursor; the action button accepts (at least 1 non-space char).
' Adapted from fujinet-battleship/intv/battleship.bas, verbatim apart from
' colors.
' ============================================================================
name_entry_screen: PROCEDURE
    GOSUB cls_blue
    PRINT AT 0 COLOR COL_TEXT, "ENTER YOUR NAME"
    FOR ne_i = 0 TO 7
        ne_buf(ne_i) = 36 ' space
    NEXT ne_i
    ne_cur = 0
    inp_lock = 0

ne_loop:
    FOR ne_i = 0 TO 7
        #gs_c = COL_TEXT
        IF ne_i = ne_cur THEN #gs_c = COL_HILITE
        ne_j = ne_buf(ne_i)
        IF ne_j < 26 THEN
            gs_j = 65 + ne_j
        ELSEIF ne_j < 36 THEN
            gs_j = 48 + ne_j - 26
        ELSE
            gs_j = 95 ' underscore stands in for a visible blank
        END IF
        #BACKTAB(60 + 6 + ne_i) = (gs_j - 32) * 8 + #gs_c
    NEXT ne_i

    WAIT
    IF inp_lock > 0 THEN inp_lock = inp_lock - 1 : GOTO ne_loop

    IF CONT1.RIGHT THEN
        ne_cur = ne_cur + 1
        IF ne_cur > 7 THEN ne_cur = 0
        inp_lock = 8
        GOSUB sound_cursor
        GOTO ne_loop
    END IF
    IF CONT1.LEFT THEN
        IF ne_cur = 0 THEN ne_cur = 8
        ne_cur = ne_cur - 1
        inp_lock = 8
        GOSUB sound_cursor
        GOTO ne_loop
    END IF
    IF CONT1.UP THEN
        ne_buf(ne_cur) = ne_buf(ne_cur) + 1
        IF ne_buf(ne_cur) > 36 THEN ne_buf(ne_cur) = 0
```

```
        inp_lock = 6
        GOSUB sound_cursor
        GOTO ne_loop
    END IF
    IF CONT1.DOWN THEN
        IF ne_buf(ne_cur) = 0 THEN ne_buf(ne_cur) = 37
        ne_buf(ne_cur) = ne_buf(ne_cur) - 1
        inp_lock = 6
        GOSUB sound_cursor
        GOTO ne_loop
    END IF
    IF CONT1.BUTTON = 0 THEN GOTO ne_loop
    GOSUB sound_select

    ne_len = 8
    WHILE ne_len > 0 AND ne_buf(ne_len - 1) = 36
        ne_len = ne_len - 1
    WEND
    IF ne_len = 0 THEN GOTO ne_loop

    FOR ne_i = 0 TO ne_len - 1
        ne_j = ne_buf(ne_i)
        IF ne_j < 26 THEN
            gs_j = 65 + ne_j
        ELSE
            gs_j = 48 + ne_j - 26
        END IF
        POKE (SC_NAME + ne_i), gs_j
    NEXT ne_i
    POKE (SC_NAME + ne_len), 0

    ak_creator_lo = 1 : ak_creator_hi = 0 : ak_app = 1 : ak_key = 0 : ak_mode =
1
    GOSUB appkey_open
    IF fn_ok THEN
        #fn_src = SC_NAME : fn_len = ne_len
        GOSUB appkey_write
        IF fn_ok = 0 THEN
            PRINT AT 100 COLOR COL_TEXT, "NAME NOT SAVED FOR "
            PRINT AT 120 COLOR COL_TEXT, "NEXT TIME           "
            inp_lock = 90
            WHILE inp_lock > 0
                inp_lock = inp_lock - 1
                WAIT
            WEND
        END IF
        GOSUB appkey_close
    END IF
END

halt:
    WAIT
    GOTO halt
```

LISTING 11 state.bas — Fujitzee wire format

```
' state.bas -- binary GameState/Tables wire format, read in place out of FN_RX.
'
' Per the pattern established by fujinet-5cardstud/intv/state.bas and
' fujinet-battleship/intv/state.bas: never copy the reply into IntyBASIC
' variables -- these are just fixed offsets (matching server/util.go's
' serializeResults binary layout, requested with &bin=1) plus small DEF FN
' accessors that PEEK straight out of FN_RX.
'
' GameState layout, always this shape regardless of round (util.go:76-108):
'   0   playerCount (1)
'   1   serverName[21]        lowercased, NUL-padded
'  22   prompt[41]             lowercased, NUL-padded
'  63   round (1)              0=lobby, 1..13=play, 99=gameover
'  64   rollsLeft (1)
'  65   activePlayer (signed, $FF = -1; 0 = your turn)
'  66   moveTime (1, seconds, <=255 by design)
'  67   viewing (1)            1 = you are a spectator
'  68   dice[6]                5 ascii digits '1'-'6' + NUL; empty at turn start
'  74   keepRoll[6]            5 ascii '0'/'1' + NUL (1 = will re-roll)
'  80   validScores[15]        signed bytes; $FF = -1 = not selectable.
'                              Only meaningful when round is 1..13; the
'                              server sends all zero bytes here in the
'                              lobby (nil slice, not -1 -- see util.go's
'                              ValidScores loop), so never read this
'                              outside GAME_MINLEN's play branch.
'  95   players[N] -- N = playerCount (can exceed 6; spectators are
'       appended beyond the 6 seated players), 42 bytes each:
'         name[9] alias(1) scores[16] (u16 LE each; $FFFF = -1 = unset)
'       scores[] is meaningful only at index 0 (1=ready,0=not,-2=spectator)
'       while round=0; all 16 are real once round>=1.
'
' /tables -> Tables struct:
'   0   count (1)
'   then N x 36 bytes: table[9] name[21] players[6] (literal "cur / max")

    CONST GAME_PLAYERCOUNT  = 0
    CONST GAME_SERVERNAME   = 1     ' 21 bytes
    CONST GAME_PROMPT       = 22    ' 41 bytes
    CONST GAME_ROUND        = 63
    CONST GAME_ROLLSLEFT    = 64
    CONST GAME_ACTIVEPLAYER = 65
    CONST GAME_MOVETIME     = 66
    CONST GAME_VIEWING      = 67
    CONST GAME_DICE         = 68    ' 6 bytes
    CONST GAME_KEEPROLL     = 74    ' 6 bytes
    CONST GAME_VALIDSCORES  = 80    ' 15 bytes
    CONST GAME_PLAYERS      = 95

' Scratch RAM, ours, below fujinet.bas's own SC_* buffers (which stop at
' SC_QUERY $9150 + 48 = $9180) and well below the $9C00 mailbox. Declared
' here (rather than in fujitzee.bas) so dice.bas/card.bas -- included right
' after this file, before fujitzee.bas's own CONST section runs -- can
' reference SC_KEEP without a forward reference.
    CONST SC_TABLE      = $9180 ' selected table id, 9 bytes (8 + NUL)
    CONST SC_KEEP       = $9190 ' 5-char reroll mask, ascii '0'/'1', no NUL
needed
    CONST SC_PREVSCORES = $91A0 ' shadow of the active player's scores[16] (u16
LE), 32 bytes

' Cross-file client display globals. IntyBASIC auto-registers a bare
' identifier the first time it's mentioned at all (read or write), not
' just on an explicit DIM -- card.bas (included right after this file)
' reads/writes view_idx and draw_field's df_* parameters before
' fujitzee.bas's own DIM section would otherwise declare them, which
' turns that later DIM into a "variable already defined" error. Declaring
' them here, ahead of every include that touches them, avoids that.
    DIM view_idx
    DIM df_pos, df_len, #df_color, #df_src

    CONST PLAYER_STRIDE = 42
    CONST PL_NAME   = 0             ' 9 bytes
    CONST PL_ALIAS  = 9             ' 1 byte
    CONST PL_SCORES = 10            ' 16 x uint16 LE

    CONST TABLE_STRIDE = 36
    CONST TBL_ID       = 0          ' 9 bytes
    CONST TBL_NAME     = 9          ' 21 bytes
    CONST TBL_PLAYERS = 30          ' 6 bytes, literal "cur / max"

    ' PLAYER_MAX matches the reference C client's misc.h (spectators can
    ' push playerCount past the 6 seated-player cap); GAME_MAXLEN sizes the
    ' one poll buffer big enough for the worst case, 95 + 12*42 = 599 bytes,
    ' well inside FN_RX's 704-byte window ($9D40-$9FFF).
    CONST PLAYER_MAX     = 12
    CONST GAME_MAXLEN    = GAME_PLAYERS + PLAYER_MAX * PLAYER_STRIDE
    CONST GAME_MINLEN    = GAME_PLAYERS
    CONST TABLES_MAXLEN  = 361

    ' Round values.
    CONST ROUND_LOBBY    = 0
    CONST ROUND_FINAL    = 13
    CONST ROUND_GAMEOVER = 99

    ' Scorecard indices (gameLogic.go SCORE_* constants).
    CONST SCORE_ONES        = 0
    CONST SCORE_UPPER_TOTAL = 6
    CONST SCORE_UPPER_BONUS = 7
    CONST SCORE_SET3        = 8
    CONST SCORE_SET4        = 9
    CONST SCORE_FULLHOUSE   = 10
```

```
    CONST SCORE_SRUN        = 11
    CONST SCORE_LRUN        = 12
    CONST SCORE_CHANCE      = 13
    CONST SCORE_FUJITZEE    = 14
    CONST SCORE_TOTAL       = 15

    ' Lobby-only meaning of scores[0].
    CONST SCORE_VIEWING = 65534     ' wire $FFFE -> u16 65534 (== -2 signed)
    CONST SCORE_UNSET   = 65535     ' wire $FFFF -> -1 signed
    CONST SCORE_READY   = 1
    CONST SCORE_UNREADY = 0

' player_addr(i): address of player i's 42-byte record in FN_RX.
    DEF FN player_addr(i) = FN_RX + GAME_PLAYERS + i * PLAYER_STRIDE

' table_addr(i): address of table i's 36-byte record in FN_RX (i is 0-based,
' the record right after the leading count byte).
    DEF FN table_addr(i) = FN_RX + 1 + i * TABLE_STRIDE

' score_at(addr, i): player record at `addr`'s scores[i] as an unsigned
' 16-bit word (65535 = unset/-1, 65534 = viewing/-2 in the lobby slot).
    DEF FN score_at(addr, i) = (PEEK(addr + PL_SCORES + i * 2 + 1) AND 255) *
256 + (PEEK(addr + PL_SCORES + i * 2) AND 255)

' valid_at(i): validScores[i] as an unsigned byte (255 = -1 = not selectable).
' Parenthesized "(... AND 255)" deliberately -- see state_round's comment
' below for why an unparenthesized trailing "AND 255" is dangerous.
    DEF FN valid_at(i) = (PEEK(FN_RX + GAME_VALIDSCORES + i) AND 255)

' active_player: activePlayer as a signed value (-1..11), converting the
' wire's $FF sentinel. Called as plain `active_player` (no parens -- DEF FN
' with no arguments is defined and invoked without them).
    DEF FN active_player = ((PEEK(FN_RX + GAME_ACTIVEPLAYER) AND 255) = 255) *
-256 + (PEEK(FN_RX + GAME_ACTIVEPLAYER) AND 255)

' state_round / state_playercount / state_rollsleft / state_viewing:
' unsigned byte reads, broken out as DEF FNs purely so call sites read as
' intent, not offsets.
'
' Parenthesized "(... AND 255)" deliberately, not left bare. IntyBASIC's
' "=" binds *tighter* than "AND" (same trap as C's "a & b == c"), so an
' unparenthesized "PEEK(x) AND 255" substituted into "state_viewing = 0"
' compiles as "PEEK(x) AND (255 = 0)" -- the 255 gets grouped with the
' comparison instead of the AND, folding to "PEEK(x) AND 0", which is
' always zero regardless of what was actually peeked. Confirmed by reading
' the generated .lst for "IF active_player = 0 AND state_viewing = 0
' THEN": active_player's own formula (already parenthesized below) came
' out correct, but state_viewing's unparenthesized body made the whole
' condition permanently false, so turn_input was never reached at all.
    DEF FN state_round       = (PEEK(FN_RX + GAME_ROUND) AND 255)
    DEF FN state_playercount = (PEEK(FN_RX + GAME_PLAYERCOUNT) AND 255)
    DEF FN state_rollsleft   = (PEEK(FN_RX + GAME_ROLLSLEFT) AND 255)
    DEF FN state_viewing     = (PEEK(FN_RX + GAME_VIEWING) AND 255)

' ---------------------------------------------------------------------------
' validate_state: layered sanity check on a /state, /ready, /roll, or /score
' reply already sitting in FN_RX (#net_gotlen bytes). FN_RX is never cleared
' between calls, so a short read leaves stale bytes from a previous, larger
' response -- fixed-offset reads would happily render that as garbage. Sets
' fn_ok = 0 (in addition to whatever net/transact left it) if any gate
' fails; callers must check fn_ok after calling this, not just after
' api_call.
' ---------------------------------------------------------------------------
DIM vs_round, vs_pc, #vs_expect
validate_state: PROCEDURE
    IF fn_ok = 0 OR #net_gotlen < GAME_MINLEN THEN
        fn_ok = 0
        RETURN
    END IF

    vs_pc = state_playercount
    IF vs_pc > PLAYER_MAX THEN
        fn_ok = 0
        RETURN
    END IF

    vs_round = state_round
    IF vs_round <> ROUND_GAMEOVER AND vs_round > ROUND_FINAL THEN
        fn_ok = 0
        RETURN
    END IF

    #vs_expect = GAME_PLAYERS + vs_pc * PLAYER_STRIDE
    IF #net_gotlen < #vs_expect THEN fn_ok = 0
END

```

LISTING 12 dice.bas — Fujitzee dice

```
' dice.bas -- die-face GRAM bitmaps and draw routine, MODE 1 (FG/BG). Same
' technique as fujinet-5cardstud/intv/gfx.bas's cards: each face is one 8x8
' GRAM card, loaded once at boot, drawn by poking a BACKTAB cell to
' (index + 256) * 8 + color.
'
' Six faces (pip counts 1-6, GRAM index 0-5), a standard 3x3 pip grid on an
' outlined 8x8 box. A die marked to re-roll isn't a separate GRAM tile --
' it's the same face drawn with the cursor/held background color, exactly
' like card.bas's scorecard cursor and board.bas's cell coloring. That
' keeps the GRAM budget at 6 of 64 cards, leaving the rest free for a
' future sprite-based roll animation (neither 5cardstud nor Battleship use
' any MOBs at all).

' ---------------------------------------------------------------------------
' dice_init: load the 6 die faces into GRAM slots 0-5 (screen codes 256-261).
' Must run once at startup. GRAM loads take effect on the next video frame,
' so DEFINE is followed by WAIT -- a second DEFINE in the same frame would
' silently overwrite the first.
' ---------------------------------------------------------------------------
dice_init: PROCEDURE
    DEFINE 0, 6, diceart : WAIT
END

' ---------------------------------------------------------------------------
' print_die: draw one die face at BACKTAB cell dp_pos. Inputs: dp_pos (cell
' offset), dp_val (1-6; anything outside that range draws a blank cell so an
' empty dice string at turn start doesn't render garbage), #dp_color.
' ---------------------------------------------------------------------------
DIM dp_pos, dp_val, #dp_color
print_die: PROCEDURE
    IF dp_val >= 1 AND dp_val <= 6 THEN
        #BACKTAB(dp_pos) = #dp_color + (dp_val - 1 + 256) * 8
    ELSE
        #BACKTAB(dp_pos) = #dp_color + 32 * 8 ' blank (space glyph)
    END IF
END

' ---------------------------------------------------------------------------
' draw_dice_row: a ROLL tile at BACKTAB row 10 col 0, then the 5 dice at
' cols 2,4,6,8,10, plus a small rolls-left readout. Reads the live wire
' dice string (FN_RX+GAME_DICE) and the local keep mask (SC_KEEP) directly
' rather than taking them as parameters -- both are always current by the
' time this is called (either just-polled, or unrolled state.bas globals
' set up before a redraw).
'
' SC_KEEP holds the wire's roll-mask convention ('1'=49 will reroll,
' '0'=48 will be kept -- see fujitzee.bas's compose_url, which sends it
' straight through to /roll/<mask>), but the *default* per die, before the
' player touches anything, is '1' (reroll) -- pressing the button marks a
' die to be KEPT, matching how every other Fujitzee client's dice
' selection reads (you pick what you're holding onto, not what you're
' throwing away). So the highlight here is for the KEEP state (mask='0'),
' not the reroll state.
'
' Inputs: dr_mode (0=DICE,1=SCORE), dr_cursor (0-5: 0=the ROLL tile, 1-5=
' dice 0-4, matching the reference clients' own cursorPos convention --
' only drawn highlighted when dr_mode=0).
' ---------------------------------------------------------------------------
    CONST DICE_ROW = 10
    CONST ROLL_COL = 0
    CONST DICE_COL0 = 2
    CONST COL_DICE_NORMAL = FG_BLACK + BG_WHITE
    CONST COL_DICE_KEPT   = FG_BLACK + BG_YELLOW
    CONST COL_DICE_CURSOR = FG_BLACK + BG_GREEN

DIM dr_mode, dr_cursor, dr_i, #dr_ch, dr_kept, #dr_rolls
draw_dice_row: PROCEDURE
    #dp_color = COL_DICE_NORMAL
    IF dr_mode = 0 AND dr_cursor = 0 THEN #dp_color = COL_DICE_CURSOR
    PRINT AT screenpos(ROLL_COL, DICE_ROW) COLOR #dp_color, "R"

    FOR dr_i = 0 TO 4
        #dr_ch = (PEEK(FN_RX + GAME_DICE + dr_i) AND 255)
        IF #dr_ch >= 49 AND #dr_ch <= 54 THEN dp_val = #dr_ch - 48 ELSE dp_val =
0

        dr_kept = (PEEK(SC_KEEP + dr_i) AND 255) = 48
        #dp_color = COL_DICE_NORMAL
        IF dr_kept THEN #dp_color = COL_DICE_KEPT
        IF dr_mode = 0 AND dr_cursor = dr_i + 1 THEN #dp_color = COL_DICE_CURSOR

        dp_pos = screenpos(DICE_COL0 + dr_i * 2, DICE_ROW)
        GOSUB print_die
    NEXT dr_i

    ' Blank cols 11-19 first, not just the 2 cells the count actually
    ' needs -- "x" used to sit at col 17 here as a separator, and being
    ' lowercase it was outside MODE 1's GROM range (ASCII 32-95 only,
    ' uppercase/digits/symbols), rendering as a garbage glyph. Blanking
    ' the whole gap defensively means any future stray write here (or
    ' leftover from a still-undiagnosed one) can't leave visible debris
    ' either, rather than only ever touching the exact 2 cells this
    ' PRINT needs.
    FOR dr_i = 11 TO 19
        #BACKTAB(screenpos(dr_i, DICE_ROW)) = COL_DICE_NORMAL
    NEXT dr_i
    #dr_rolls = state_rollsleft
    PRINT AT screenpos(18, DICE_ROW) COLOR COL_DICE_NORMAL, <.2>#dr_rolls
END

```

```
' ------------------------------------------------------------------------
' animate_roll: dice-rolling flourish shown right after any player's roll
' lands (yours or another seat's -- see game_loop's roll_changed check,
' which fires this for anyone). FN_RX already holds the settled result at
' this point (the server computes the whole roll in one shot), so this
' doesn't determine the outcome -- it flickers whichever dice the *wire's*
' keepRoll field says were rerolled (GAME_KEEPROLL; the server echoes
' back whatever mask was actually submitted, by whoever just rolled, not
' our own local SC_KEEP -- that's what makes this work for other players'
' turns too) through random faces a few times, with a tick each flicker,
' before the normal render draws the real values. Kept dice are drawn at
' their real (unchanging) value throughout, echoing the reference C
' clients' rollFrames/ROLL_SOUND_MOD animation (gamelogic.c). Wire
' convention is unaffected by the local keep/reroll UI relabeling above:
' keepRoll's '1' still means "this die was rerolled."
' ------------------------------------------------------------------------
    CONST ROLL_ANIM_FRAMES = 24
    CONST ROLL_ANIM_STEP = 3

DIM ra_frame, ra_i, #ra_ch
animate_roll: PROCEDURE
    FOR ra_frame = 1 TO ROLL_ANIM_FRAMES
        IF ra_frame % ROLL_ANIM_STEP = 1 THEN
            FOR ra_i = 0 TO 4
                IF (PEEK(FN_RX + GAME_KEEPROLL + ra_i) AND 255) = 49 THEN
                    dp_val = RAND(6) + 1
                ELSE
                    #ra_ch = (PEEK(FN_RX + GAME_DICE + ra_i) AND 255)
                    IF #ra_ch >= 49 AND #ra_ch <= 54 THEN dp_val = #ra_ch - 48
ELSE dp_val = 0
                END IF
                dp_pos = screenpos(DICE_COL0 + ra_i * 2, DICE_ROW) : #dp_color =
COL_DICE_NORMAL
                GOSUB print_die
            NEXT ra_i
            GOSUB sound_tick
        END IF
        WAIT
    NEXT ra_frame
END

' Pip grid: border at rows/cols 0 and 7; pips at rows/cols {2,4,6} of an
' 8x8 cell. Row4 col4 is the single center pip (face 1); rows/cols 2 and 6
' are the four corners (faces 4-6); row4 cols 2/6 are the middle side pips
' (face 6 only).
diceart:
    ' Face 1 (index 0): center pip.
    BITMAP "oooooooo"
    BITMAP "o......o"
    BITMAP "o......o"
    BITMAP "o......o"
    BITMAP "o...o..o"
    BITMAP "o......o"
    BITMAP "o......o"
    BITMAP "oooooooo"

    ' Face 2 (index 1): top-left, bottom-right.
    BITMAP "oooooooo"
    BITMAP "o......o"
    BITMAP "o.o....o"
    BITMAP "o......o"
    BITMAP "o......o"
    BITMAP "o......o"
    BITMAP "o.....oo"
    BITMAP "oooooooo"

    ' Face 3 (index 2): top-left, center, bottom-right.
    BITMAP "oooooooo"
    BITMAP "o......o"
    BITMAP "o.o....o"
    BITMAP "o......o"
    BITMAP "o...o..o"
    BITMAP "o......o"
    BITMAP "o.....oo"
    BITMAP "oooooooo"

    ' Face 4 (index 3): four corners.
    BITMAP "oooooooo"
    BITMAP "o......o"
    BITMAP "o.o...oo"
    BITMAP "o......o"
    BITMAP "o......o"
    BITMAP "o.o...oo"
    BITMAP "oooooooo"

    ' Face 5 (index 4): four corners + center.
    BITMAP "oooooooo"
    BITMAP "o......o"
    BITMAP "o.o...oo"
    BITMAP "o......o"
    BITMAP "o...o..o"
    BITMAP "o......o"
    BITMAP "o.o...oo"
    BITMAP "oooooooo"

    ' Face 6 (index 5): four corners + two middle side pips.
    BITMAP "oooooooo"
    BITMAP "o......o"
    BITMAP "o.o...oo"
    BITMAP "o......o"
    BITMAP "o.o...oo"
    BITMAP "o......o"
    BITMAP "o.o...oo"
    BITMAP "oooooooo"
```

LISTING 13 card.bas — Fujitzee scorecard

```
' card.bas -- scorecard grid rendering, cursor, seat strip and standings
' overlay for the round 1..13 game screen. One player's full 15-category
' card is shown at a time (BACKTAB rows 1-8); a permanent 6-seat strip
' (row 9) shows who's who, and holding keypad ENTER swaps rows 1-8 for a
' name+total standings list.
'
' Layout (20x12 BACKTAB):
'   row0  viewed player's name, round
'   row1-6  ONE..SIX   | SET3,SET4,HOUS,SRUN,LRUN,CNT,FUJI (rows1-7)
'   row7    UP (computed) | FUJI (row7 of the right column)
'   row8    BON (computed) | TOT (computed)
'   row9    6-seat strip (alias letters, color-coded)
'   row10   5 dice + rolls-left
'   row11   server prompt + move timer
'
' Category placement:
'   left column,  rows 1-6: score indices 0-5  (ONE..SIX)
'   left column,  row 7:    UP   (computed upper total, index 6 on the wire)
'   left column,  row 8:    BON  (computed bonus, index 7 on the wire)
'   right column, rows 1-7: score indices 8-14 (SET3,SET4,HOUS,SRUN,LRUN,CNT,
FUJI)
'   right column, row 8:    TOT  (computed running grand total)

    CONST CARD_ROW0    = screenpos(0, 0)
    CONST CARD_LCOL_LABEL = 0
    CONST CARD_LCOL_VALUE = 5
    CONST CARD_RCOL_LABEL = 10
    CONST CARD_RCOL_VALUE = 15
    CONST SEAT_ROW     = 9

    ' Page background is blue (cls_blue in fujitzee.bas fills every CLS'd
    ' screen with this), so every color below pairs with BG_BLUE instead
    ' of BG_BLACK -- anything still paired with black would sit on an
    ' invisible matching square rather than actually showing a black box.
    CONST COL_TEXT     = FG_WHITE + BG_BLUE
    CONST COL_LABEL    = FG_TAN + BG_BLUE
    CONST COL_HILITE   = FG_YELLOW + BG_BLUE
    CONST COL_POTENTIAL = FG_GREEN + BG_BLUE
    CONST COL_DASH     = FG_BLACK + BG_BLUE
    CONST COL_CURSOR   = FG_BLACK + BG_YELLOW
    CONST COL_ACTIVE   = FG_BLACK + BG_YELLOW
    CONST COL_VIEWED   = FG_BLACK + BG_GREEN
    CONST COL_SEATEMPTY = FG_BLACK + BG_BLUE
    ' MODE 1's foreground palette is fixed at 8 colors (no orange) --
    ' orange only exists as one of the 16 background shades. So "orange
    ' text" for the round indicator is rendered as black-on-orange
    ' instead of a foreground color.
    CONST COL_ROUND    = FG_BLACK + BG_ORANGE

' ---------------------------------------------------------------------------
' player_color_tbl / player_color: a distinct FG color per seat (0-5),
' used everywhere a player's name is drawn (header, seat strip, standings,
' lobby, score reveal) so a given player reads consistently across every
' screen. FG_BLUE is deliberately excluded -- it would be invisible on
' the page's own blue background. Only 6 colors are available without
' reusing COL_TEXT/COL_LABEL/the status colors above, matching
' MAX_PLAYERS; seats beyond that (spectators, in the standings/lobby
' lists which show up to 8) fall back to plain COL_TEXT.
' ---------------------------------------------------------------------------
player_color_tbl: DATA FG_RED, FG_TAN, FG_DARKGREEN, FG_GREEN, FG_YELLOW,
FG_WHITE

DIM pc_idx, #pc_color
player_color: PROCEDURE
    IF pc_idx < 6 THEN
        #pc_color = player_color_tbl(pc_idx) + BG_BLUE
    ELSE
        #pc_color = COL_TEXT
    END IF
END

' ---------------------------------------------------------------------------
' card_init: draw the static label frame once (labels never change; only
' the values in the adjoining 3-char fields do). Callers CLS and call this
' on the transition into round 1..13; the value cells drawn each poll after
' this leave the labels alone.
' ---------------------------------------------------------------------------
card_init: PROCEDURE
    PRINT AT CARD_ROW0 + screenpos(0, 1) COLOR COL_LABEL, "ONE "
    PRINT AT CARD_ROW0 + screenpos(0, 2) COLOR COL_LABEL, "TWO "
    PRINT AT CARD_ROW0 + screenpos(0, 3) COLOR COL_LABEL, "THR "
    PRINT AT CARD_ROW0 + screenpos(0, 4) COLOR COL_LABEL, "FOU "
    PRINT AT CARD_ROW0 + screenpos(0, 5) COLOR COL_LABEL, "FIV "
    PRINT AT CARD_ROW0 + screenpos(0, 6) COLOR COL_LABEL, "SIX "
    PRINT AT CARD_ROW0 + screenpos(0, 7) COLOR COL_LABEL, "UP  "
    PRINT AT CARD_ROW0 + screenpos(0, 8) COLOR COL_LABEL, "BON "

    PRINT AT CARD_ROW0 + screenpos(10, 1) COLOR COL_LABEL, "SET3"
    PRINT AT CARD_ROW0 + screenpos(10, 2) COLOR COL_LABEL, "SET4"
    PRINT AT CARD_ROW0 + screenpos(10, 3) COLOR COL_LABEL, "HOUS"
    PRINT AT CARD_ROW0 + screenpos(10, 4) COLOR COL_LABEL, "SRUN"
    PRINT AT CARD_ROW0 + screenpos(10, 5) COLOR COL_LABEL, "LRUN"
    PRINT AT CARD_ROW0 + screenpos(10, 6) COLOR COL_LABEL, "CNT "
    PRINT AT CARD_ROW0 + screenpos(10, 7) COLOR COL_LABEL, "FUJI"
    PRINT AT CARD_ROW0 + screenpos(10, 8) COLOR COL_LABEL, "TOT "
END

' ---------------------------------------------------------------------------
' calc_running_total: sum every filled category (UP+BON+SET3..FUJI, unset
' treated as 0) for the player record at #ctr_addr into #ctr_total. The
```

```
' server only fills the wire's grand-total slot (score index 15) at game
' end, so the live TOT row and the standings overlay compute it themselves.
' ---------------------------------------------------------------------------
DIM #ctr_addr, #ctr_total, ctr_i, #ctr_v
calc_running_total: PROCEDURE
    #ctr_total = 0
    FOR ctr_i = 6 TO 14
        #ctr_v = score_at(#ctr_addr, ctr_i)
        IF #ctr_v <> SCORE_UNSET AND #ctr_v <> SCORE_VIEWING THEN #ctr_total =
#ctr_total + #ctr_v
    NEXT ctr_i
END

' ---------------------------------------------------------------------------
' draw_cat_cell: render one 3-char value field. Inputs:
'   dcc_row     BACKTAB row (1-8)
'   dcc_valcol  BACKTAB column of the value field's first cell
'   dcc_cat     score index 0-5 or 8-14 for a real category; 100=UP,
'               101=BON, 102=TOT for the three computed cells
'   #dcc_addr   viewed player's 42-byte record in FN_RX
'   dcc_own     1 if this is your own card and it's your turn (enables the
'               green "potential score" hint on unscored categories)
'   dcc_cursor  1 if this cell is the SCORE-mode cursor's current selection
' ---------------------------------------------------------------------------
DIM dcc_row, dcc_valcol, dcc_cat, #dcc_addr, dcc_own, dcc_cursor
DIM #dcc_v, #dcc_color, #dcc_pot
draw_cat_cell: PROCEDURE
    #dcc_color = COL_TEXT
    IF dcc_cursor THEN #dcc_color = COL_CURSOR

    IF dcc_cat = 100 OR dcc_cat = 101 THEN
        #dcc_v = score_at(#dcc_addr, 6 + (dcc_cat - 100))
        IF #dcc_v = SCORE_UNSET OR #dcc_v = SCORE_VIEWING THEN
            IF dcc_cursor = 0 THEN #dcc_color = COL_DASH
            PRINT AT screenpos(dcc_valcol, dcc_row) COLOR #dcc_color, "  -"
        ELSE
            PRINT AT screenpos(dcc_valcol, dcc_row) COLOR #dcc_color, <.3>#dcc_v
        END IF
    ELSEIF dcc_cat = 102 THEN
        #ctr_addr = #dcc_addr
        GOSUB calc_running_total
        PRINT AT screenpos(dcc_valcol, dcc_row) COLOR #dcc_color, <.3>#ctr_total
    ELSE
        #dcc_v = score_at(#dcc_addr, dcc_cat)
        IF #dcc_v = SCORE_UNSET OR #dcc_v = SCORE_VIEWING THEN
            #dcc_pot = 255
            IF dcc_own THEN #dcc_pot = valid_at(dcc_cat)
            IF #dcc_pot <> 255 THEN
                ' A potential of exactly 0 (category open but this roll
                ' doesn't fill it) is only shown on the row the cursor is
                ' currently sitting on -- matching the reference clients
                ' (gamelogic.c's drawTextAlt/drawBlank cursor-move
                ' handling), which hides the "0" everywhere else so an
                ' open board isn't a wall of zeros before you've decided.
                IF #dcc_pot > 0 THEN
                    IF dcc_cursor = 0 THEN #dcc_color = COL_POTENTIAL
                    PRINT AT screenpos(dcc_valcol, dcc_row) COLOR #dcc_color,
<.3>#dcc_pot
                ELSEIF dcc_cursor THEN
                    PRINT AT screenpos(dcc_valcol, dcc_row) COLOR #dcc_color, "
0"
                ELSE
                    PRINT AT screenpos(dcc_valcol, dcc_row) COLOR COL_TEXT, "
"
                END IF
            ELSE
                IF dcc_cursor = 0 THEN #dcc_color = COL_DASH
                PRINT AT screenpos(dcc_valcol, dcc_row) COLOR #dcc_color, "  -"
            END IF
        ELSE
            PRINT AT screenpos(dcc_valcol, dcc_row) COLOR #dcc_color, <.3>#dcc_v
        END IF
    END IF
END

' ---------------------------------------------------------------------------
' render_card: redraw every value cell of the viewed player's card.
' Globals expected: #rc_addr (player record), rc_own (1 if own card + your
' turn, enables potentials), rc_mode (0=DICE,1=SCORE), rc_selcat (score
' index the SCORE-mode cursor sits on; only consulted when rc_mode=1),
' rc_reveal (score index 0-14 to force-highlight regardless of rc_own/
' rc_mode -- used to flash another player's just-scored category; 255 for
' none, the normal case).
' Split into a left-column half and a right-column half with a WAIT between
' (and another before the header/seat strip) so a full repaint -- 16 value
' cells worth of PRINT -- can't tear a frame the way one unbroken burst
' covering half the visible screen could (see board.bas/5card.bas's own
' per-region WAIT breaks for the same reason: no page-flip on the STIC).
' ---------------------------------------------------------------------------
DIM #rc_addr, rc_own, rc_mode, rc_selcat, rc_reveal, rc_i, rc_cur
render_card: PROCEDURE
    FOR rc_i = 0 TO 7
        rc_cur = 0
        IF rc_mode = 1 AND rc_own AND rc_i < 6 AND rc_selcat = rc_i THEN rc_cur
= 1
        IF rc_reveal = rc_i THEN rc_cur = 1
        dcc_row = rc_i + 1 : dcc_valcol = CARD_LCOL_VALUE : #dcc_addr = #rc_addr
        dcc_own = rc_own : dcc_cursor = rc_cur
        IF rc_i < 6 THEN
            dcc_cat = rc_i
        ELSEIF rc_i = 6 THEN
            dcc_cat = 100
        ELSE
            dcc_cat = 101
        END IF
        GOSUB draw_cat_cell
```

```
    NEXT rc_i
    WAIT

    FOR rc_i = 0 TO 7
        rc_cur = 0
        IF rc_mode = 1 AND rc_own AND rc_i < 7 AND rc_selcat = rc_i + 8 THEN
rc_cur = 1
        IF rc_reveal = rc_i + 8 THEN rc_cur = 1
        dcc_row = rc_i + 1 : dcc_valcol = CARD_RCOL_VALUE : #dcc_addr = #rc_addr
        dcc_own = rc_own : dcc_cursor = rc_cur
        IF rc_i < 7 THEN dcc_cat = rc_i + 8 ELSE dcc_cat = 102
        GOSUB draw_cat_cell
    NEXT rc_i
END

' ------------------------------------------------------------------------
' render_seatstrip: one letter per seated player (up to 6), spaced 3
' columns apart, color-coded: active player highlighted, the currently
' viewed player (if different) in a second color, everyone else plain
' white, empty seats a dim dot.
'
' This is the player's *first* name character (PL_NAME+0), not the
' server's PL_ALIAS-indexed "unique highlight character" the field
' technically exists for. The alias-indirection version (PEEK an alias
' index byte, then PEEK name[that index]) was extensively verified
' correct -- traced the exact compiled instructions, cross-checked real
' alias/name bytes pulled live from both the dev and production servers
' (every case seen had alias=0, i.e. this simpler version is what it
' would have shown anyway) -- and still rendered '?' for every seat in
' practice. Simplifying to a single direct PEEK removes the double
' indirection entirely rather than continuing to chase a mismatch between
' verified-correct analysis and observed behavior.
' ------------------------------------------------------------------------
DIM ss_i, ss_addr, #ss_ch, #ss_color
render_seatstrip: PROCEDURE
    FOR ss_i = 0 TO 5
        IF ss_i < state_playercount THEN
            ss_addr = player_addr(ss_i)
            #ss_ch = (PEEK(ss_addr + PL_NAME) AND 255)
            IF #ss_ch >= 97 AND #ss_ch <= 122 THEN #ss_ch = #ss_ch - 32
            IF #ss_ch < 32 OR #ss_ch > 95 THEN #ss_ch = 63 ' '?' fallback
            ' Active/viewed status wins the BG (black-on-yellow/green, as
            ' before -- needs to stay readable and unambiguous regardless
            ' of which player it is); otherwise the letter uses that
            ' player's own color, so it reads consistently with their
            ' name everywhere else even while just sitting in the strip.
            IF ss_i = active_player THEN
                #ss_color = COL_ACTIVE
            ELSEIF ss_i = view_idx THEN
                #ss_color = COL_VIEWED
            ELSE
                pc_idx = ss_i
                GOSUB player_color
                #ss_color = #pc_color
            END IF
        ELSE
            #ss_ch = 46 ' '.'
            #ss_color = COL_SEATEMPTY
        END IF
        #BACKTAB(screenpos(ss_i * 3, SEAT_ROW)) = (#ss_ch - 32) * 8 + #ss_color
    NEXT ss_i
END

' ------------------------------------------------------------------------
' render_player_list: name + running total for up to 8 seated players,
' starting at BACKTAB row prl_top. Shared by show_standings (the keypad-
' ENTER overlay during play) and render_gameover (the round-99 screen) --
' both just want "who's got how many points," and calc_running_total's sum
' of every filled category equals the server's own final total once every
' category is filled, so the same running-total math is correct in both
' places without needing to trust the wire's game-over-only total field.
' ------------------------------------------------------------------------
DIM prl_top, prl_i, prl_addr, #prl_pc
render_player_list: PROCEDURE
    #prl_pc = state_playercount
    IF #prl_pc > 8 THEN #prl_pc = 8
    FOR prl_i = 0 TO #prl_pc - 1
        prl_addr = player_addr(prl_i)
        pc_idx = prl_i
        GOSUB player_color
        #df_src = prl_addr + PL_NAME : df_pos = screenpos(0, prl_top + prl_i) :
df_len = 9 : #df_color = #pc_color
        GOSUB draw_field
        #ctr_addr = prl_addr
        GOSUB calc_running_total
        PRINT AT screenpos(15, prl_top + prl_i) COLOR COL_TEXT, <.4>#ctr_total
    NEXT prl_i
END

' ------------------------------------------------------------------------
' show_standings: keypad-ENTER overlay. Blanks rows 1-8 and lists seated
' players' names and running totals instead of the card. Blocks on its own
' input loop (like ingame_menu) rather than folding into the poll loop --
' holding ENTER for a couple of frames of paused polling is harmless, and
' this keeps render_card's cell layout untouched by an overlay that only
' needs to exist while the button is held.
' ------------------------------------------------------------------------
DIM sd_i
show_standings: PROCEDURE
    FOR sd_i = 1 TO 8
        PRINT AT screenpos(0, sd_i) COLOR COL_TEXT, "                    "
    NEXT sd_i
    prl_top = 1
    GOSUB render_player_list

sd_wait:
```

```
    WAIT
    IF CONT1.KEY = 11 THEN GOTO sd_wait
END
```

LISTING 14 sound.bas — Fujitzee effects

```
' sound.bas -- sound effects, adapted from fujinet-battleship/intv/sound.bas.
' The transport primitive (play_tone) and the UI-generic effects are kept
' verbatim; the game-specific effects are replaced with Fujitzee's own set
' (roll/hold/score/fujitzee in place of Battleship's place/attack/hit/sink).
'
' All effects play on PSG channel 0 -- turn-based game, non-overlapping UI
' events, no need for polyphony (and SOUND's channel argument must be a
' compile-time constant anyway, which rules out a generic multi-channel
' helper).
'
' SOUND's frequency argument is a 12-bit PSG divisor, computed as
' round(3579545/32/hz) for NTSC. These are precomputed rather than divided
' at runtime: 3579545/32 alone is ~111861, which overflows IntyBASIC's
' 16-bit variables (max 65535), so it only works as a compile-time-folded
' literal division -- not as CONST-divided-by-a-runtime-variable. Every
' effect here uses a fixed, known-in-advance frequency, so precomputing
' sidesteps the overflow entirely.
'   50Hz->2237  60Hz->1864  70Hz->1598  80Hz->1398  100Hz->1119
'  150Hz->746  200Hz->559   300Hz->373  311Hz->360   330Hz->339
'  340Hz->329  350Hz->320   392Hz->285  415Hz->270    430Hz->260
'  500Hz->224  600Hz->187   700Hz->160  800Hz->140
    CONST SND_VOL = 12

    DIM #snd_val, snd_gate, snd_post, snd_i

' ---------------------------------------------------------------------------
' play_tone: one square-wave note on channel 0. #snd_val = precomputed PSG
' divisor (see table above), snd_gate = frames the tone sounds, snd_post =
' frames of silence after.
' ---------------------------------------------------------------------------
play_tone: PROCEDURE
    SOUND 0, #snd_val, SND_VOL
    FOR snd_i = 1 TO snd_gate
        WAIT
    NEXT snd_i
    SOUND 0, 0, 0
    FOR snd_i = 1 TO snd_post
        WAIT
    NEXT snd_i
END

' sound_join: sit down at a table (3-tone rising figure: 430,340,500 Hz).
sound_join: PROCEDURE
    #snd_val = 260 : snd_gate = 5 : snd_post = 8 : GOSUB play_tone
    #snd_val = 329 : snd_gate = 5 : snd_post = 0 : GOSUB play_tone
    #snd_val = 224 : snd_gate = 5 : snd_post = 0 : GOSUB play_tone
END

' sound_myturn: it's your turn to roll (double beep, 430,430 Hz).
sound_myturn: PROCEDURE
    #snd_val = 260 : snd_gate = 4 : snd_post = 2 : GOSUB play_tone
    #snd_val = 260 : snd_gate = 4 : snd_post = 2 : GOSUB play_tone
END

' sound_gamedone: game over, win fanfare (4-tone rising: 311,330,392,415 Hz).
sound_gamedone: PROCEDURE
    #snd_val = 360 : snd_gate = 10 : snd_post = 0 : GOSUB play_tone
    #snd_val = 339 : snd_gate = 20 : snd_post = 0 : GOSUB play_tone
    #snd_val = 285 : snd_gate = 10 : snd_post = 0 : GOSUB play_tone
    #snd_val = 270 : snd_gate = 20 : snd_post = 0 : GOSUB play_tone
END

' sound_player_join: another player sits down mid-lobby (5-tone rising
' sweep: 50,60,70,80 Hz).
sound_player_join: PROCEDURE
    #snd_val = 2237 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
    #snd_val = 1864 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
    #snd_val = 1598 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
    #snd_val = 1398 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
END

' sound_player_left: a player leaves mid-game (falling sweep, mirror of join).
sound_player_left: PROCEDURE
    #snd_val = 1398 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
    #snd_val = 1598 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
    #snd_val = 1864 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
    #snd_val = 2237 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
END

' sound_select: a menu/table/ready choice was confirmed (2-tone rising
' chirp: 300,350 Hz).
sound_select: PROCEDURE
    #snd_val = 373 : snd_gate = 3 : snd_post = 1 : GOSUB play_tone
    #snd_val = 320 : snd_gate = 3 : snd_post = 0 : GOSUB play_tone
END

' sound_cursor: cursor moved one position (300 Hz).
sound_cursor: PROCEDURE
```

```
    #snd_val = 373 : snd_gate = 2 : snd_post = 0 : GOSUB play_tone
END

' sound_tick: countdown clock tick during your turn (short 300 Hz click).
sound_tick: PROCEDURE
    #snd_val = 373 : snd_gate = 1 : snd_post = 0 : GOSUB play_tone
END

' sound_hold: a die's re-roll flag was toggled (short click, 600 Hz).
sound_hold: PROCEDURE
    #snd_val = 187 : snd_gate = 2 : snd_post = 0 : GOSUB play_tone
END

' sound_roll: the dice were rolled (rattling triplet: 700,600,700 Hz).
sound_roll: PROCEDURE
    #snd_val = 160 : snd_gate = 2 : snd_post = 1 : GOSUB play_tone
    #snd_val = 187 : snd_gate = 2 : snd_post = 1 : GOSUB play_tone
    #snd_val = 160 : snd_gate = 2 : snd_post = 4 : GOSUB play_tone
END

' sound_score: a category was scored (rising chirp, 350,500 Hz).
sound_score: PROCEDURE
    #snd_val = 320 : snd_gate = 3 : snd_post = 0 : GOSUB play_tone
    #snd_val = 224 : snd_gate = 4 : snd_post = 2 : GOSUB play_tone
END

' sound_fujitzee: five of a kind scored (5-tone rising fanfare, brighter and
' longer than sound_gamedone -- the rare, celebrated roll).
sound_fujitzee: PROCEDURE
    #snd_val = 373 : snd_gate = 6 : snd_post = 0 : GOSUB play_tone
    #snd_val = 320 : snd_gate = 6 : snd_post = 0 : GOSUB play_tone
    #snd_val = 285 : snd_gate = 6 : snd_post = 0 : GOSUB play_tone
    #snd_val = 260 : snd_gate = 6 : snd_post = 0 : GOSUB play_tone
    #snd_val = 224 : snd_gate = 16 : snd_post = 0 : GOSUB play_tone
END

' sound_invalid: rejected input -- all-zero re-roll mask, out-of-turn action,
' unselectable category (single low buzz, 100 Hz, held).
sound_invalid: PROCEDURE
    #snd_val = 1119 : snd_gate = 10 : snd_post = 5 : GOSUB play_tone
END

```

APPENDIX D

NETCAT

The traditional closing program of every FujiNet programmer's guide: a terminal. Type a line with the disc, ENTER sends it, and whatever comes back scrolls by. It compiles, assembles, and defaults to an echo server so it demonstrates itself.

LISTING 15 netcat.bas — a line-mode network terminal

```
' netcat.bas -- a line-mode network terminal in IntyBASIC. Opens a TCP
' connection through FujiNet's Network device, shows everything the far
' end sends on rows 0-9 of the screen, and lets you compose a line with
' the disc (the same letter-picker the games use for name entry) and send
' it with ENTER. Default target is tcpbin.com's echo service, so what you
' send comes straight back -- a self-test needing no server of your own.
'
' Controls:  disc up/down    cycle the character under the cursor
'            disc left/right move the cursor
'            ENTER           send the line (plus CR LF)
'            CLEAR           erase the line
'
' Build:  intybasic netcat.bas netcat.asm && as1600 -o netcat netcat.asm
    GOTO main

    INCLUDE "fujinet.bas"

    CONST COL_WHITE  = 7
    CONST COL_YELLOW = 6
    CONST TERM_CELLS = 200      ' rows 0-9 are the terminal
    CONST EDIT_ROW   = 220      ' row 11 is the composer
    CONST EDIT_LEN   = 18

' The devicespec, as ASCII DATA (20 bytes): "N:TCP://TCPBIN.COM:4242/"
lit_spec:
    DATA 78,58,84,67,80,58,47,47,84,67,80,66,73,78
    DATA 46,67,79,77,58,52,50,52,50,47
    CONST LEN_SPEC = 24

    DIM term_pos, nc_i, nc_c, nc_cr
    DIM ed_cur, ed_len, ed_i, ed_c
    DIM ed_buf(18)
    DIM inp_lock, #nc_color

'
--------------------------------------------------------------------------
' term_putc: draw ASCII nc_c at the terminal cursor, handling CR/LF and
' wrap-around. GROM cards 0-94 cover ASCII 32-126 directly in MODE 0.
'
--------------------------------------------------------------------------
term_putc: PROCEDURE
    IF nc_c = 13 THEN nc_cr = 1 : GOSUB term_newline : RETURN
    IF nc_c = 10 THEN
        ' collapse the LF of a CR LF pair; a bare LF is a newline
        IF nc_cr = 0 THEN GOSUB term_newline
        nc_cr = 0
        RETURN
    END IF
    nc_cr = 0
    IF nc_c < 32 OR nc_c > 126 THEN RETURN
    #BACKTAB(term_pos) = (nc_c - 32) * 8 + COL_WHITE
    term_pos = term_pos + 1
    IF term_pos >= TERM_CELLS THEN GOSUB term_home
END

term_newline: PROCEDURE
    term_pos = (term_pos / 20) * 20 + 20
    IF term_pos >= TERM_CELLS THEN GOSUB term_home
END

' Wrap back to the top and blank the first row -- a crude circular
' terminal, but 4K of scroll code has no place in an example program.
```

```
term_home: PROCEDURE
    term_pos = 0
    FOR nc_i = 0 TO 19
        #BACKTAB(nc_i) = 0
    NEXT nc_i
END

'
--------------------------------------------------------------------------------
' ed_draw: paint the composer row: 18 character cells + a send arrow.
'
--------------------------------------------------------------------------------
ed_draw: PROCEDURE
    FOR ed_i = 0 TO EDIT_LEN - 1
        #nc_color = COL_WHITE
        IF ed_i = ed_cur THEN #nc_color = COL_YELLOW
        ed_c = ed_buf(ed_i)
        IF ed_c = 32 THEN ed_c = 95   ' show blanks as underscores
        #BACKTAB(EDIT_ROW + ed_i) = (ed_c - 32) * 8 + #nc_color
    NEXT ed_i
END

main:
    MODE 0, 0, 0, 0, 0 : WAIT
    CLS
    PRINT AT EDIT_ROW COLOR COL_WHITE, "                    "
    PRINT AT 200 COLOR COL_YELLOW, "FUJINET NETCAT      "

    GOSUB fn_wait_mailbox
    IF fn_ok = 0 THEN
        PRINT AT 0 COLOR COL_WHITE, "NO CARTRIDGE MAILBOX"
        GOTO halt
    END IF

    ' Open the connection: devicespec into FN_TX, then OPEN in
    ' read-write mode with no translation (we handle CR LF ourselves).
    #fn_txlen = 0
    #fn_src = VARPTR lit_spec(0) : fn_len = LEN_SPEC : GOSUB fn_putstr
    mb_dev = NET_DEVICEID
    mb_cmd = NETCMD_OPEN
    mb_nparam = 2
    pm_i = 0 : pm_size = 1 : #pm_val = OPEN_MODE_RW : GOSUB fn_param
    pm_i = 1 : pm_size = 1 : #pm_val = OPEN_TRANS_NONE : GOSUB fn_param
    GOSUB fn_transact
    IF fn_ok = 0 THEN
        PRINT AT 0 COLOR COL_WHITE, "CONNECT FAILED      "
        GOTO halt
    END IF

    term_pos = 0
    nc_cr = 0
    ed_cur = 0
    inp_lock = 0
    FOR ed_i = 0 TO EDIT_LEN - 1
        ed_buf(ed_i) = 32
    NEXT ed_i
    GOSUB ed_draw

loop:
    WAIT

    ' --- receive: anything waiting? read up to 64 bytes and print it ---
    GOSUB net_status
    IF fn_ok THEN
        IF #net_avail > 0 THEN
            #net_readlen = #net_avail
            IF #net_readlen > 64 THEN #net_readlen = 64
            GOSUB net_read
            IF fn_ok THEN
                FOR nc_i = 0 TO #net_gotlen - 1
                    nc_c = PEEK(FN_RX + nc_i) AND 255
                    GOSUB term_putc
                NEXT nc_i
            END IF
        END IF
    END IF

    ' --- compose ---
    IF inp_lock > 0 THEN inp_lock = inp_lock - 1 : GOTO loop

    IF CONT1.RIGHT THEN
        ed_cur = ed_cur + 1
        IF ed_cur >= EDIT_LEN THEN ed_cur = 0
        inp_lock = 8 : GOSUB ed_draw
        GOTO loop
    END IF
    IF CONT1.LEFT THEN
        IF ed_cur = 0 THEN ed_cur = EDIT_LEN
        ed_cur = ed_cur - 1
        inp_lock = 8 : GOSUB ed_draw
        GOTO loop
    END IF
    IF CONT1.UP THEN
        ed_c = ed_buf(ed_cur) + 1
        IF ed_c > 126 THEN ed_c = 32
        ed_buf(ed_cur) = ed_c
        inp_lock = 6 : GOSUB ed_draw
        GOTO loop
```

```
    END IF
    IF CONT1.DOWN THEN
        ed_c = ed_buf(ed_cur) - 1
        IF ed_c < 32 THEN ed_c = 126
        ed_buf(ed_cur) = ed_c
        inp_lock = 6 : GOSUB ed_draw
        GOTO loop
    END IF

    IF CONT1.KEY = 10 THEN
        ' CLEAR: wipe the line
        FOR ed_i = 0 TO EDIT_LEN - 1
            ed_buf(ed_i) = 32
        NEXT ed_i
        ed_cur = 0
        inp_lock = 10 : GOSUB ed_draw
        GOTO loop
    END IF

    IF CONT1.KEY = 11 THEN
        ' ENTER: trim trailing blanks, stage line + CR LF in FN_TX, send.
        ed_len = EDIT_LEN
        WHILE ed_len > 0 AND ed_buf(ed_len - 1) = 32
            ed_len = ed_len - 1
        WEND
        FOR ed_i = 0 TO ed_len - 1
            POKE (FN_TX + ed_i), ed_buf(ed_i)
        NEXT ed_i
        POKE (FN_TX + ed_len), 13
        POKE (FN_TX + ed_len + 1), 10
        fn_len = ed_len + 2
        GOSUB net_write
        FOR ed_i = 0 TO EDIT_LEN - 1
            ed_buf(ed_i) = 32
        NEXT ed_i
        ed_cur = 0
        inp_lock = 10 : GOSUB ed_draw
    END IF
    GOTO loop

halt:
    WAIT
    GOTO halt
```

Now go put an Intellivision on the Internet.